



CMPT 413/713: Natural Language Processing

Sequence to Sequence Models (Seq2Seq)

Fall 2021

2021-10-21

Adapted from slides from Danqi Chen and Karthik Narasimhan
(with some content from slides from Abigail See, Graham Neubig)

Sequence Models

Overview

- Review of Vanilla RNN
- Issues with Gradient Flows
- LSTMs and GRUs
- Applications
- Variants: Stacked RNNs, Bidirectional RNNs

Modeling Sequence

- Ideal for processing sequential data containing possibly long-term dependencies.
- Various implementations (e.g. simple RNN, LSTM, GRU) expose the same API

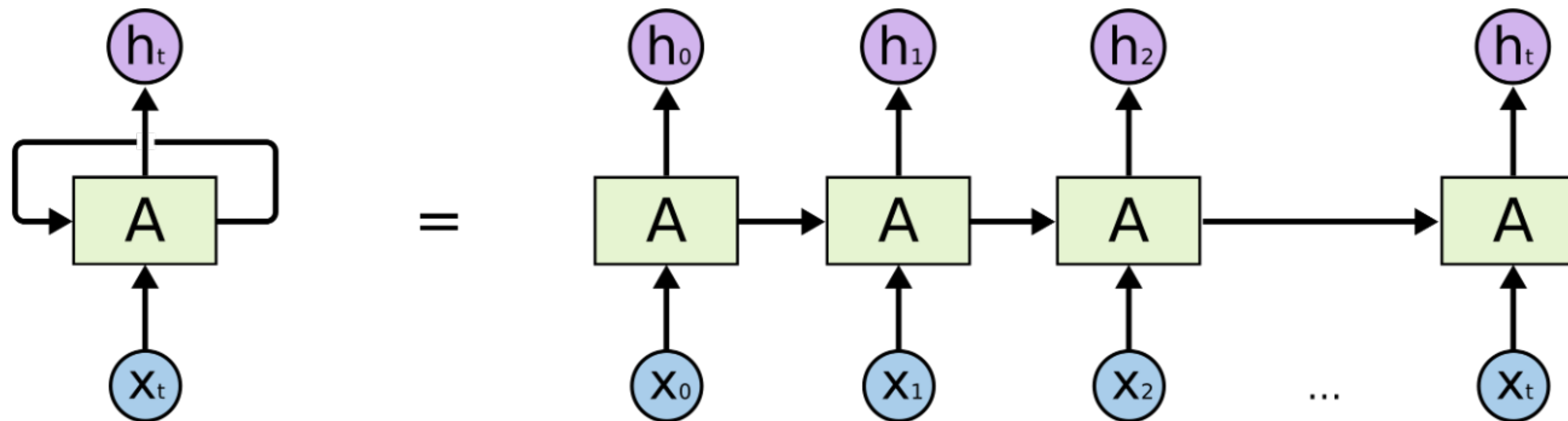
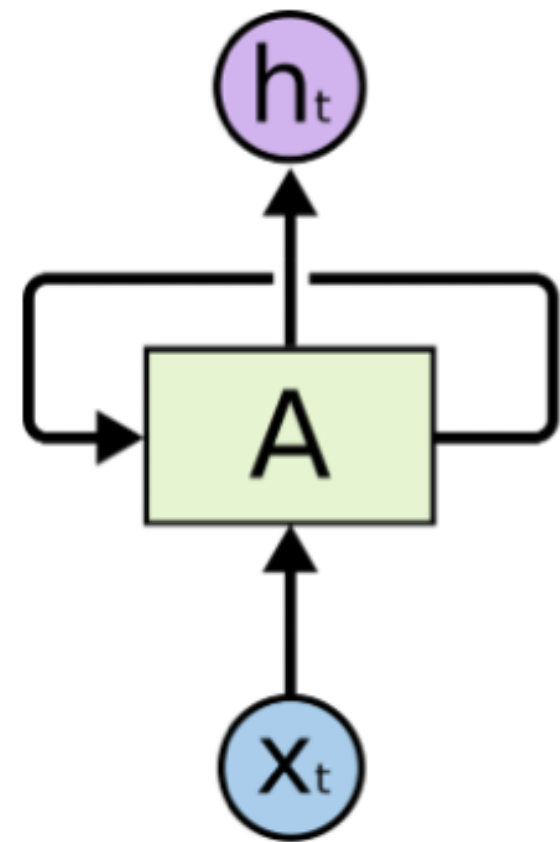


Image Credit: Christopher Olah

<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Simple (vanilla) RNNs



$\mathbf{h}_0 \in \mathbb{R}^d$ is an initial state

function with weights W

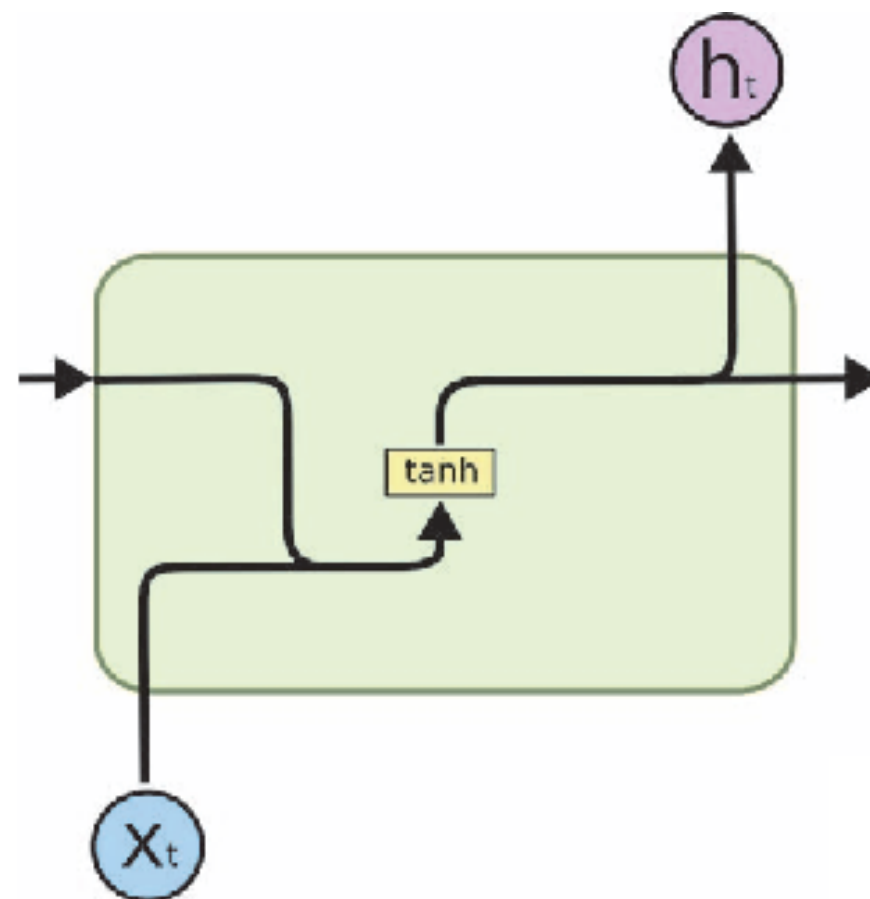
$$\mathbf{h}_t = f_W(\mathbf{h}_{t-1}, \mathbf{x}_t) \in \mathbb{R}^d$$

new state old state input at time t

$\mathbf{h}_t$: hidden states which store information from $\mathbf{x}_1$ to $\mathbf{x}_t$

Output label for each time step: Denote $\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{W}_o \mathbf{h}_t)$, $\mathbf{W}_o \in \mathbb{R}^{|V| \times d}$

Simple (vanilla) RNNs:



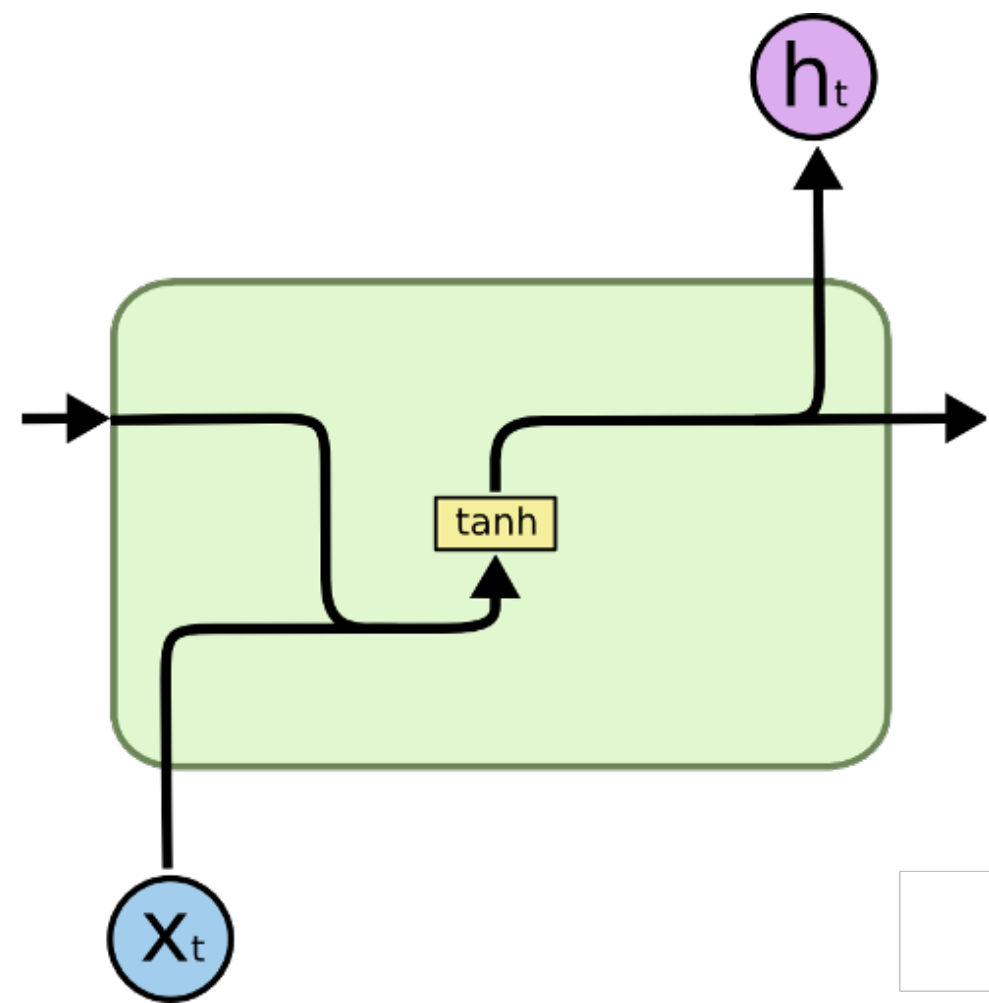
$$\mathbf{h}_t = g(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{U} \mathbf{x}_t + \mathbf{b}) \in \mathbb{R}^d$$

g : nonlinearity (e.g. tanh),

$$\mathbf{W}_{hh} \in \mathbb{R}^{d \times d}, \mathbf{U} \in \mathbb{R}^{d \times d_{in}}, \mathbf{b} \in \mathbb{R}^d$$

Simple RNN vs GRU vs LSTM

$$\mathbf{h}_t = g(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t + \mathbf{b})$$



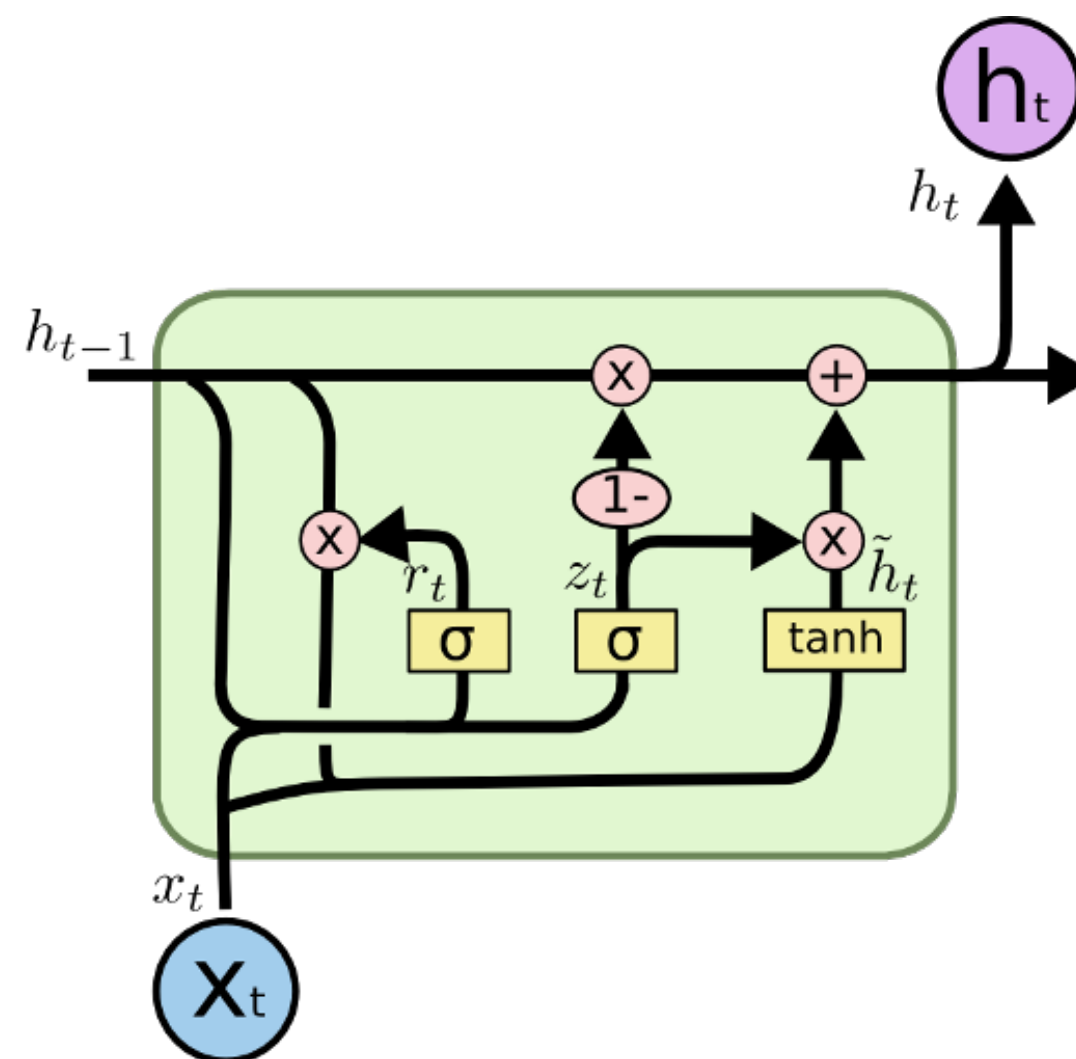
Simple RNN

$$\mathbf{r}_t = \sigma(\mathbf{W}^r\mathbf{h}_{t-1} + \mathbf{U}^r\mathbf{x}_t + \mathbf{b}^r)$$

$$\mathbf{z}_t = \sigma(\mathbf{W}^z\mathbf{h}_{t-1} + \mathbf{U}^z\mathbf{x}_t + \mathbf{b}^z)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{U}\mathbf{x}_t + \mathbf{b})$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$



GRU

$$\mathbf{i}_t = \sigma(\mathbf{W}^i\mathbf{h}_{t-1} + \mathbf{U}^i\mathbf{x}_t + \mathbf{b}^i)$$

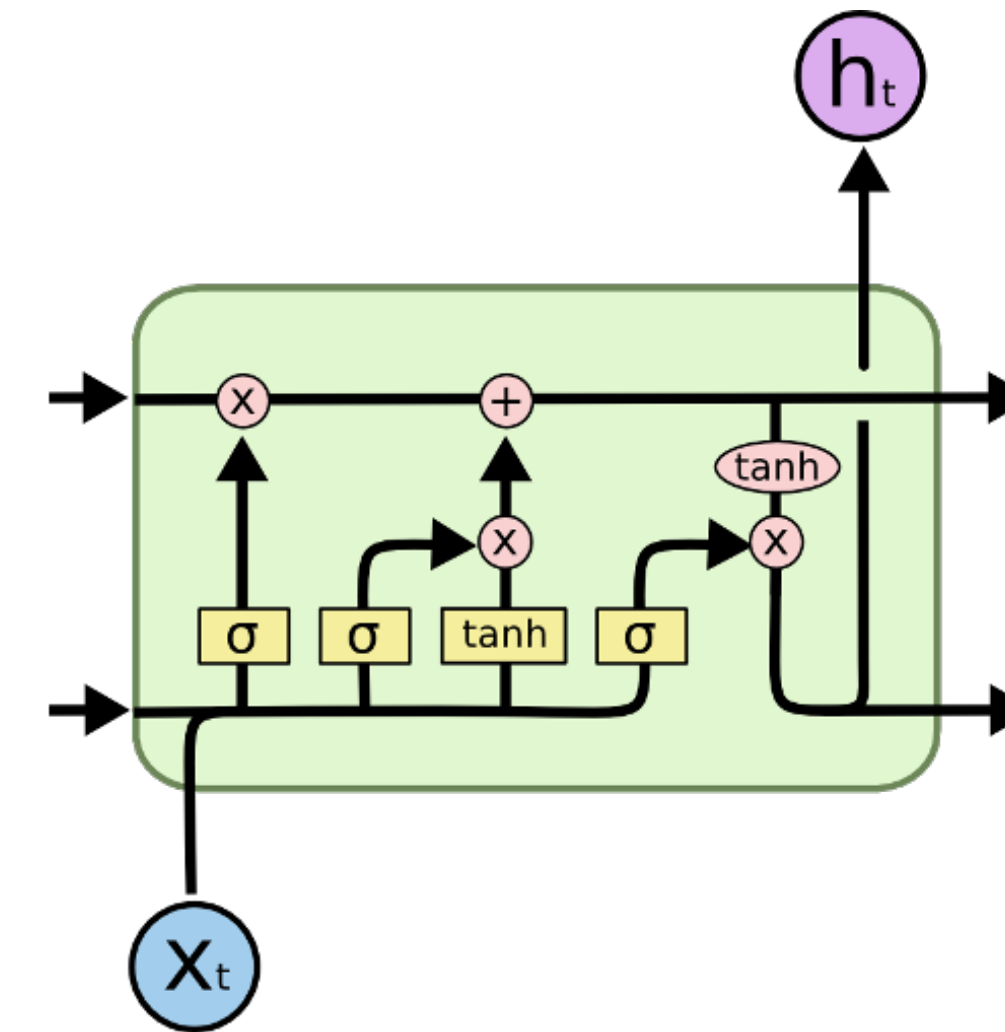
$$\mathbf{f}_t = \sigma(\mathbf{W}^f\mathbf{h}_{t-1} + \mathbf{U}^f\mathbf{x}_t + \mathbf{b}^f)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}^o\mathbf{h}_{t-1} + \mathbf{U}^o\mathbf{x}_t + \mathbf{b}^o)$$

$$\mathbf{g}_t = \tanh(\mathbf{W}^g\mathbf{h}_{t-1} + \mathbf{U}^g\mathbf{x}_t + \mathbf{b}^g)$$

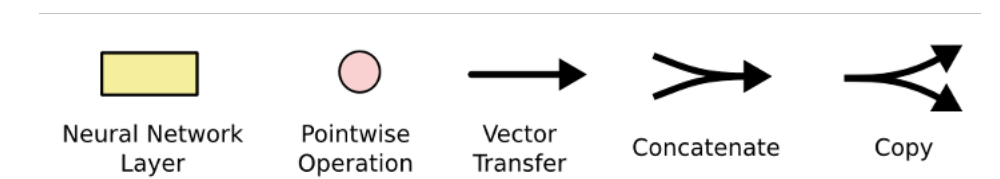
$$\mathbf{c}_t = \mathbf{c}_{t-1} \odot \mathbf{f}_t + \mathbf{g}_t \odot \mathbf{i}_t$$

$$\mathbf{h}_t = \tanh(\mathbf{c}_t) \odot \mathbf{o}_t$$



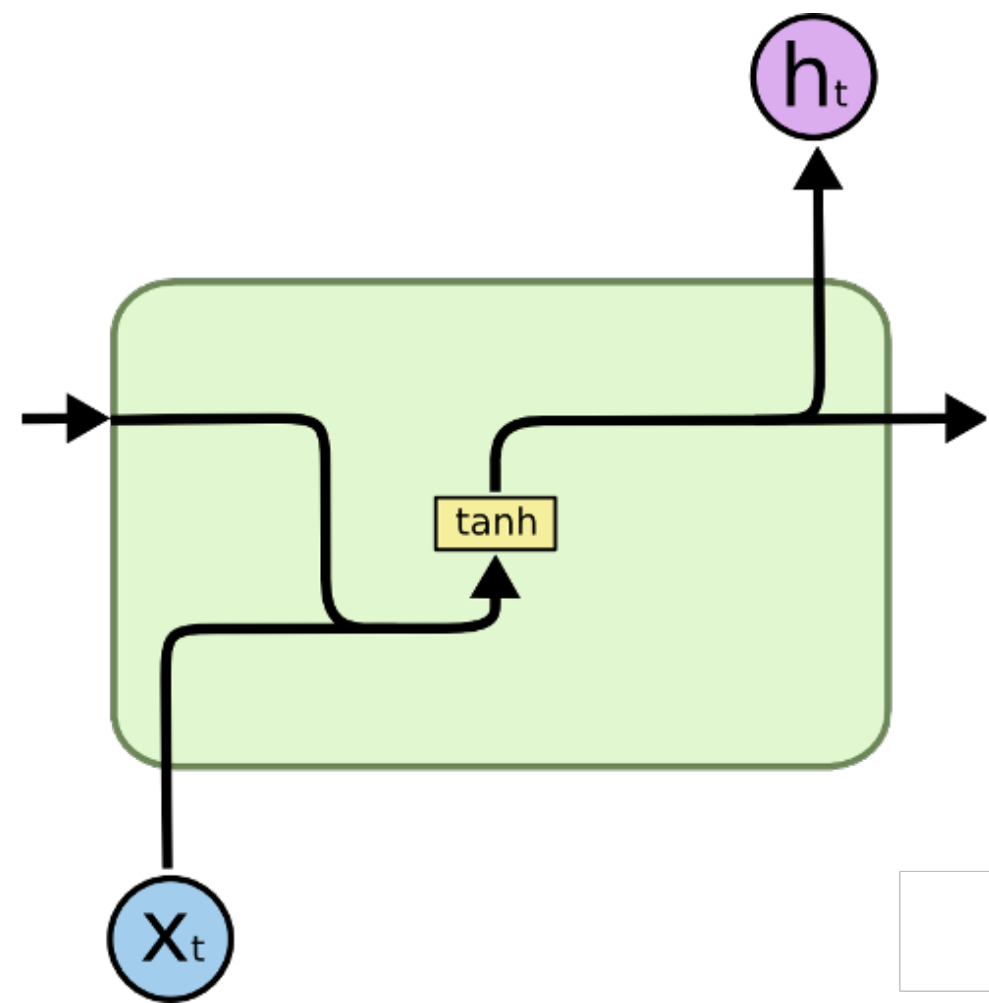
LSTM

<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>



Simple RNN vs GRU vs LSTM

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

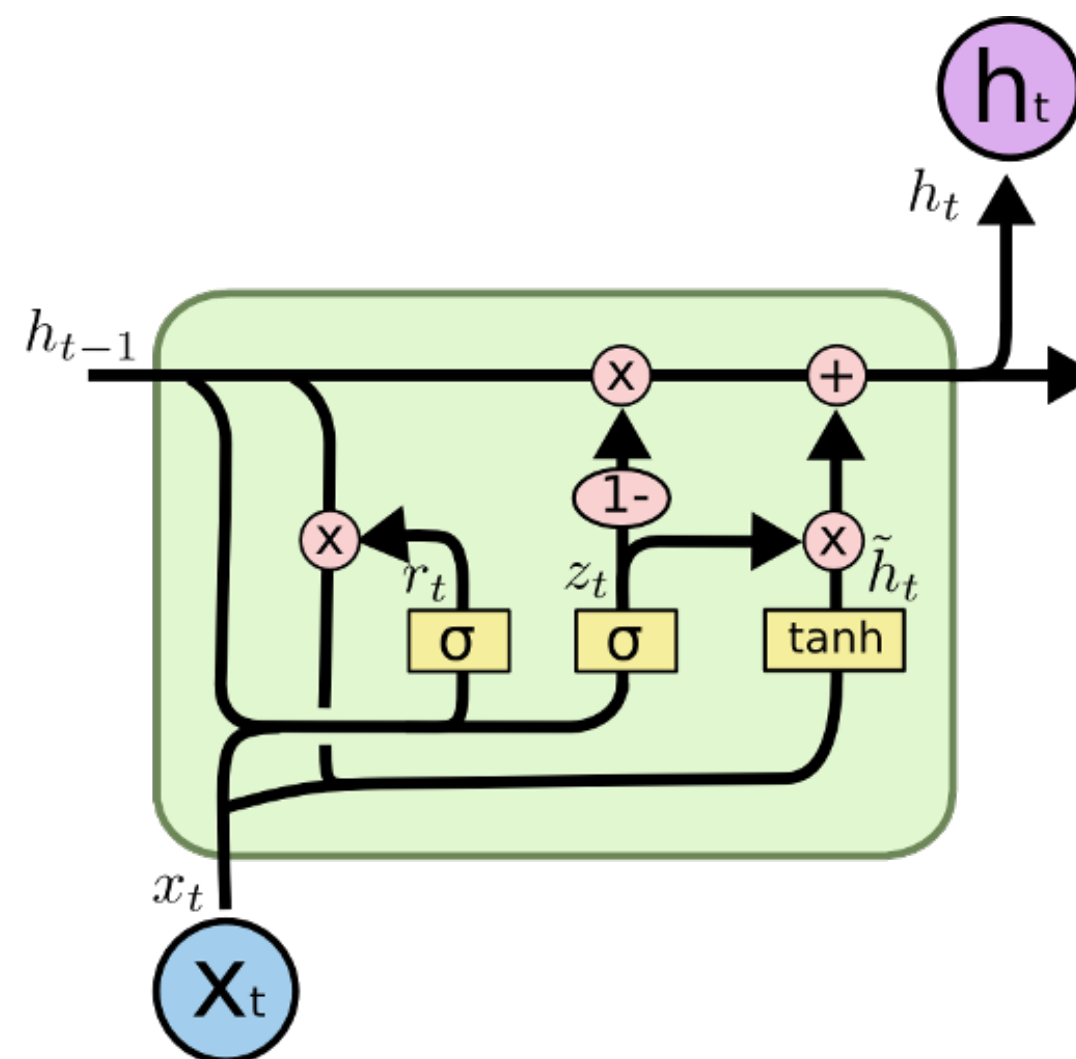


Simple RNN

$$\begin{pmatrix} r \\ z \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \end{pmatrix} W_1 \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$\tilde{h}_t = \tanh \left(W_2 \begin{pmatrix} r \odot h_{t-1} \\ x_t \end{pmatrix} \right)$$

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$$

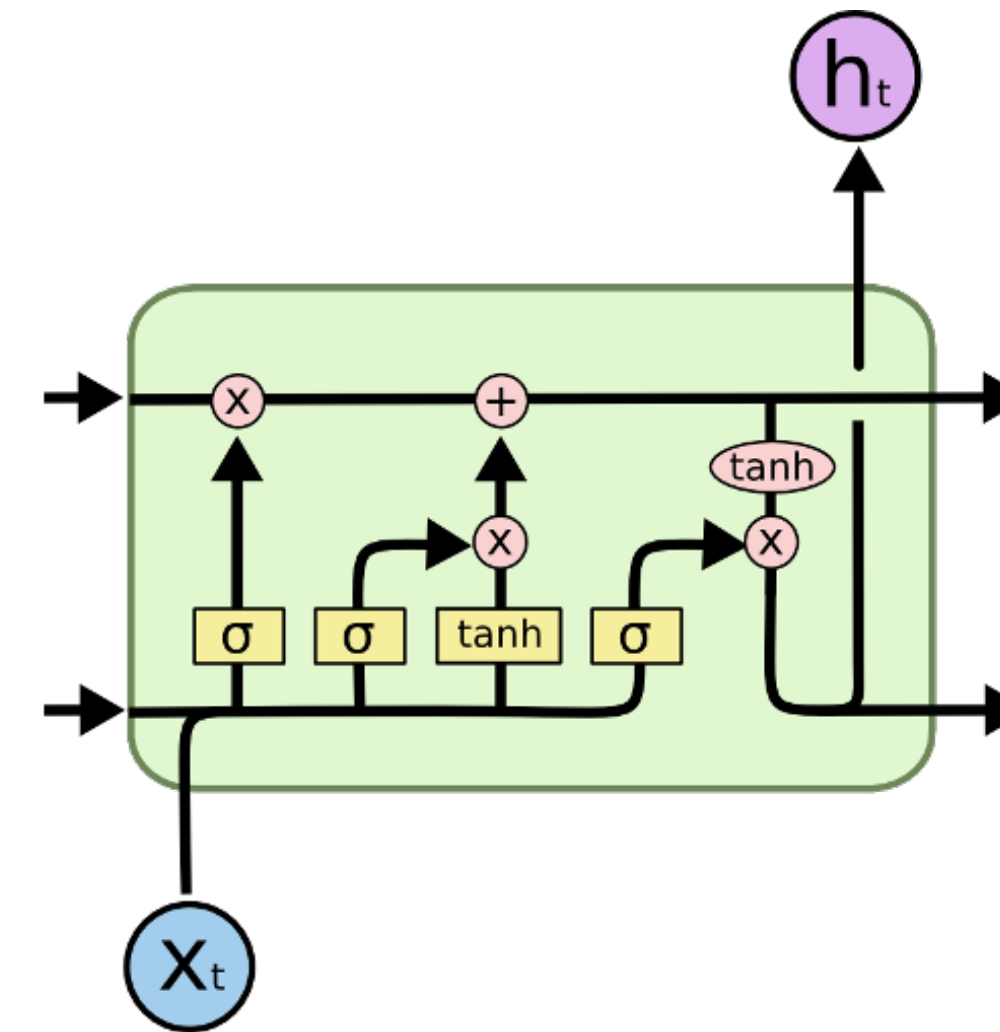


GRU

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

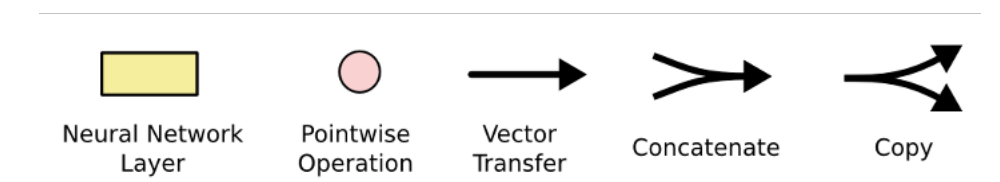
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

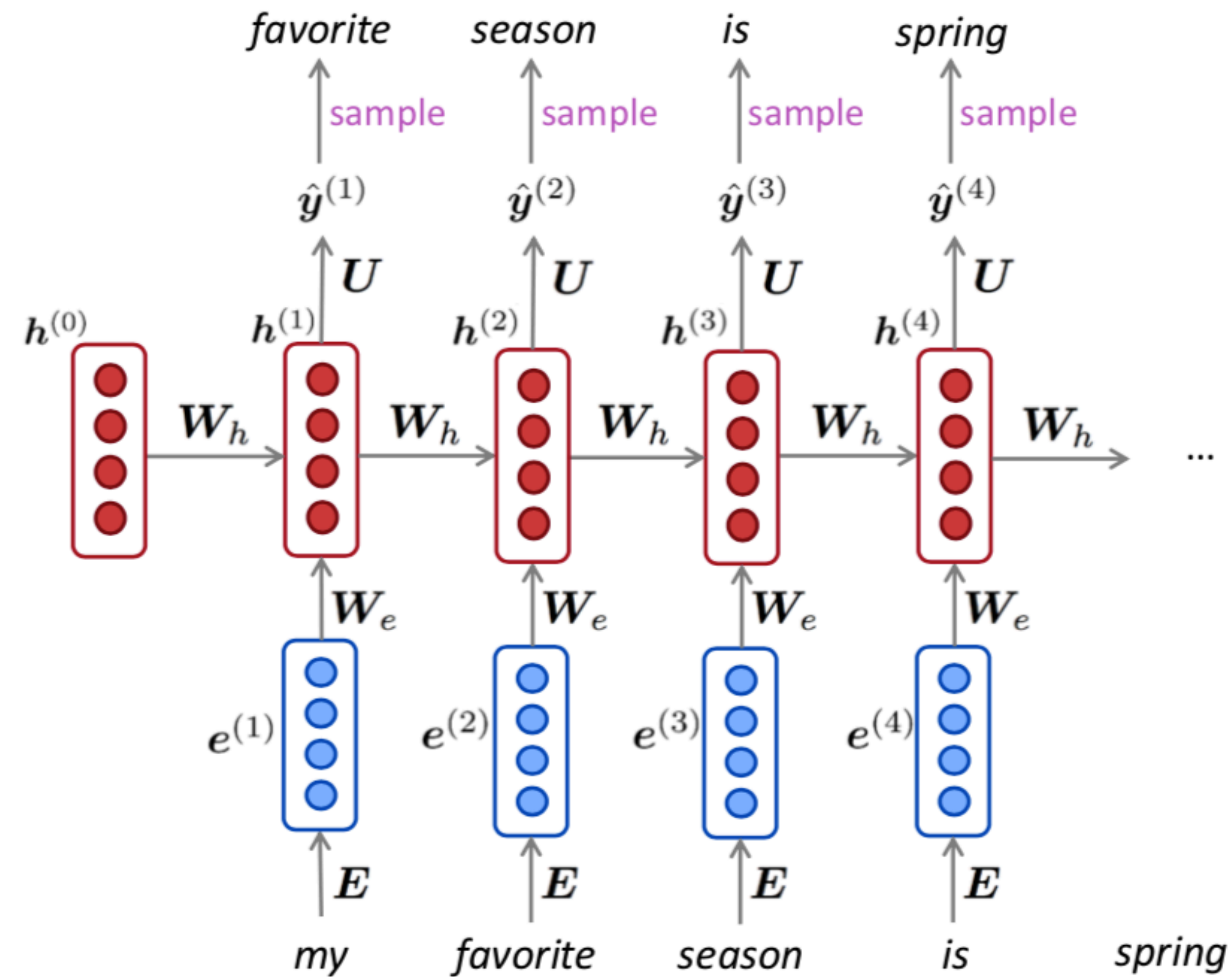


LSTM

<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>



Application: Text Generation



You can generate text by **repeated sampling**.
Sampled output is next step's input.

Fun with RNNs

Obama speeches

Good afternoon. God bless you.

The United States will step up to the cost of a new challenges of the American people that will share the fact that we created the problem. They were attacked and so that they have to say that all the task of the final days of war that I will not be able to get this done. The promise of the men and women who were still going to take out the fact that the American people have fought to make sure that they have to be able to protect our part. It was a chance to stand together to completely look for the commitment to borrow from the American people. And the fact is the men and women in uniform and the millions of our country with the law system that we should be a strong stretchs of the forces that we can afford to increase our spirit of the American people and the leadership of our country who are on the Internet of American lives.

Thank you very much. God bless you, and God bless the United States of America.

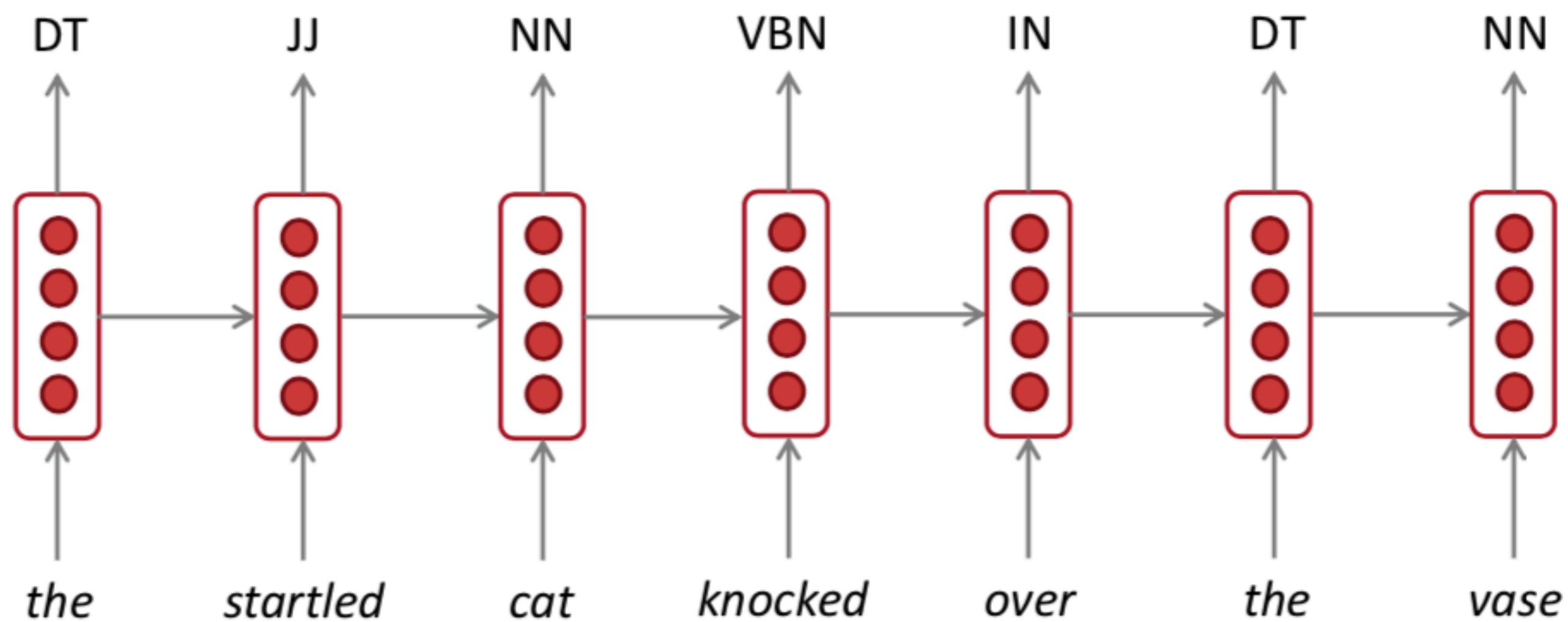
Latex generation

```
\begin{proof}
We may assume that  $\mathcal{I}$  is an abelian sheaf on  $\mathcal{C}$ .
\item Given a morphism  $\Delta : \mathcal{F} \rightarrow \mathcal{I}$ 
is an injective and let  $\mathfrak{q}$  be an abelian sheaf on  $X$ .
Let  $\mathcal{F}$  be a fibered complex. Let  $\mathcal{F}$  be a category.
\begin{enumerate}
\item \hyperref[setain-construction-phantom]{Lemma}
\label{lemma-characterize-quasi-finite}
Let  $\mathcal{F}$  be an abelian quasi-coherent sheaf on  $\mathcal{C}$ .
Let  $\mathcal{F}$  be a coherent  $\mathcal{O}_X$ -module. Then
 $\mathcal{F}$  is an abelian catenary over  $\mathcal{C}$ .
\item The following are equivalent
\begin{enumerate}
\item  $\mathcal{F}$  is an  $\mathcal{O}_X$ -module.
\end{enumerate}
\end{lemma}
```

Application: Sequence Tagging

Input: a sentence of n words: $x_1, \dots, x_n$

Output: $y_1, \dots, y_n, y_i \in \{1, \dots, C\}$



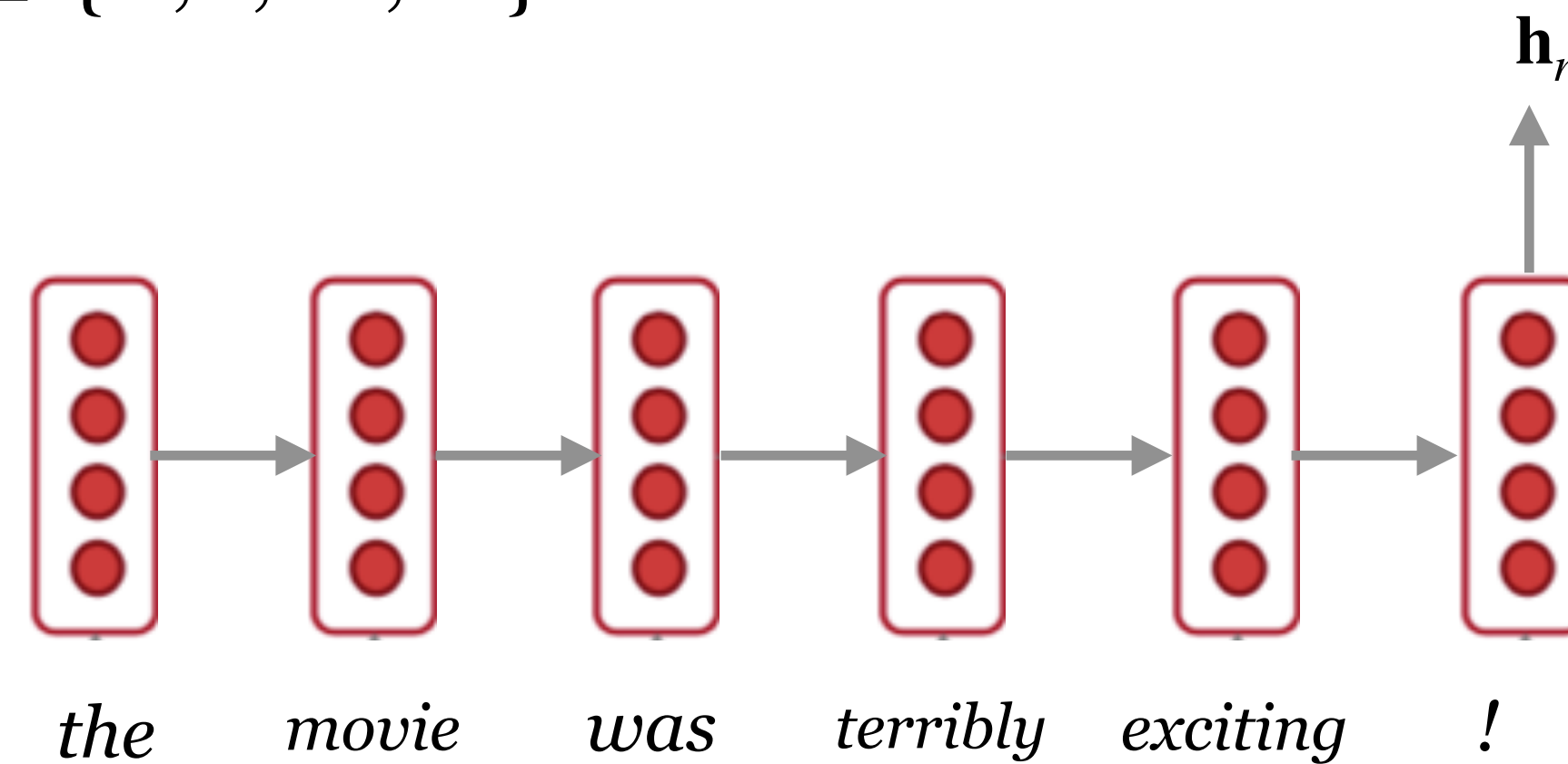
$$P(y_i = k) = \text{softmax}_k(\mathbf{W}_o \mathbf{h}_i) \quad \mathbf{W}_o \in \mathbb{R}^{C \times d}$$

$$L = -\frac{1}{n} \sum_{i=1}^n \log P(y_i = k)$$

Application: Text Classification

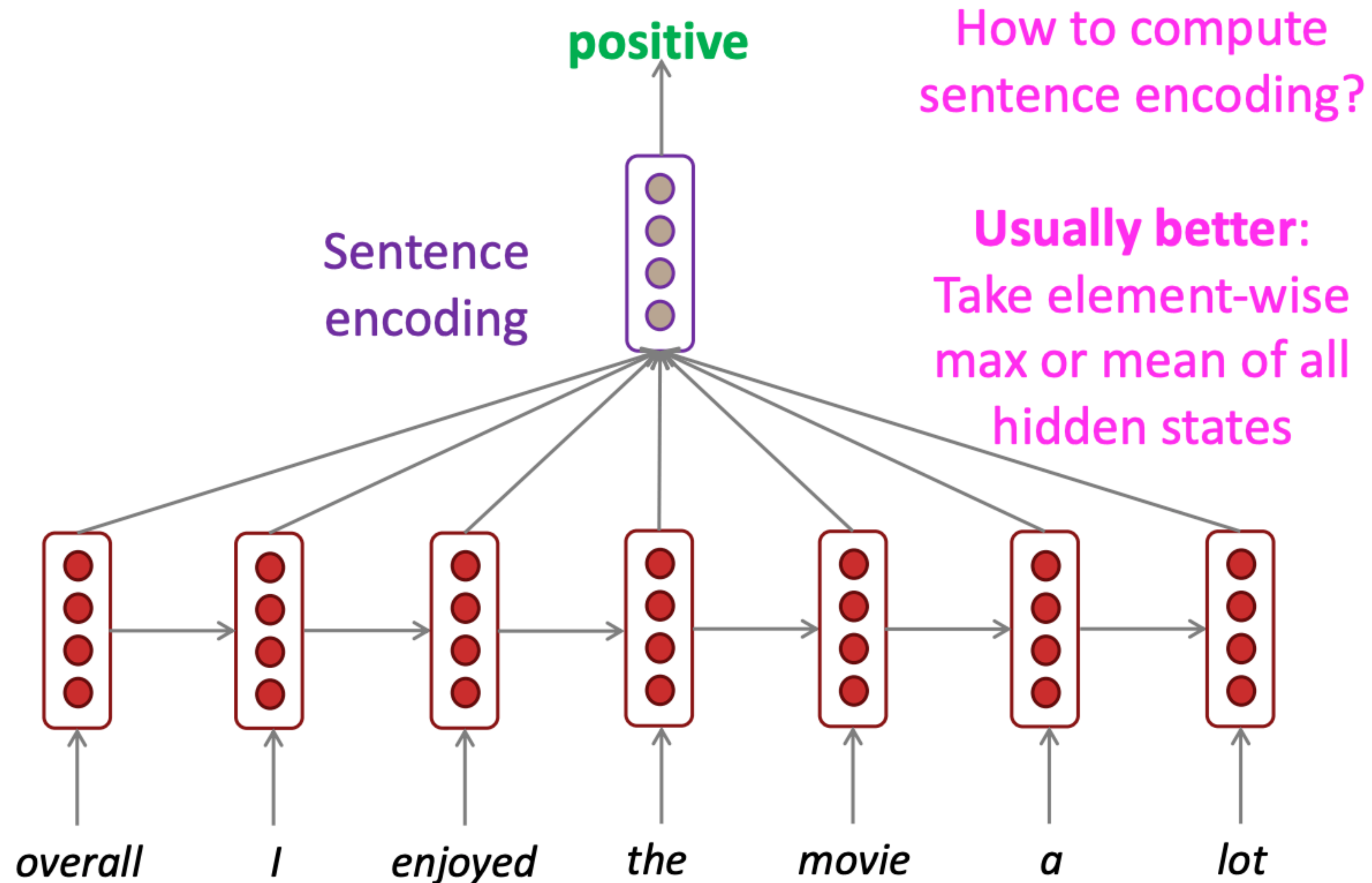
Input: a sentence of n words

Output: $y \in \{1, 2, \dots, C\}$

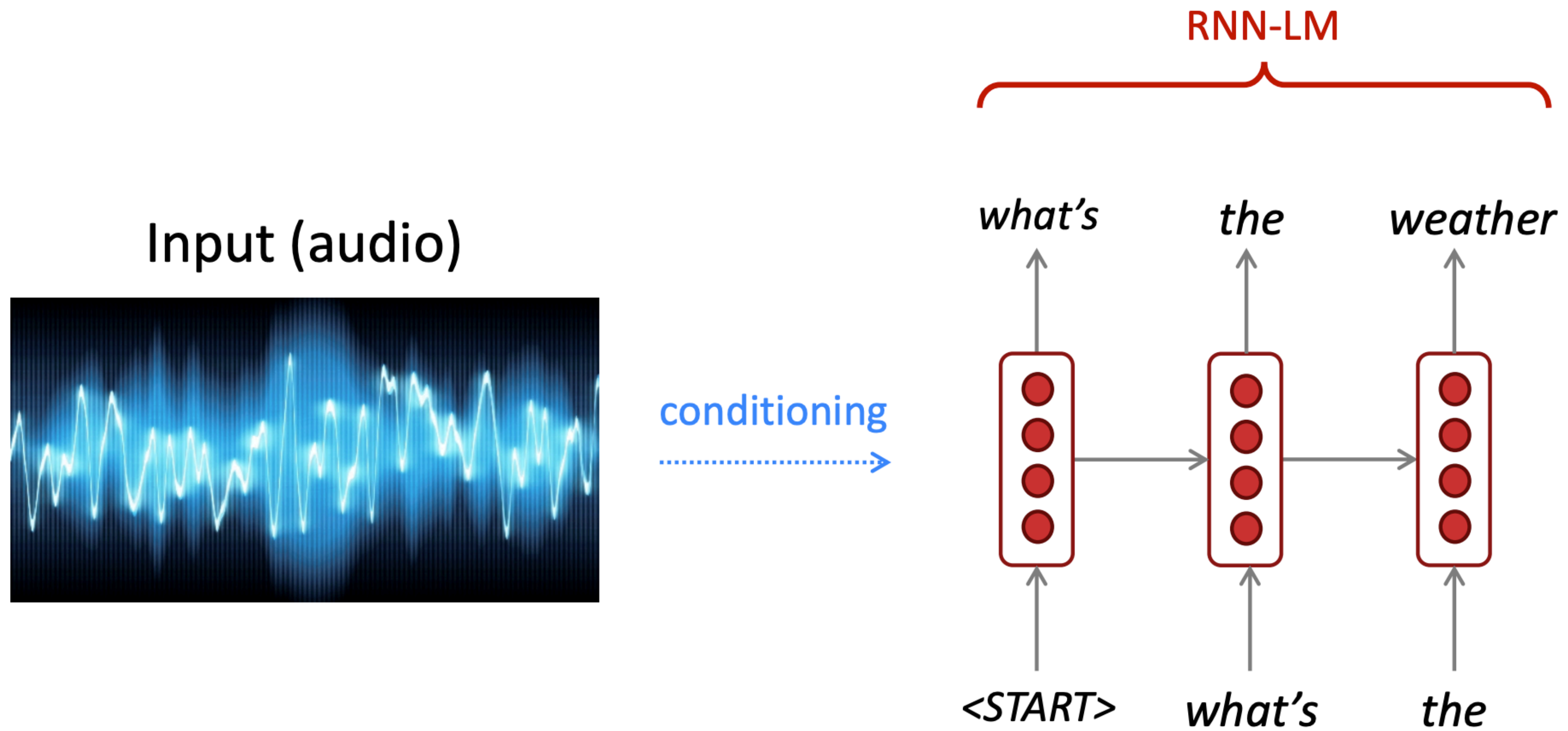


$$P(y = k) = \text{softmax}_k(\mathbf{W}_o \mathbf{h}_n) \quad \mathbf{W}_o \in \mathbb{R}^{C \times d}$$

Application: Text classification



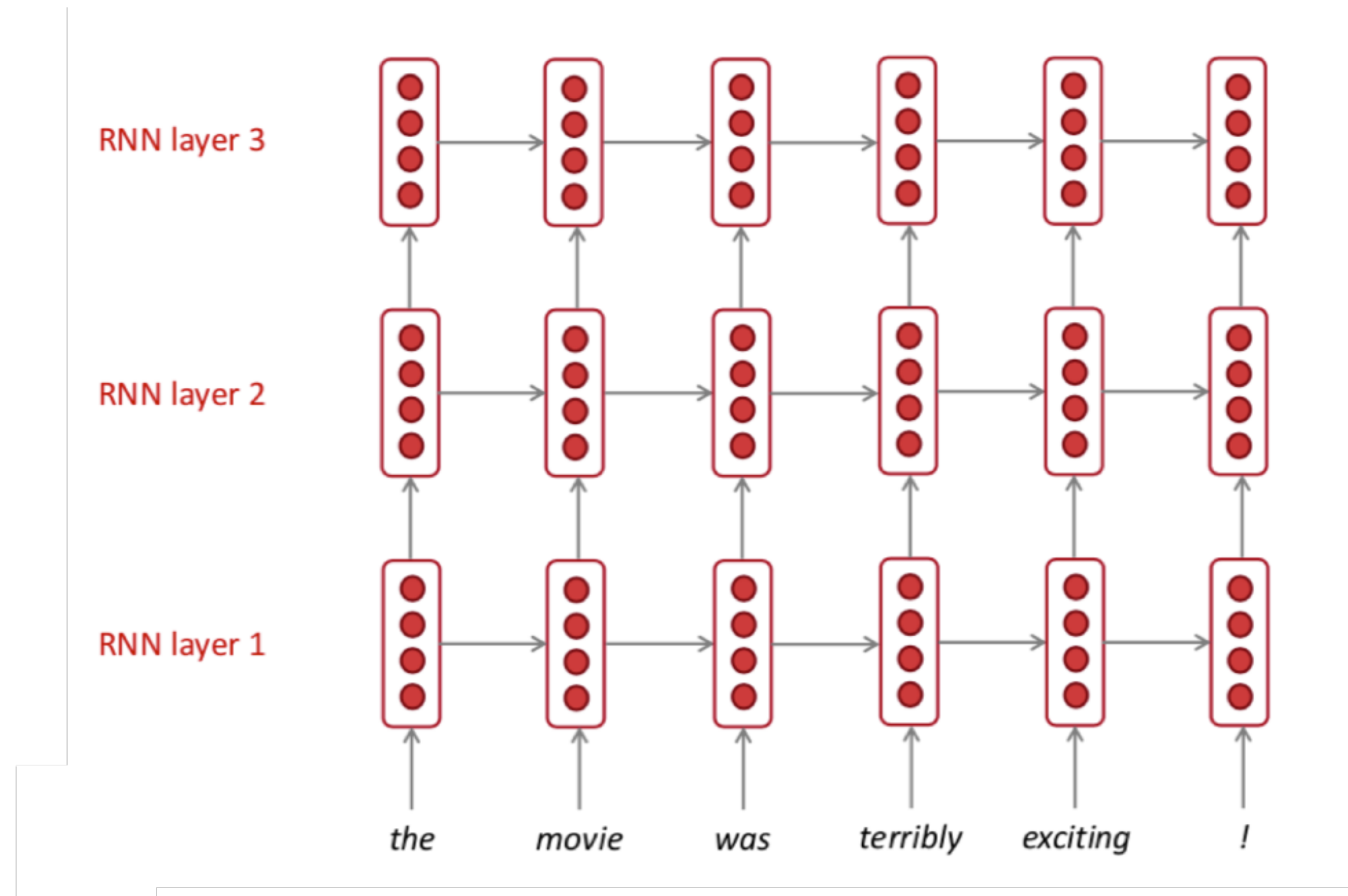
Conditional Text Generation



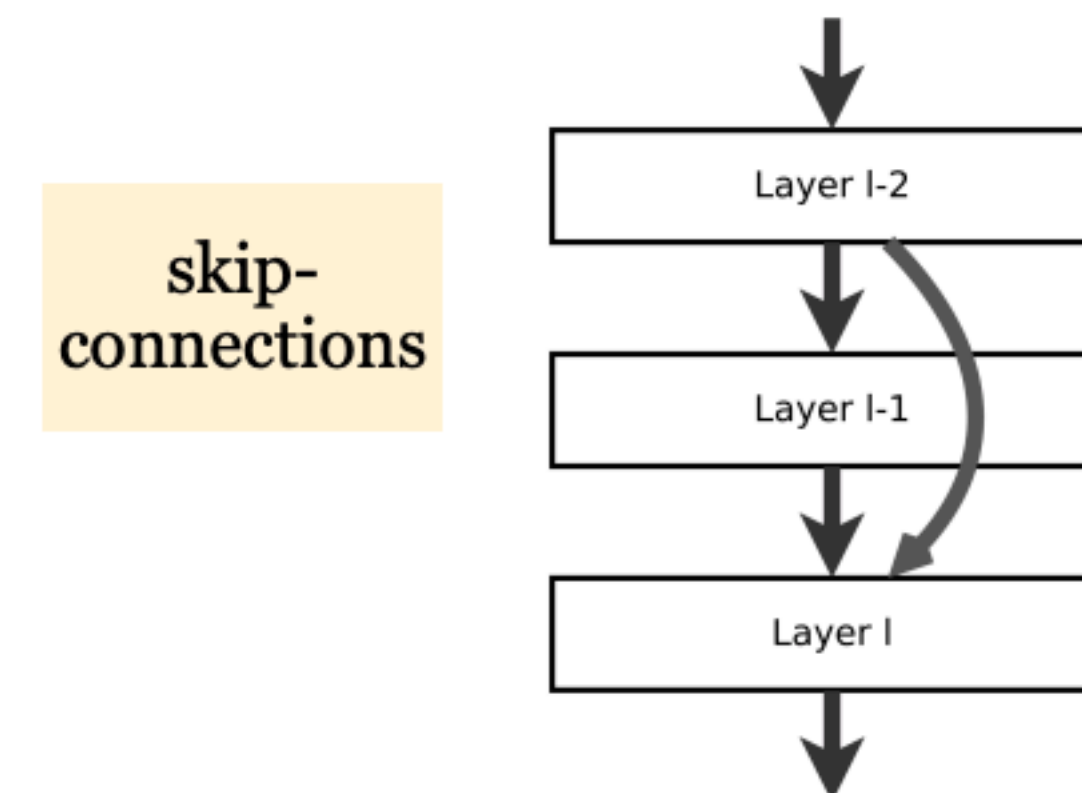
Multi-layer RNNs

- RNNs are already “deep” on one dimension (unroll over time steps)
- We can also make them “deep” in another dimension by applying multiple RNNs
- Multi-layer RNNs are also called [stacked RNNs](#).

Stacking multi-layered RNNs



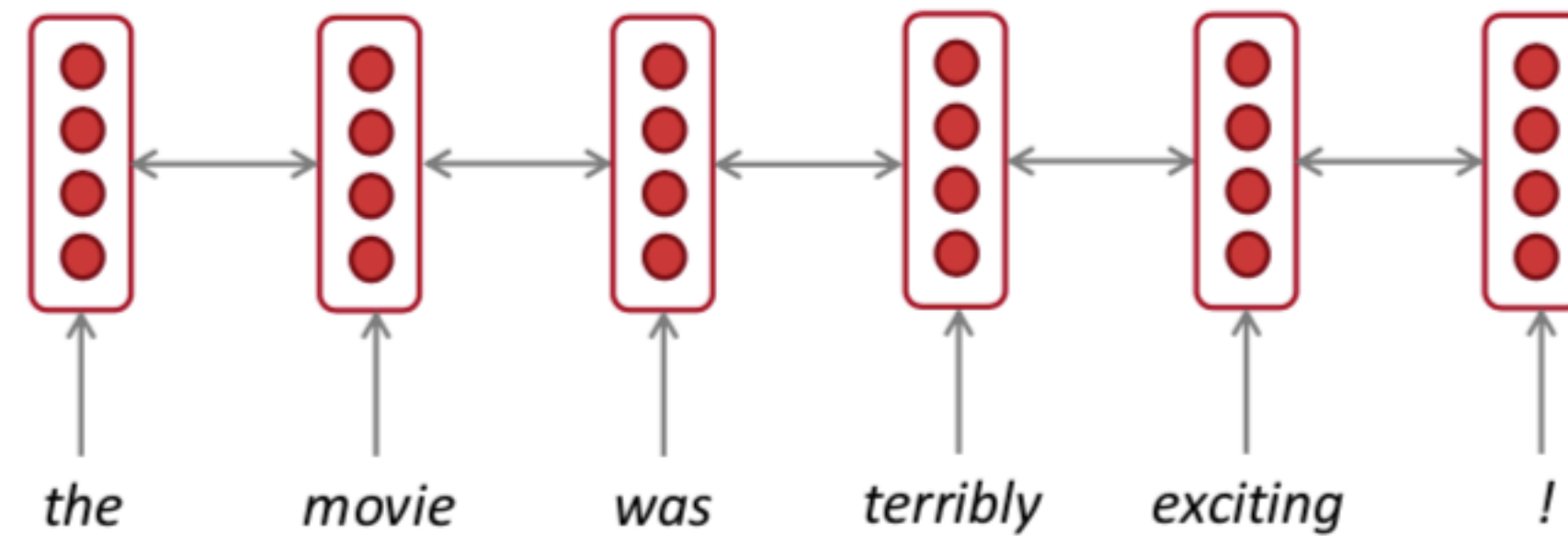
- The hidden states from RNN layer i are the inputs to RNN layer $i + 1$



- In practice, using 2 to 4 layers is common (usually better than 1 layer)
- Transformer-based networks can be up to 24 layers with lots of skip-connections.

Bidirectional RNNs

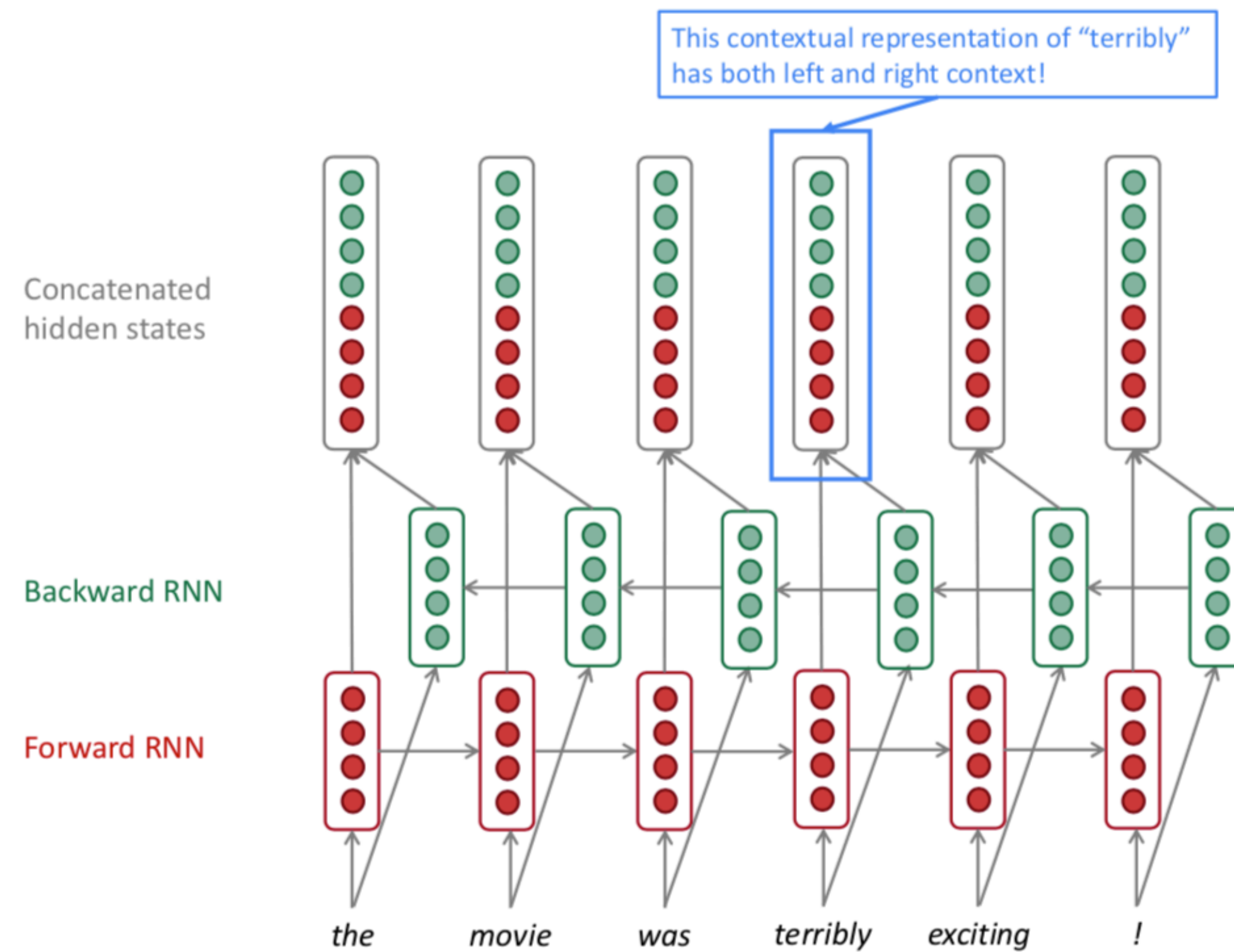
- Bidirectionality is important in language representations:



terribly:

- left context "the movie was"
- right context "exciting !"

Bidirectional RNNs



$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t) \in \mathbb{R}^d$$

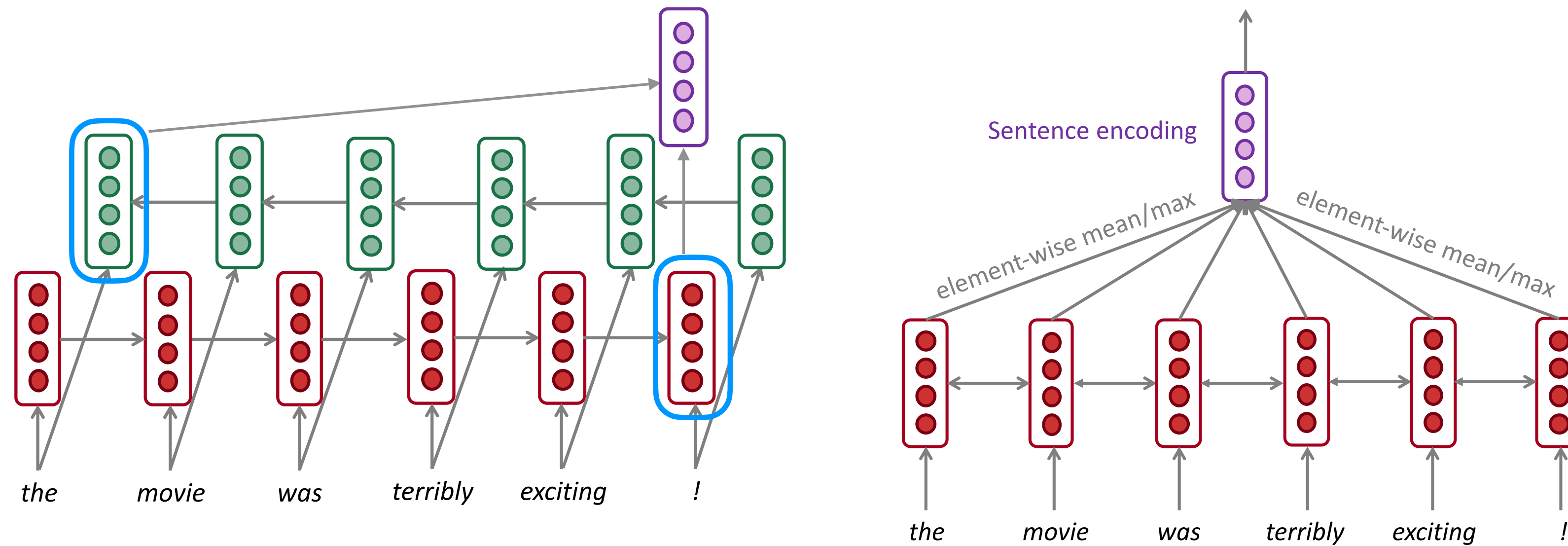
$$\vec{\mathbf{h}}_t = f_1(\vec{\mathbf{h}}_{t-1}, \mathbf{x}_t), t = 1, 2, \dots, n$$

$$\overleftarrow{\mathbf{h}}_t = f_2(\overleftarrow{\mathbf{h}}_{t+1}, \mathbf{x}_t), t = n, n-1, \dots, 1$$

$$\mathbf{h}_t = [\overleftarrow{\mathbf{h}}_t, \vec{\mathbf{h}}_t] \in \mathbb{R}^{2d}$$

Bidirectional RNNs

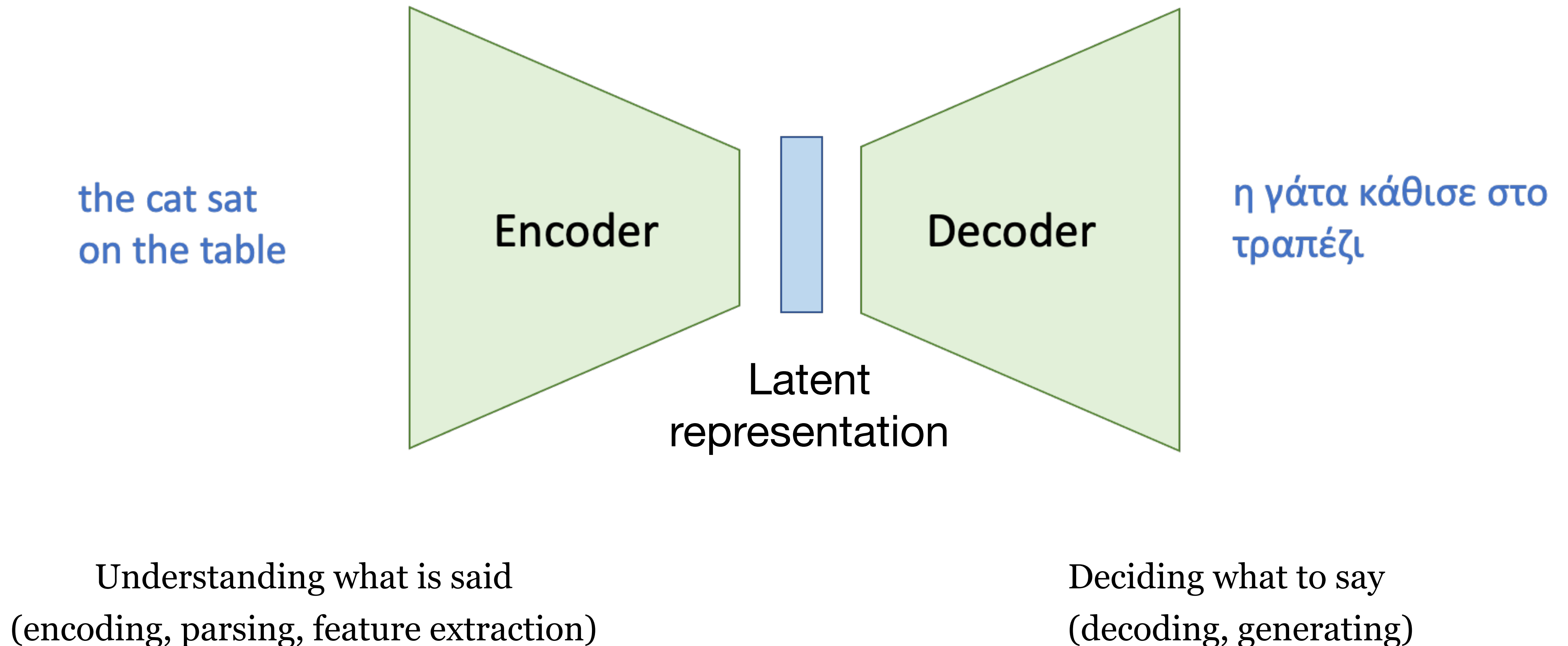
- Sequence tagging: Yes!
- Text classification: Yes! With slight modifications.



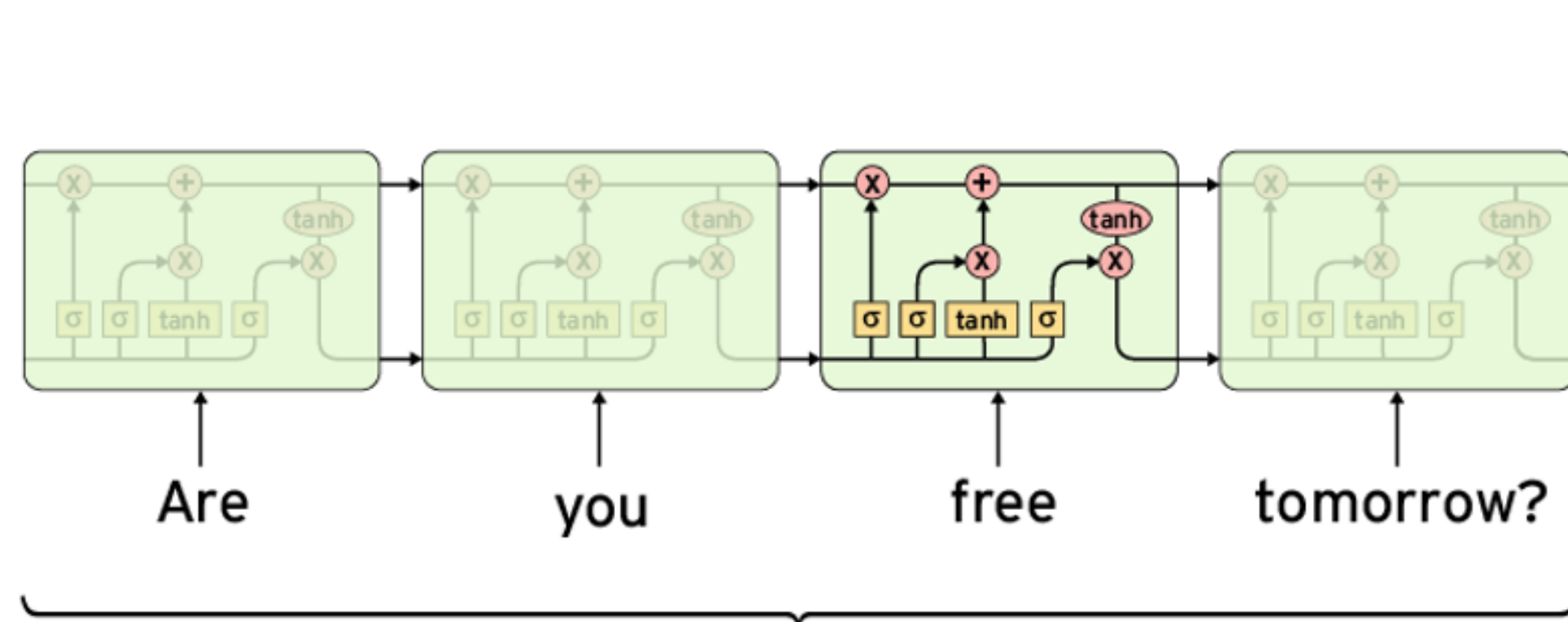
- Text generation: No. **Why?**

Sequence Generation

Want computer friendly representation for applications

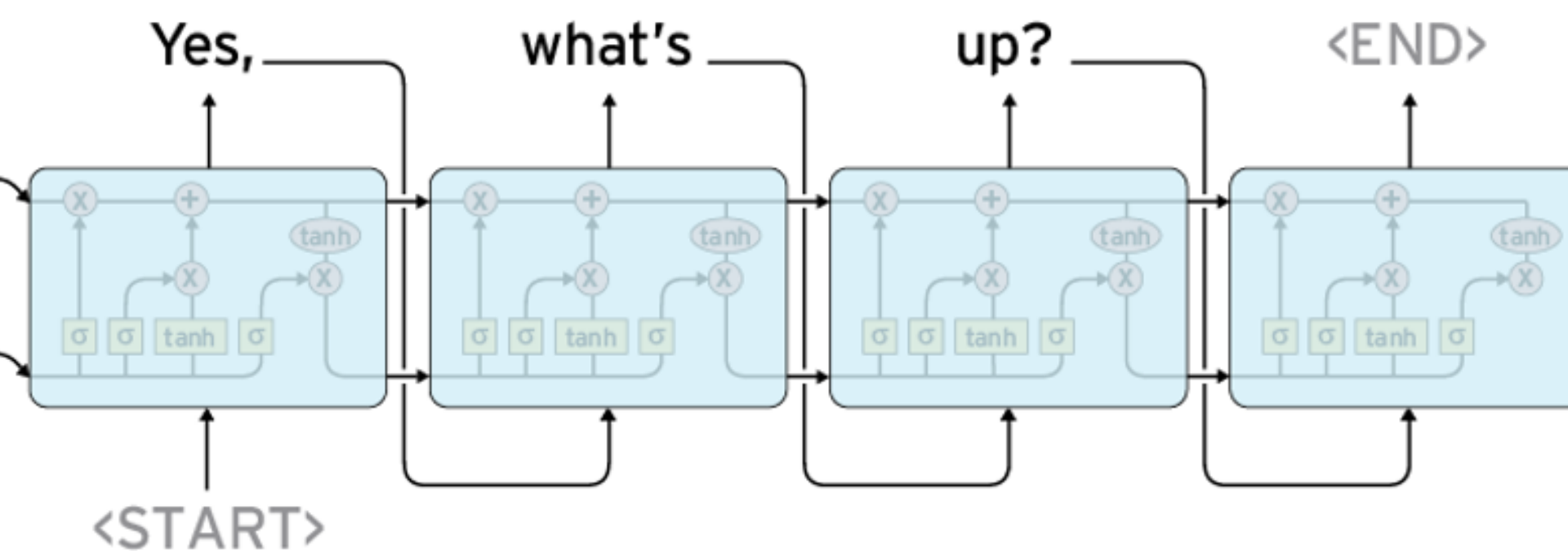


ENCODER



Incoming Email

Reply



DECODER

Seq2Seq Tasks and Applications

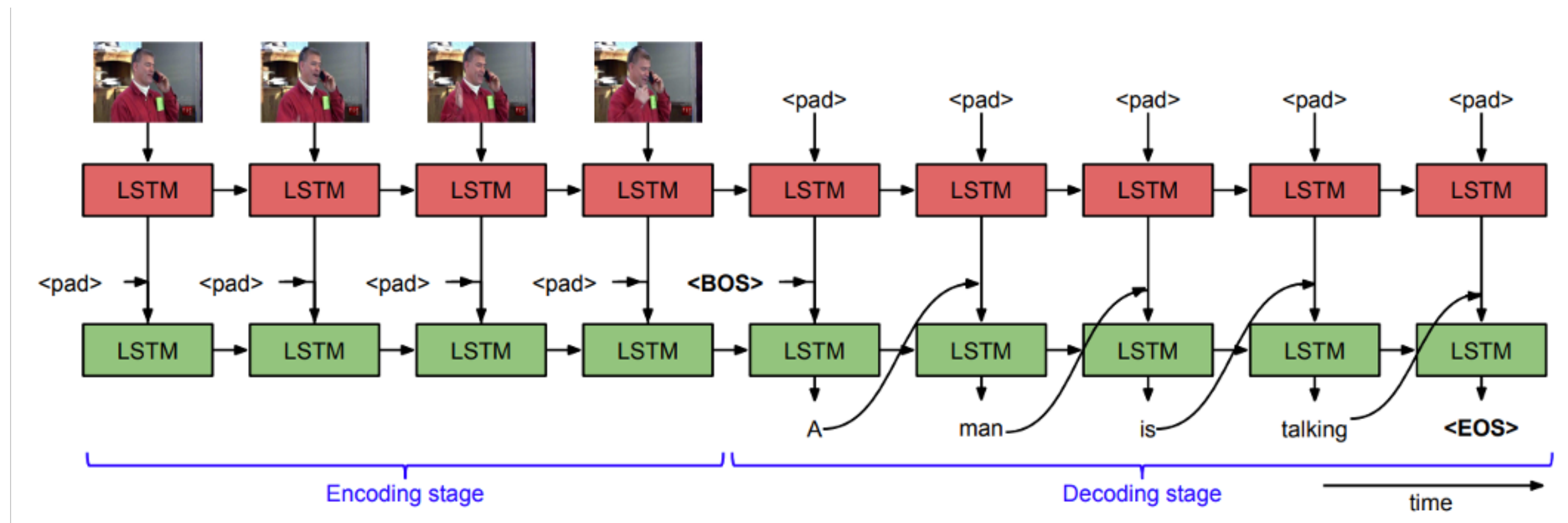
Task/Application	Input	Output
Machine Translation	French	English
Summarization	Document	Short Summary
Dialogue	Utterance	Response
Parsing	Sentence	Parse tree (as sequence)
Question Answering	Context + Question	Answer

Cross-Modal Seq2Seq

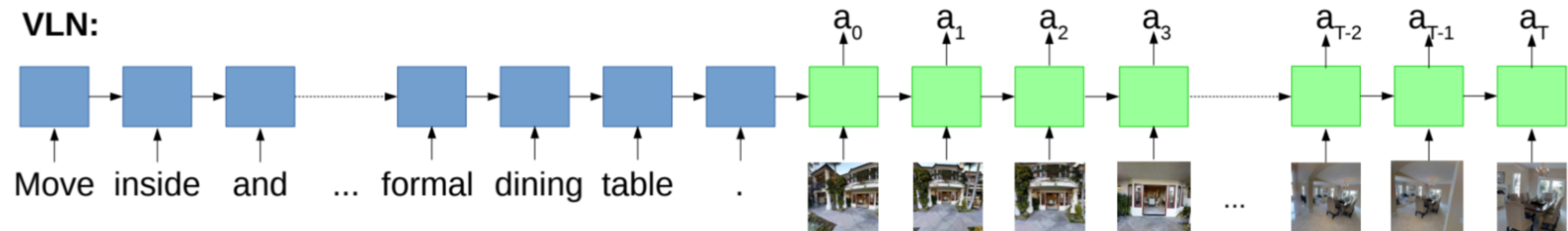
Task/Application	Input	Output
Speech Recognition	Speech Signal	Transcript
Image Captioning	Image	Text
Video Captioning	Video	Text
Vision-Language Navigation	Text	Actions

Cross-modal sequence generation

- Video captioning (video frames to text)



- Embodied AI (text + frames to actions)



Seq2Seq Tasks and Applications

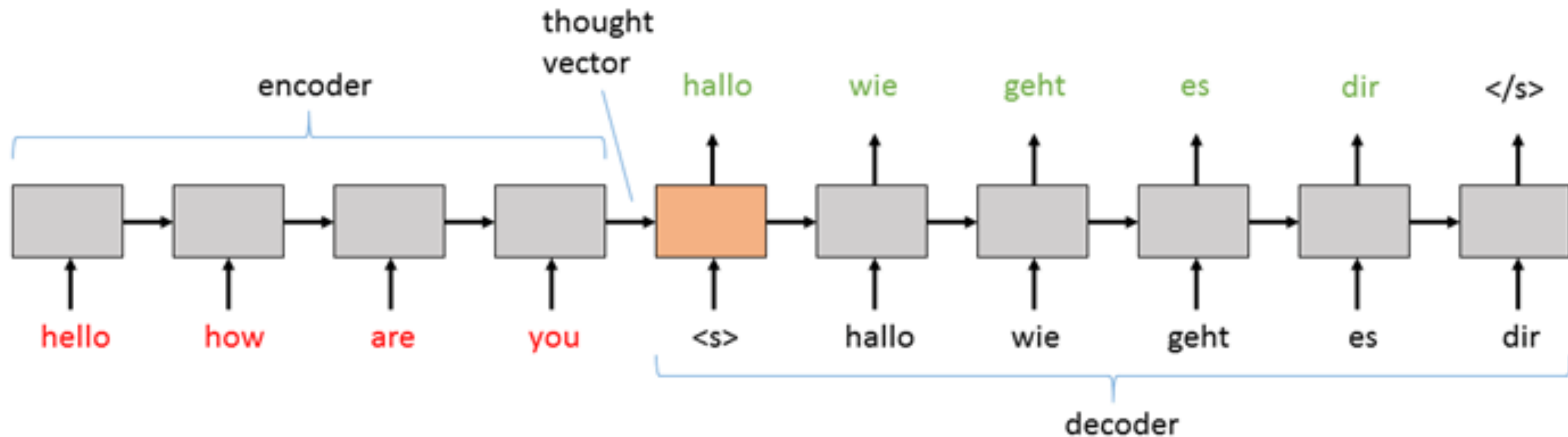
Task/Application	Input	Output
Machine Translation	French	English
Summarization	Document	Short Summary
Dialogue	Utterance	Response
Parsing	Sentence	Parse tree (as sequence)
Question Answering	Context + Question	Answer

Sequence to sequence models

Neural Machine Translation

- ▶ A **single neural network** is used to translate from source to target
- ▶ Architecture: Encoder-Decoder
 - ▶ Two main components:
 - ▶ **Encoder:** Convert source sentence (input) into a vector/matrix
 - ▶ **Decoder:** Convert encoding into a sentence in target language (output)

Sequence to Sequence learning (Seq2seq)



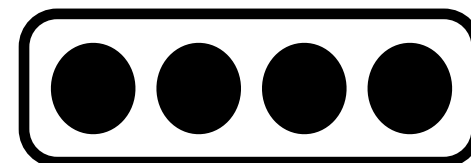
- Encode entire input sequence into a single vector **(using an RNN)**
- Decode one word at a time **(again, using an RNN!)**
- Beam search for better inference
- Learning is not trivial! (vanishing/exploding gradients)

(Sutskever et al., 2014)

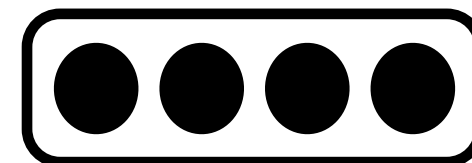
Encoder

Sentence: This cat is cute

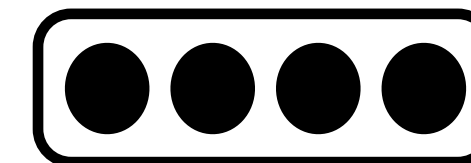
word
embedding



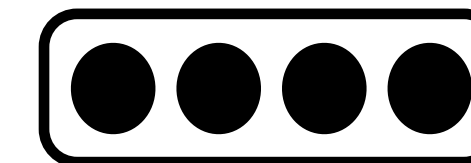
This



cat



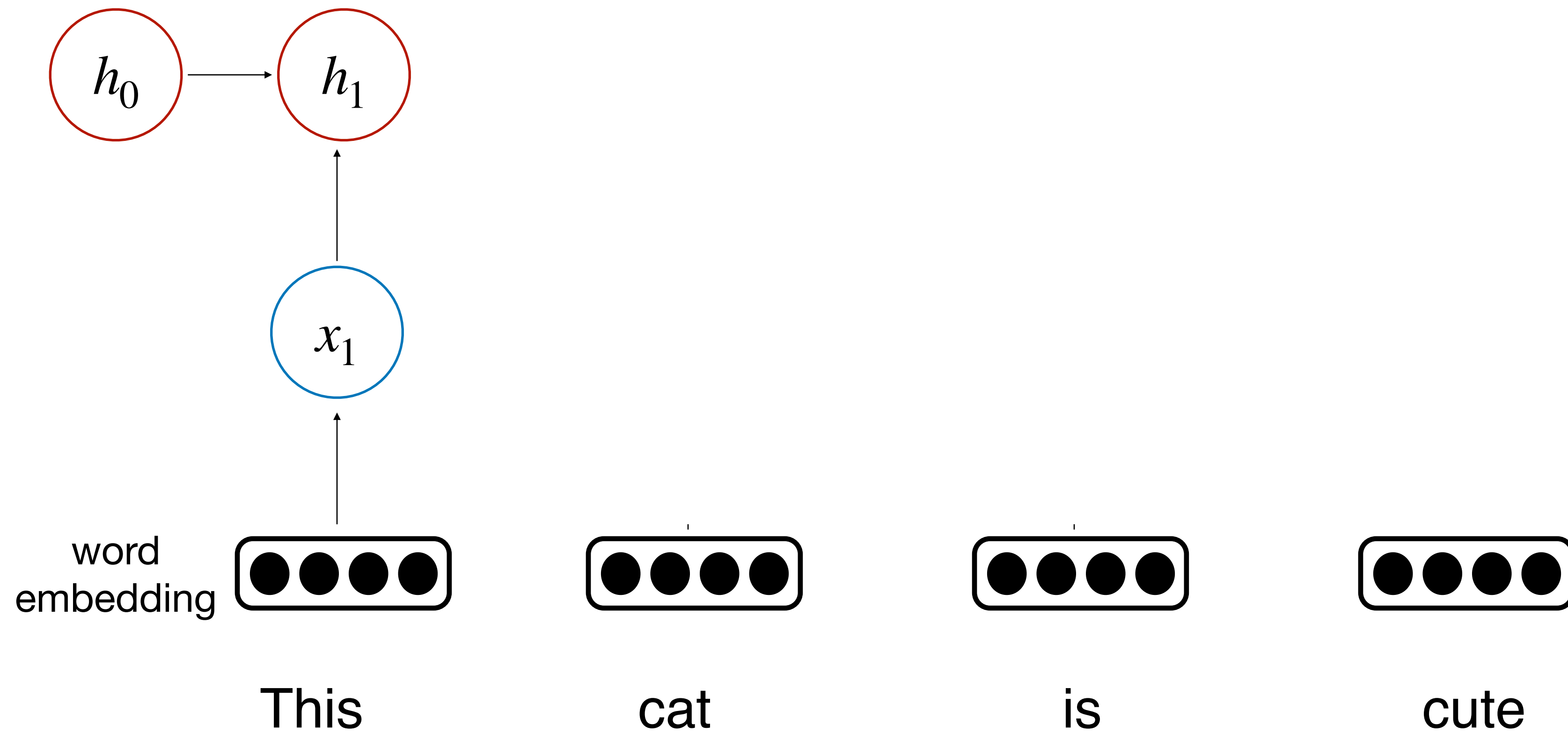
is



cute

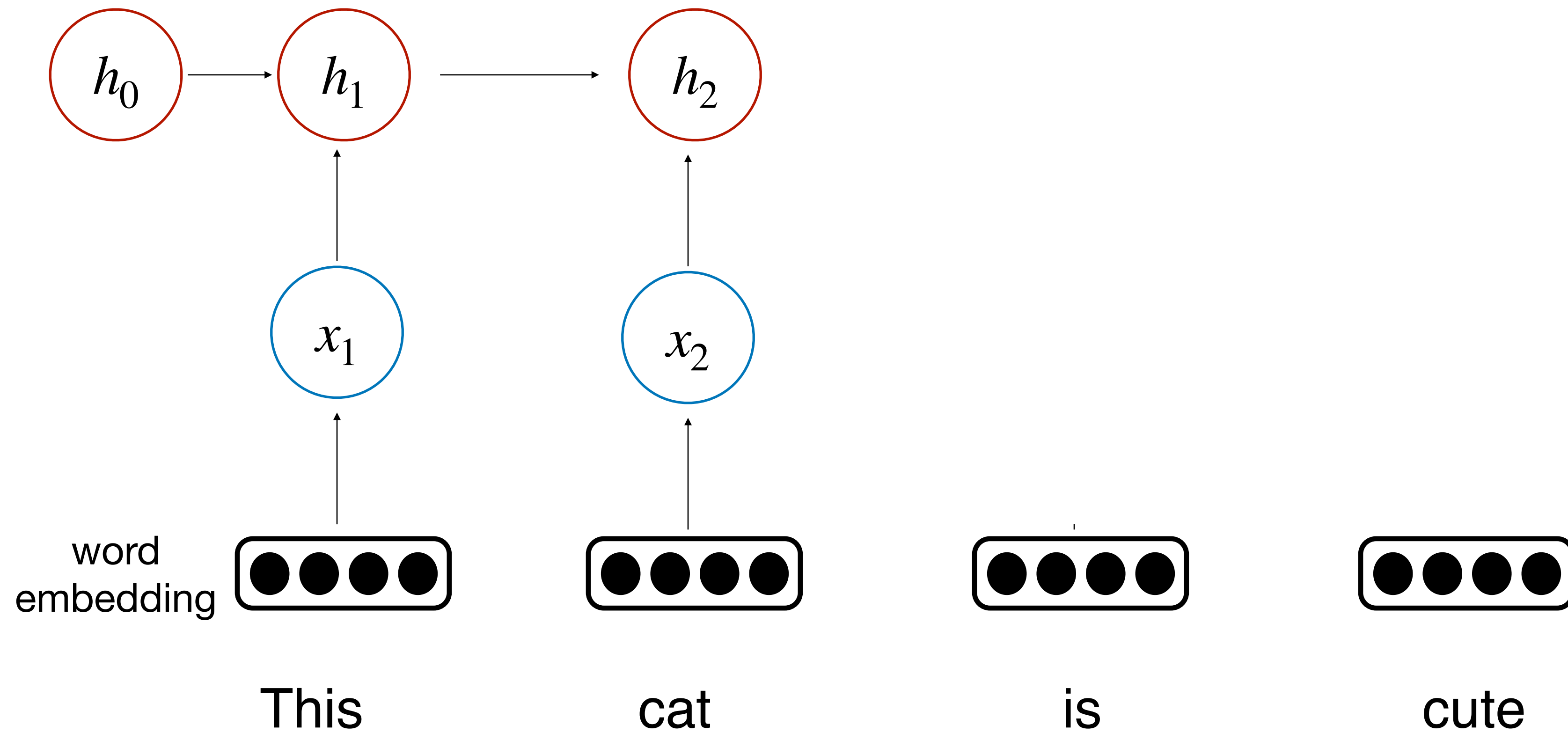
Encoder

Sentence: This cat is cute

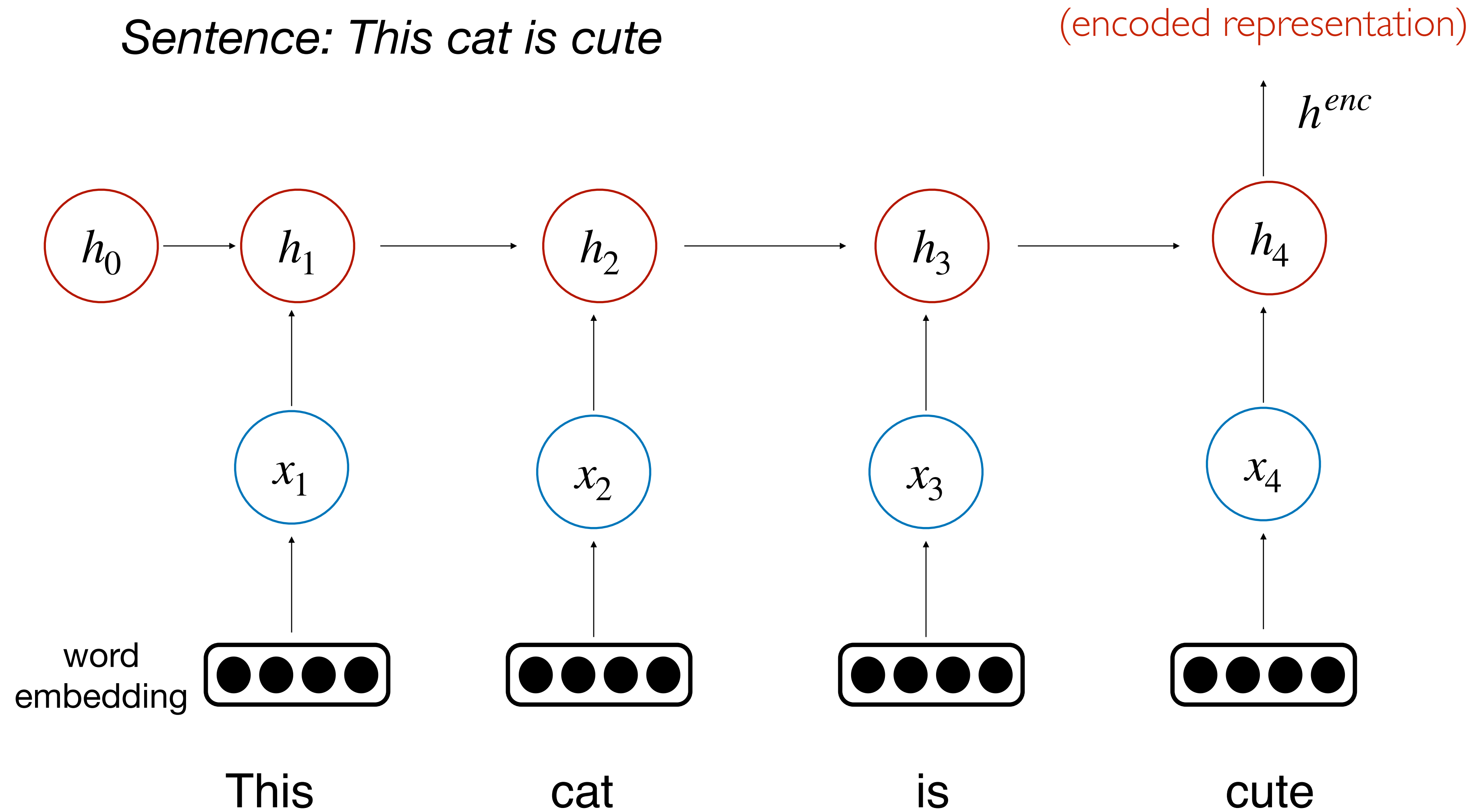


Encoder

Sentence: This cat is cute



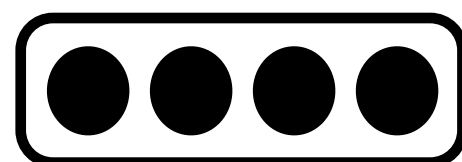
Encoder



Decoder

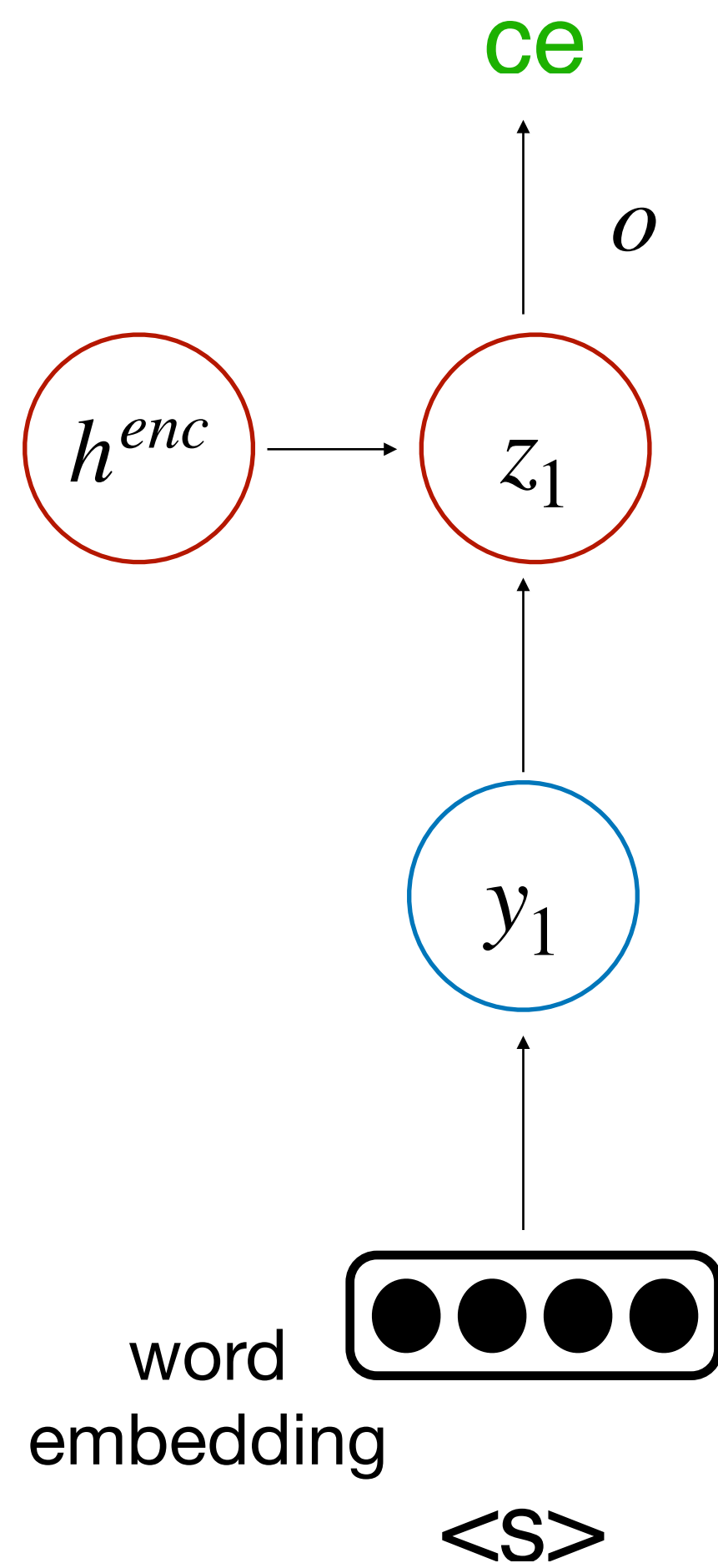
h^{enc}

word
embedding

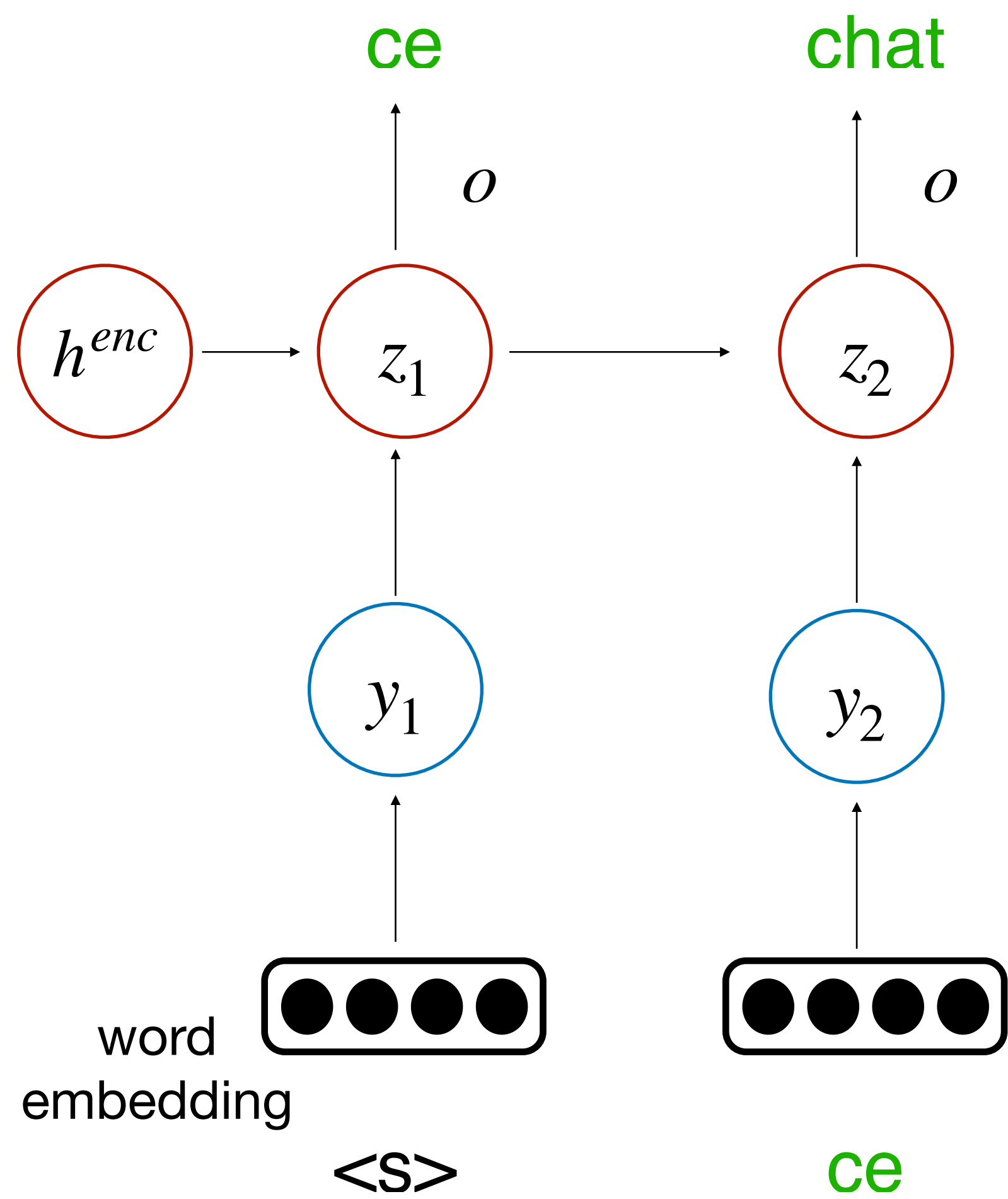


<S>

Decoder

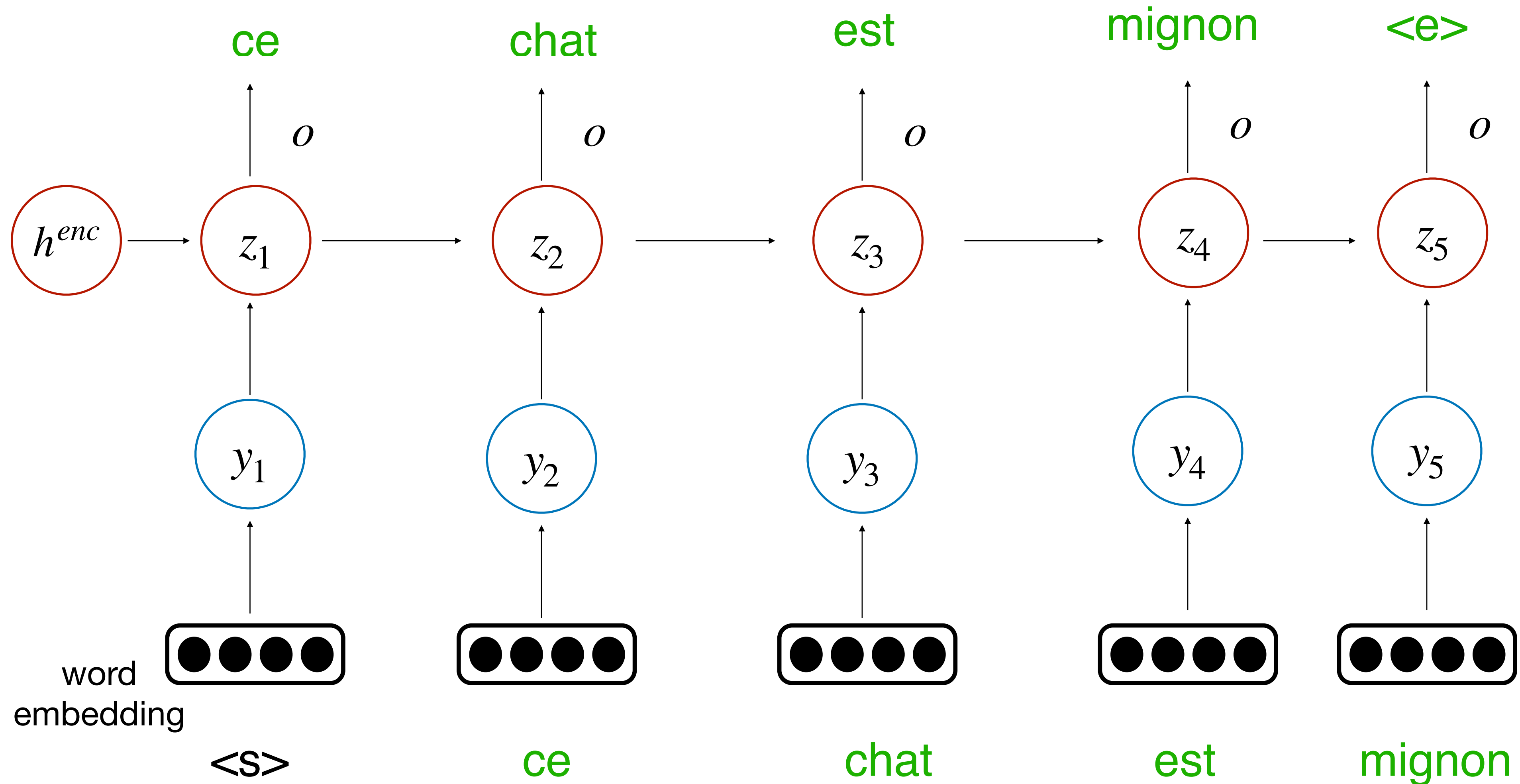


Decoder



Decoder

- A conditioned language model

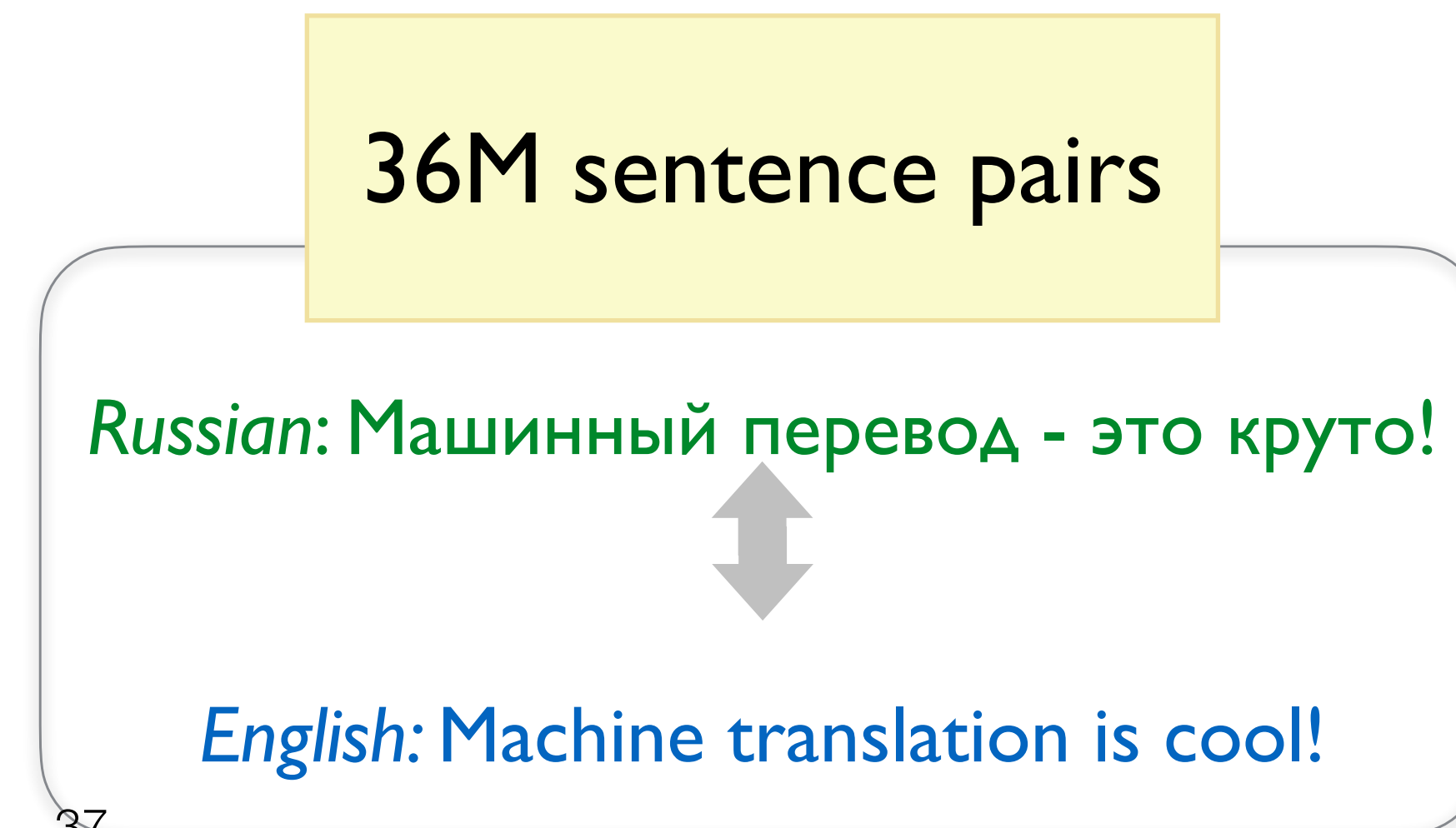


Seq2seq training

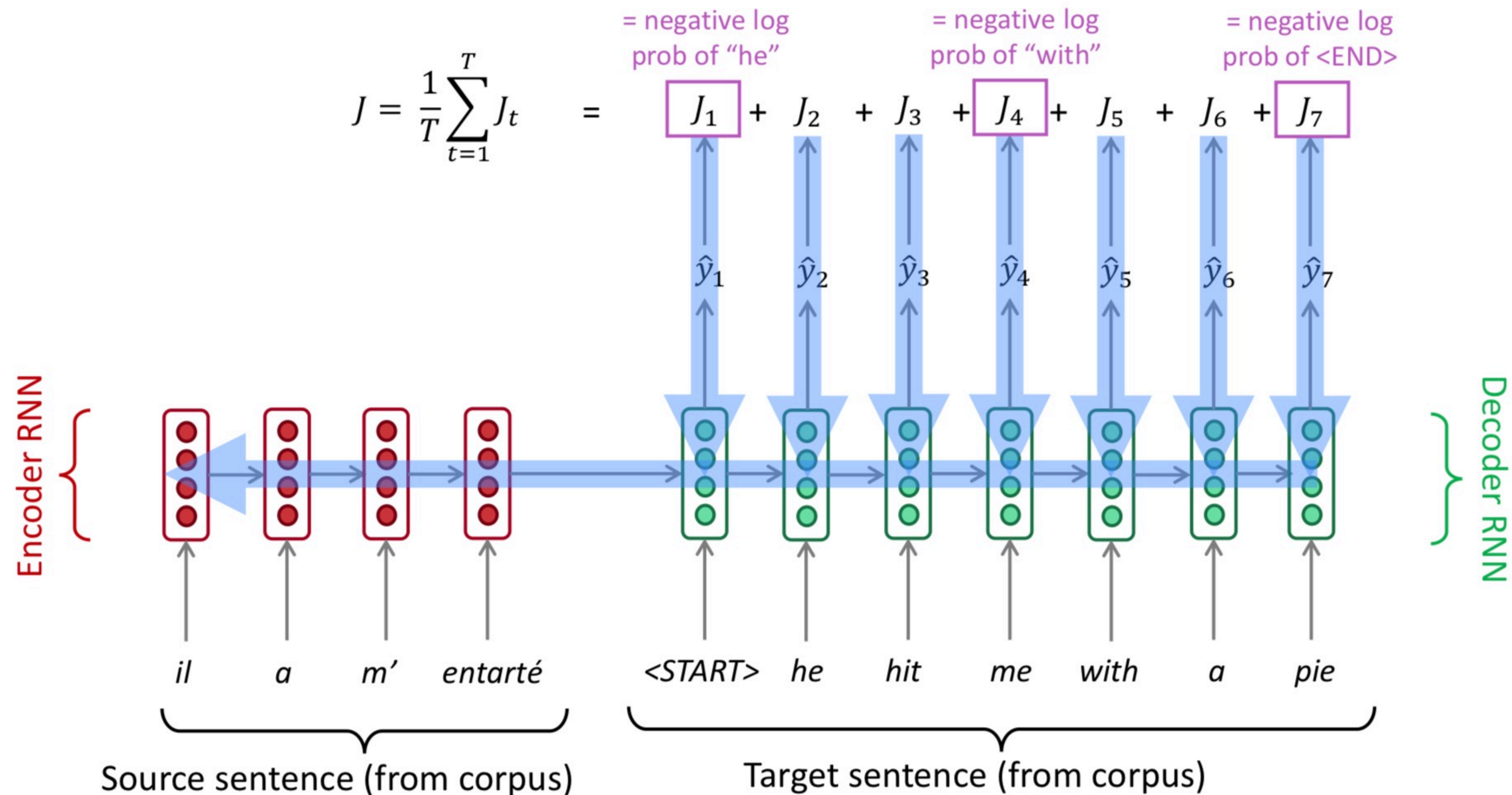
- ▶ Similar to training a language model!
- ▶ Minimize cross-entropy loss:

$$\sum_{t=1}^T -\log P(y_t | y_1, \dots, y_{t-1}, x_1, \dots, x_n)$$

- ▶ Back-propagate gradients through *both decoder and encoder*
- ▶ Need a really big corpus



Seq2seq training



Seq2seq is optimized as a single system.
Backpropagation operates "*end-to-end*".

Efficient Training: Batching

- Apply RNNs to batches of sequences
- Present data as 3D tensor of $(T \times B \times F)$
- Use mask matrix to aid with computations that ignore padded zeros

Padded sequences

1	1	1	1	0	0
1	0	0	0	0	0
1	1	1	1	1	1
1	1	1	0	0	0

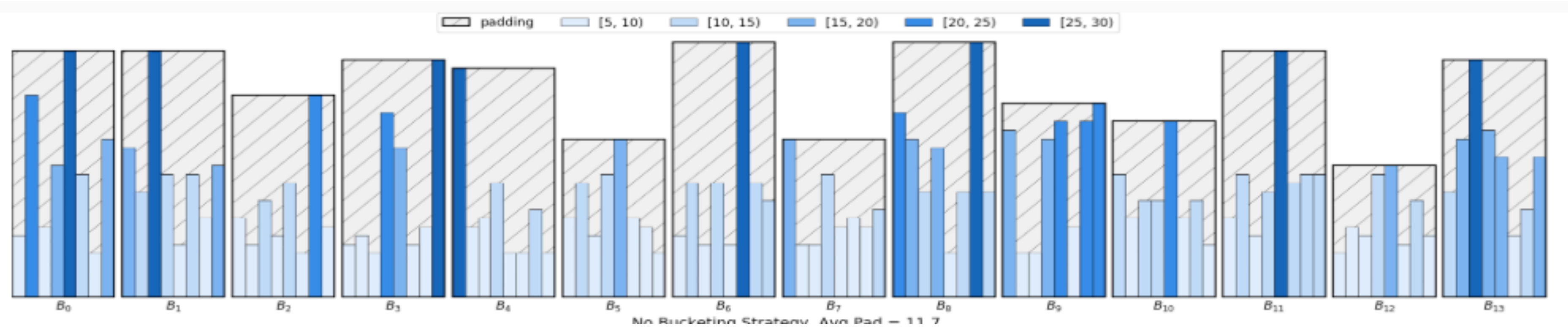
Lengths

4
1
6
3

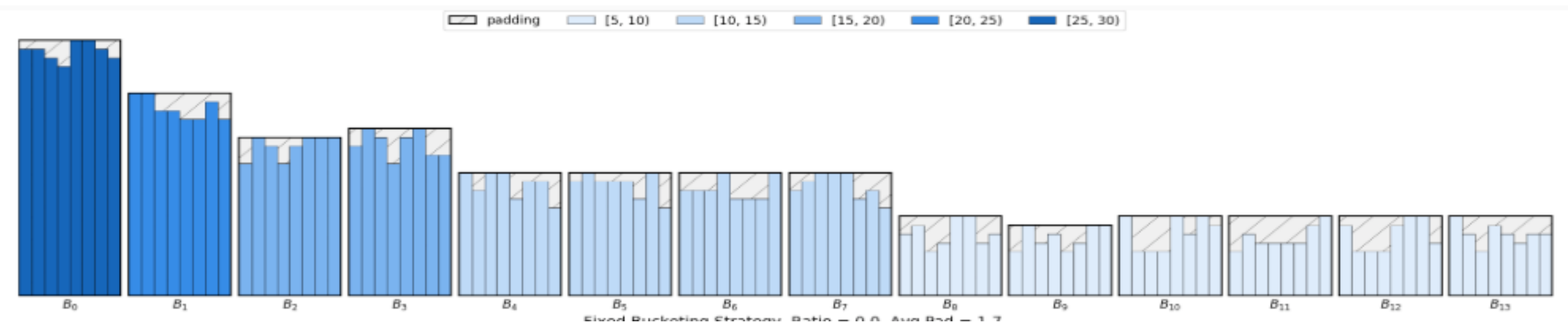
Batching

- Sorting (partially) can help to create more efficient mini-batches
- However, the input is less randomized

Unsorted



Sorted



Decoding strategies

Generation

How can we use our model (decoder) to generate sentences?

- **Sampling**: Try to generate a *random* sentence according to the probability distribution
- **Argmax**: Try to generate the *best* sentence, the sentence with the *highest* probability

Decoding Strategies

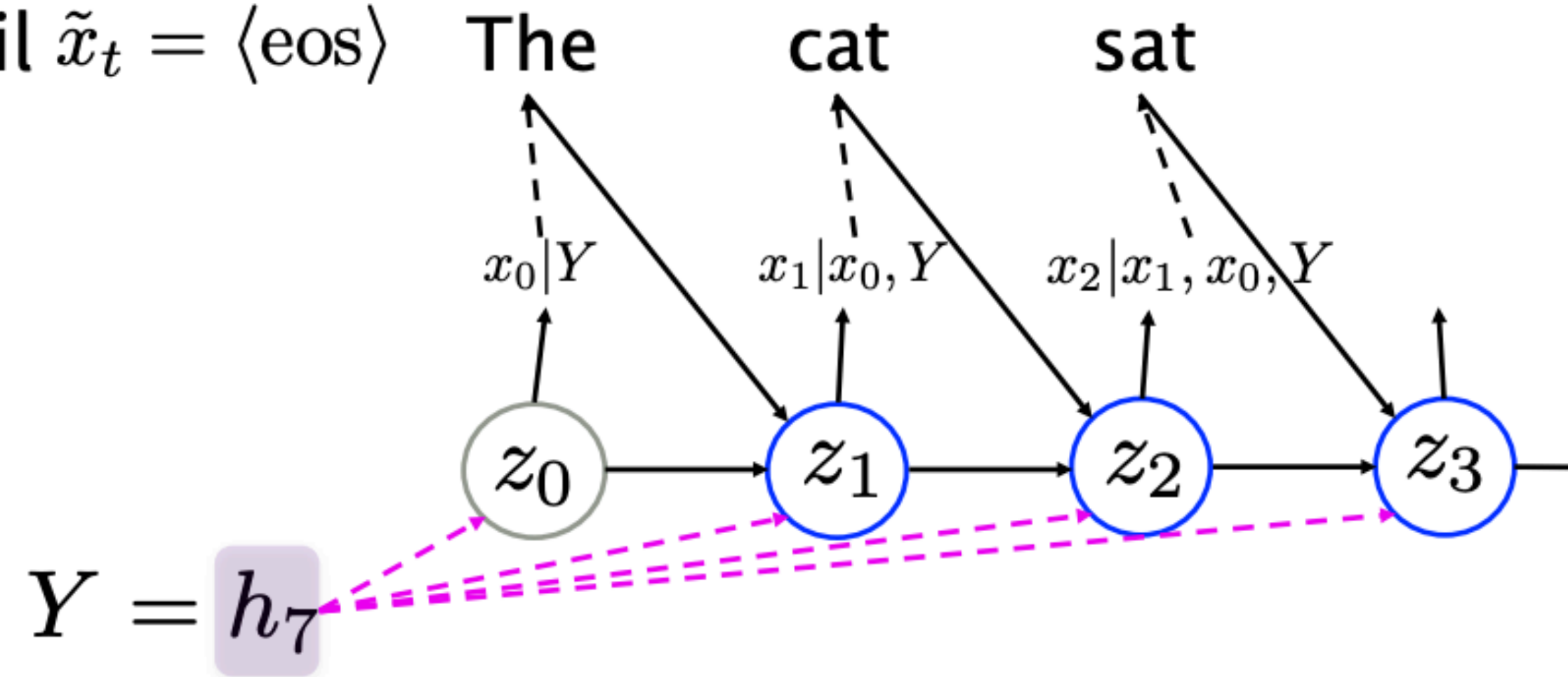
- ▶ Ancestral sampling
- ▶ Greedy decoding
- ▶ Exhaustive search
- ▶ Beam search

Ancestral Sampling

- Randomly sample words one by one
- Provides diverse output (high variance)

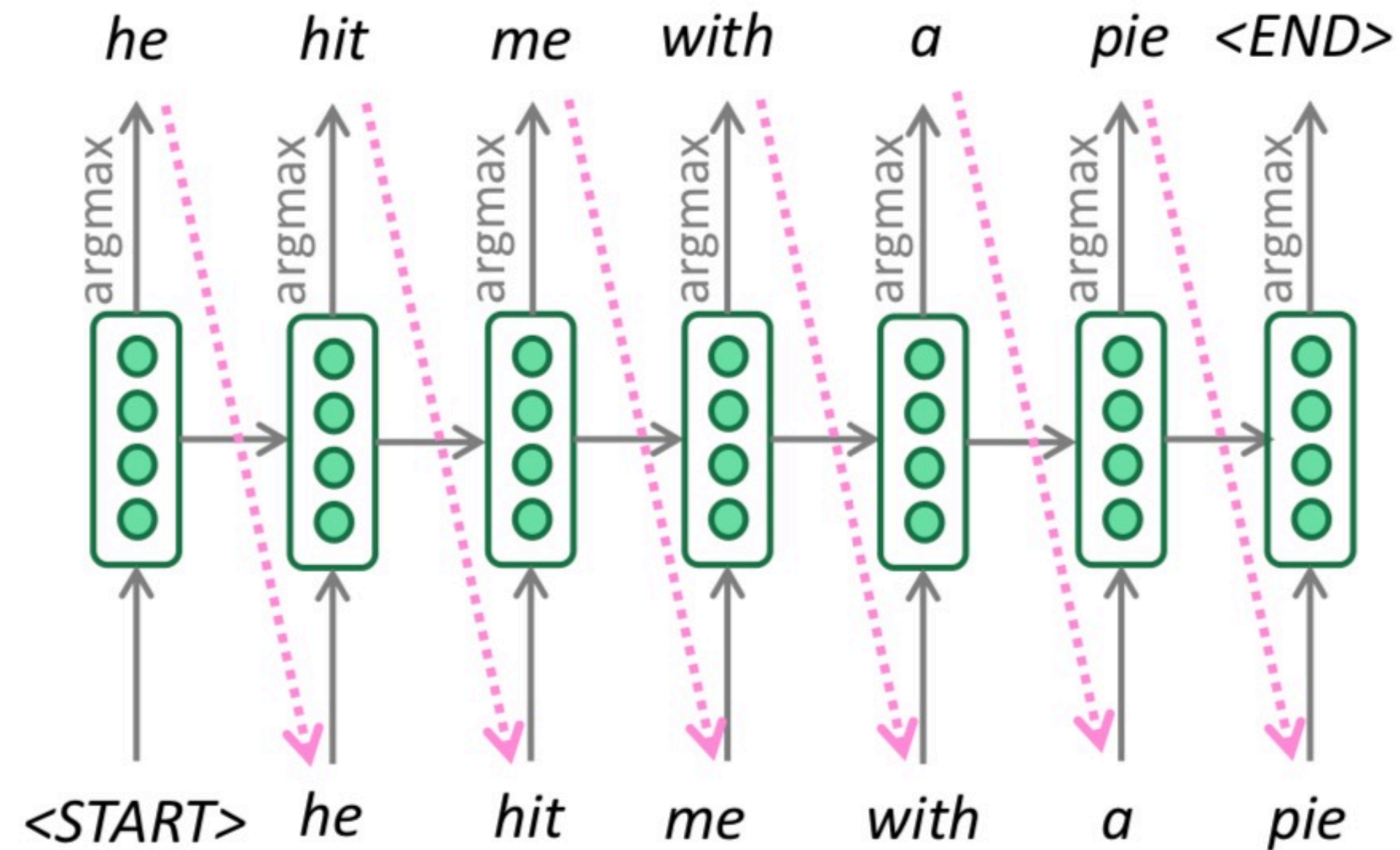
One symbol at a time from $\tilde{x}_t \sim x_t | x_{t-1}, \dots, x_1, Y$

Until $\tilde{x}_t = \langle \text{eos} \rangle$



(figure credit: Luong, Cho, and Manning)

Greedy decoding



- ▶ Compute **argmax** at every step of decoder to generate word
- ▶ **What's wrong?**

Exhaustive search?

- ▶ Find $\arg \max_{y_1, \dots, y_T} P(y_1, \dots, y_T | x_1, \dots, x_n)$
- ▶ Requires computing all possible sequences
 - ▶ $O(V^T)$ complexity!
 - ▶ Too expensive

Recall: Beam search (a middle ground)

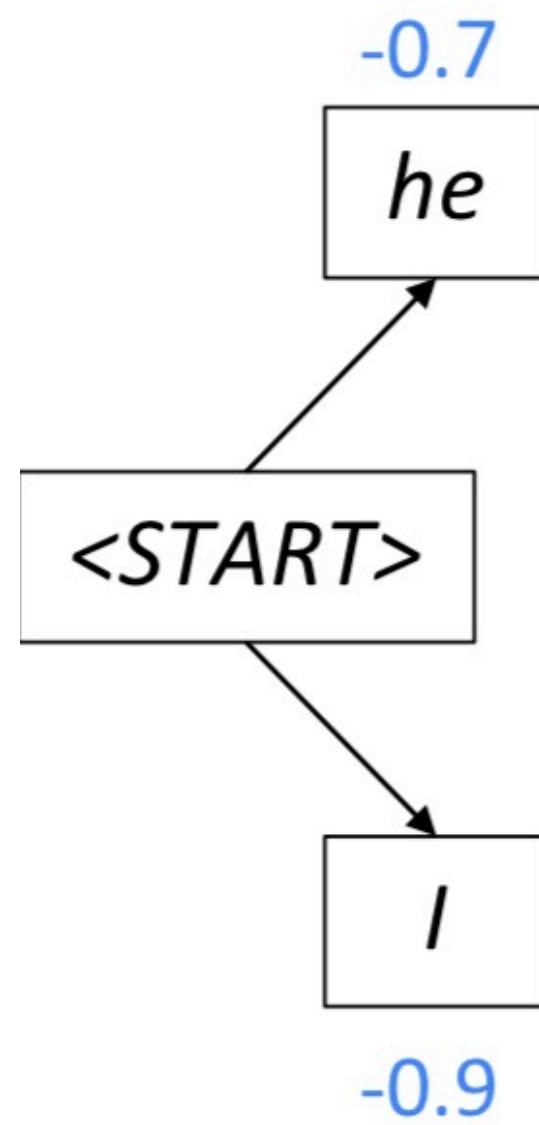
- ▶ **Key idea:** At every step, keep track of the **k most probable** partial translations (hypotheses)
- ▶ Score of each hypothesis = log probability

$$\sum_{t=1}^j \log P(y_t | y_1, \dots, y_{t-1}, x_1, \dots, x_n)$$

- ▶ Not guaranteed to be optimal
- ▶ More efficient than exhaustive search

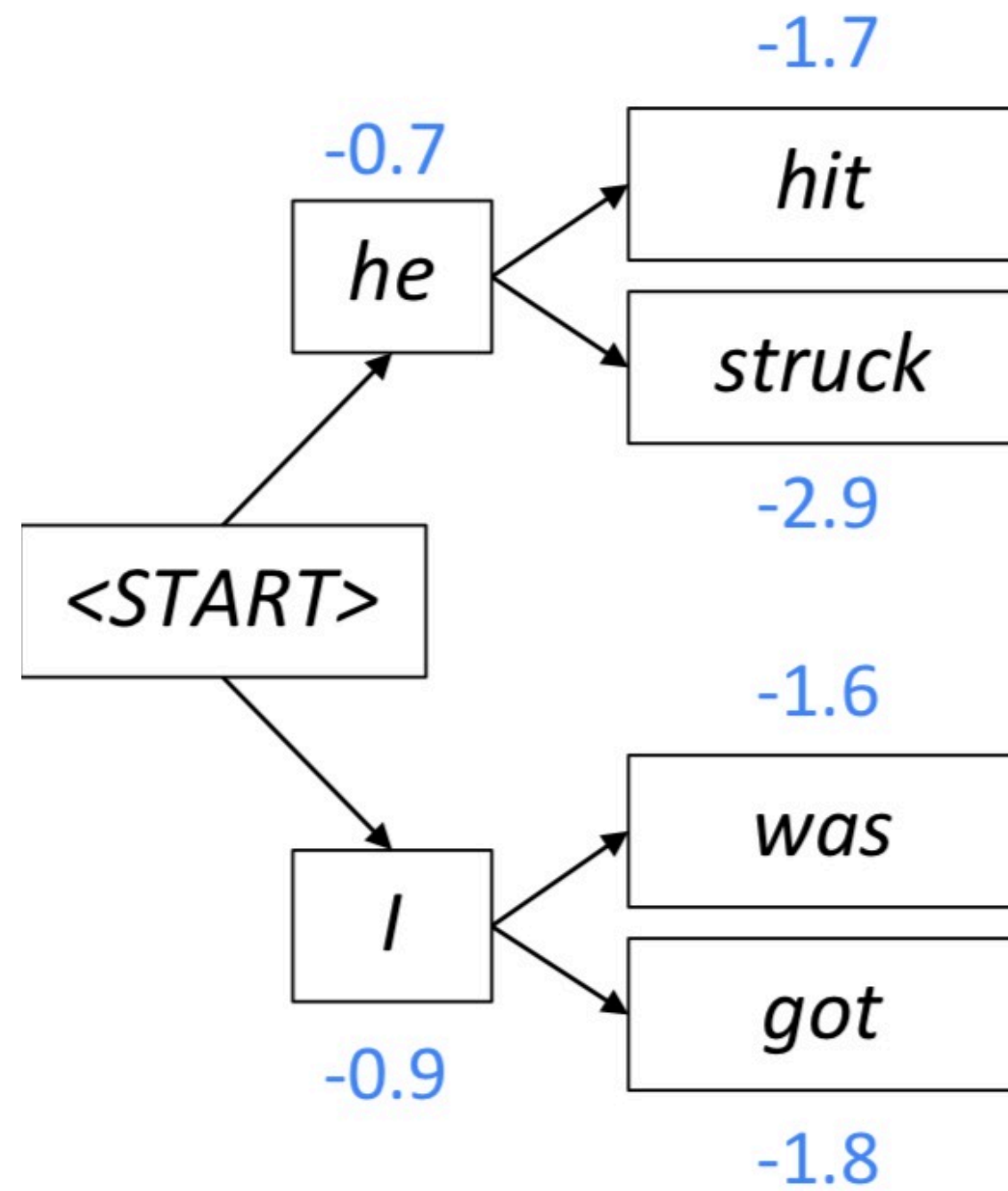
Beam decoding

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



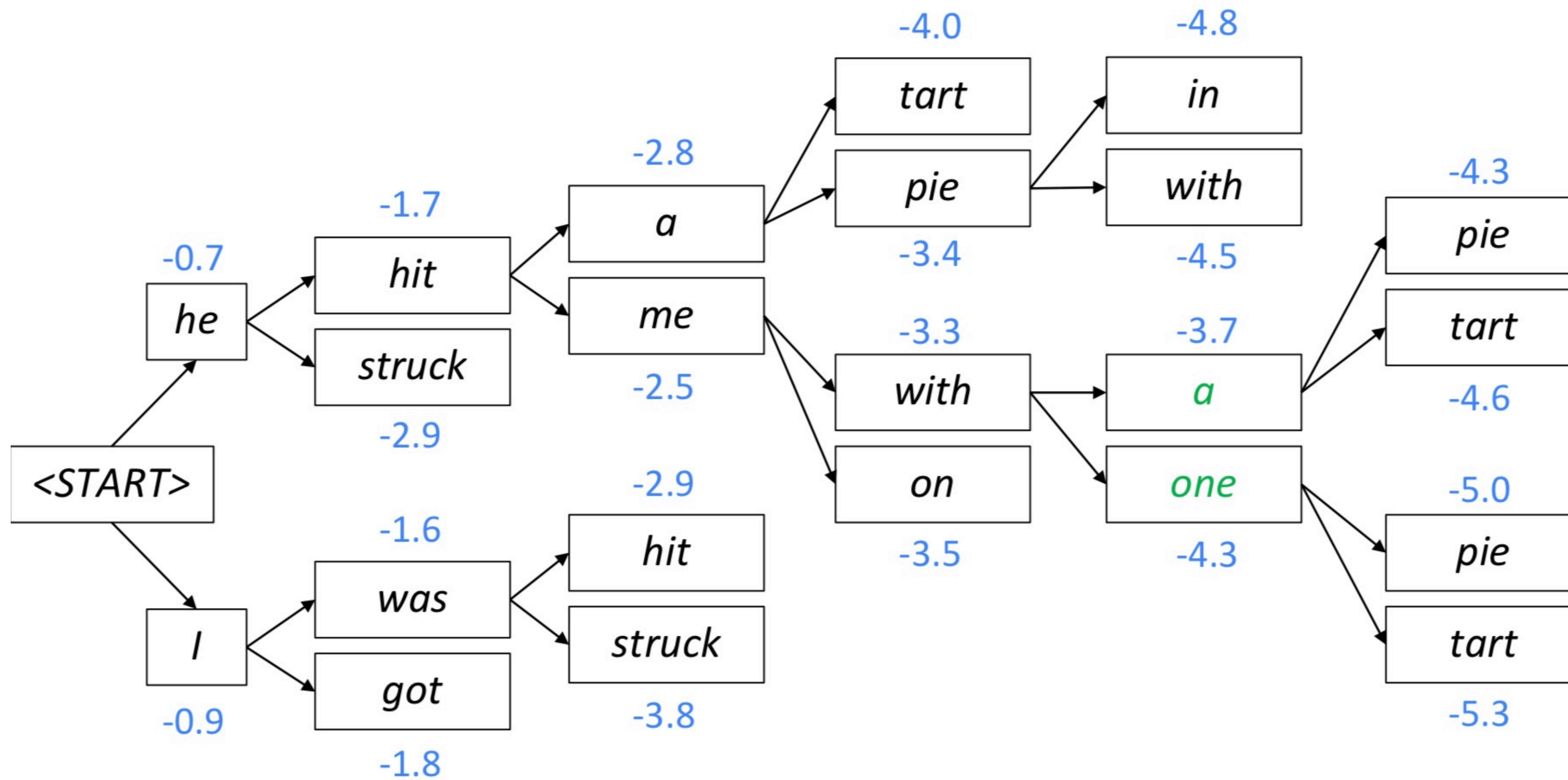
Beam decoding

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



Beam decoding

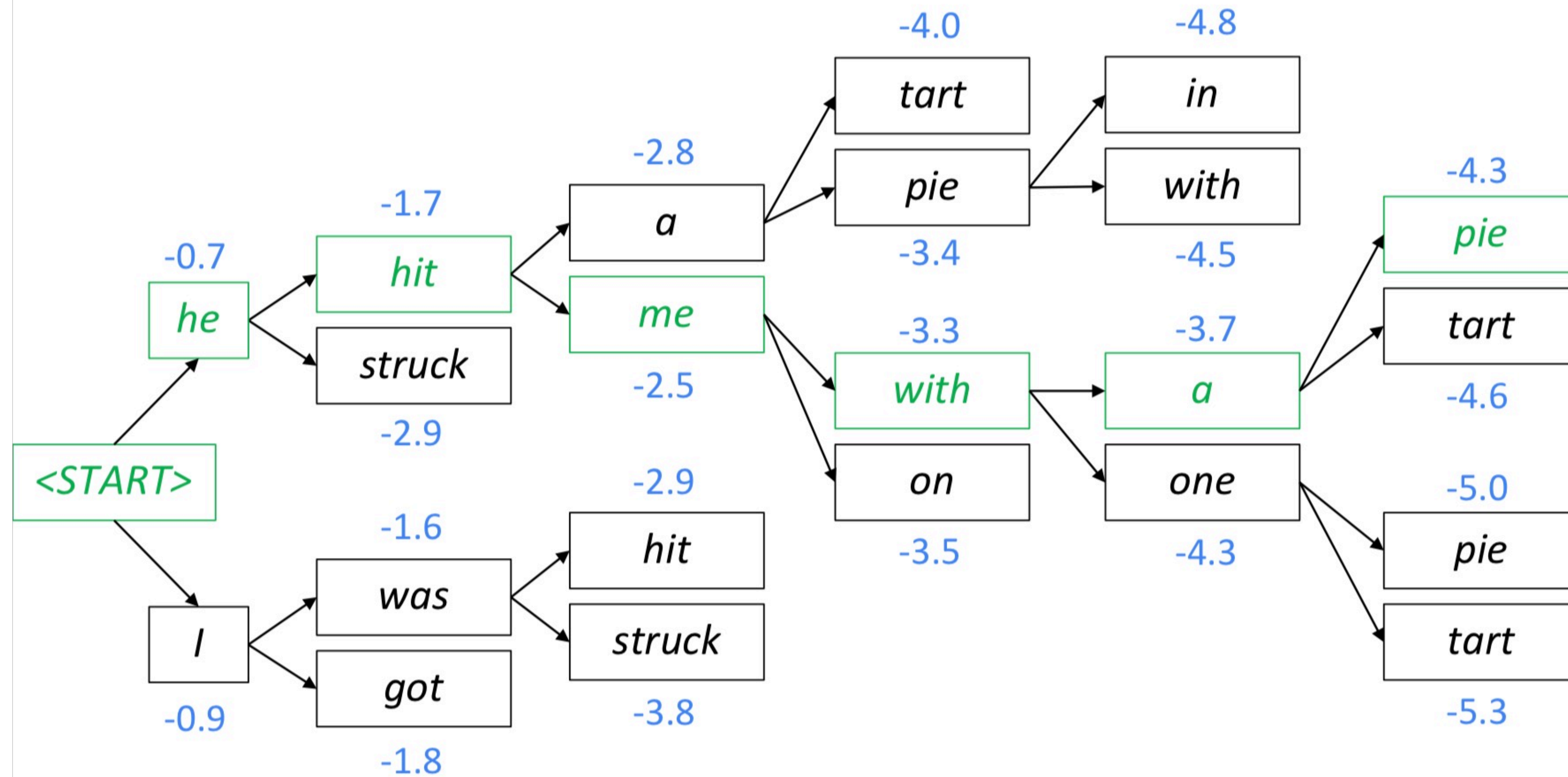
Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



(slide credit: Abigail See)

Backtrack

Beam size = $k = 2$. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



(slide credit: Abigail See)

Beam decoding

- ▶ Different hypotheses may produce $\langle eos \rangle$ (end) token at different time steps
 - ▶ When a hypothesis produces $\langle eos \rangle$, stop expanding it and place it aside
- ▶ Continue beam search until:
 - ▶ All k hypotheses produce $\langle eos \rangle$ OR
 - ▶ Hit max decoding limit T
- ▶ Select top hypotheses using the *normalized* likelihood score

$$\frac{1}{T} \sum_{t=1}^T \log P(y_t | y_1, \dots, y_{t-1}, x_1, \dots, x_n)$$

- ▶ Otherwise shorter hypotheses have higher scores

Evaluating text generation

Evaluating translation quality

- Two main criteria:
- **Adequacy:** Translation $w^{(t)}$ should adequately reflect the linguistic content of $w^{(s)}$
- **Fluency:** Translation $w^{(t)}$ should be fluent text in the target language

	Adequate?	Fluent?
<i>To Vinay it like Python</i>	yes	no
<i>Vinay debugs memory leaks</i>	no	yes
<i>Vinay likes Python</i>	yes	yes

Different translations of *A Vinay le gusta Python*

Evaluation metrics

- Manual evaluation is most accurate, but expensive
- Automated evaluation metrics:
 - Compare system hypothesis with reference translations
 - BiLingual Evaluation Understudy (BLEU) (Papineni et al., 2002)
 - Modified n-gram precision

$$p_n = \frac{\text{number of } n\text{-grams appearing in both reference and hypothesis translations}}{\text{number of } n\text{-grams appearing in the hypothesis translation}}$$

BLEU

$$\text{BLEU-N} = \exp \frac{1}{N} \sum_{n=1}^N \log p_n$$

n-gram precision

geometric mean over several values of n
(up to N=4)

Example

Reference: Vinay likes programming in Python

Hypothesis/Candidate	p_1	p_2	BLEU-2
Vinay likes Python	3/3	1/2	0.7071
To Vinay it like Python	2/5	0	???

<https://www.aclweb.org/anthology/P02-1040.pdf>

BLEU

$$\text{BLEU-N} = \exp \frac{1}{N} \sum_{n=1}^N \log p_n$$

n-gram precision

geometric mean over several values of n
(up to N=4)

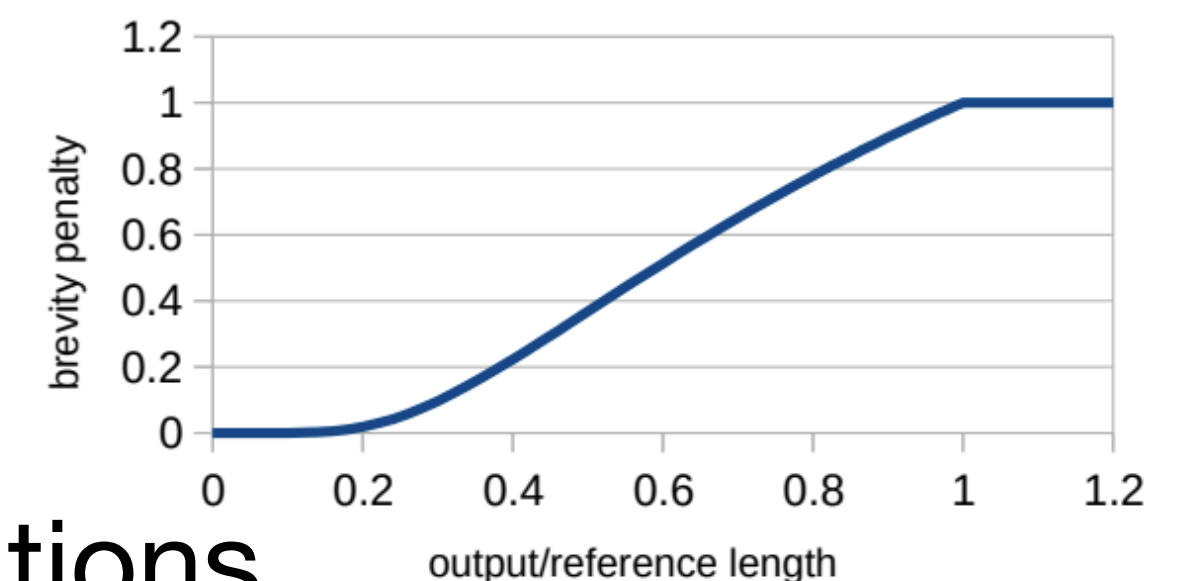
Two modifications:

- To avoid $\log 0$, all precisions are smoothed Various smoothing techniques add 1 to numerator/denominator
- Each n-gram in reference can be used at most once

- Ex. **Hypothesis:** *to to to to to* vs **Reference:** *to be or not to be*
should not get a unigram precision of 1 ($p_1 = 2/5$)

clipped count

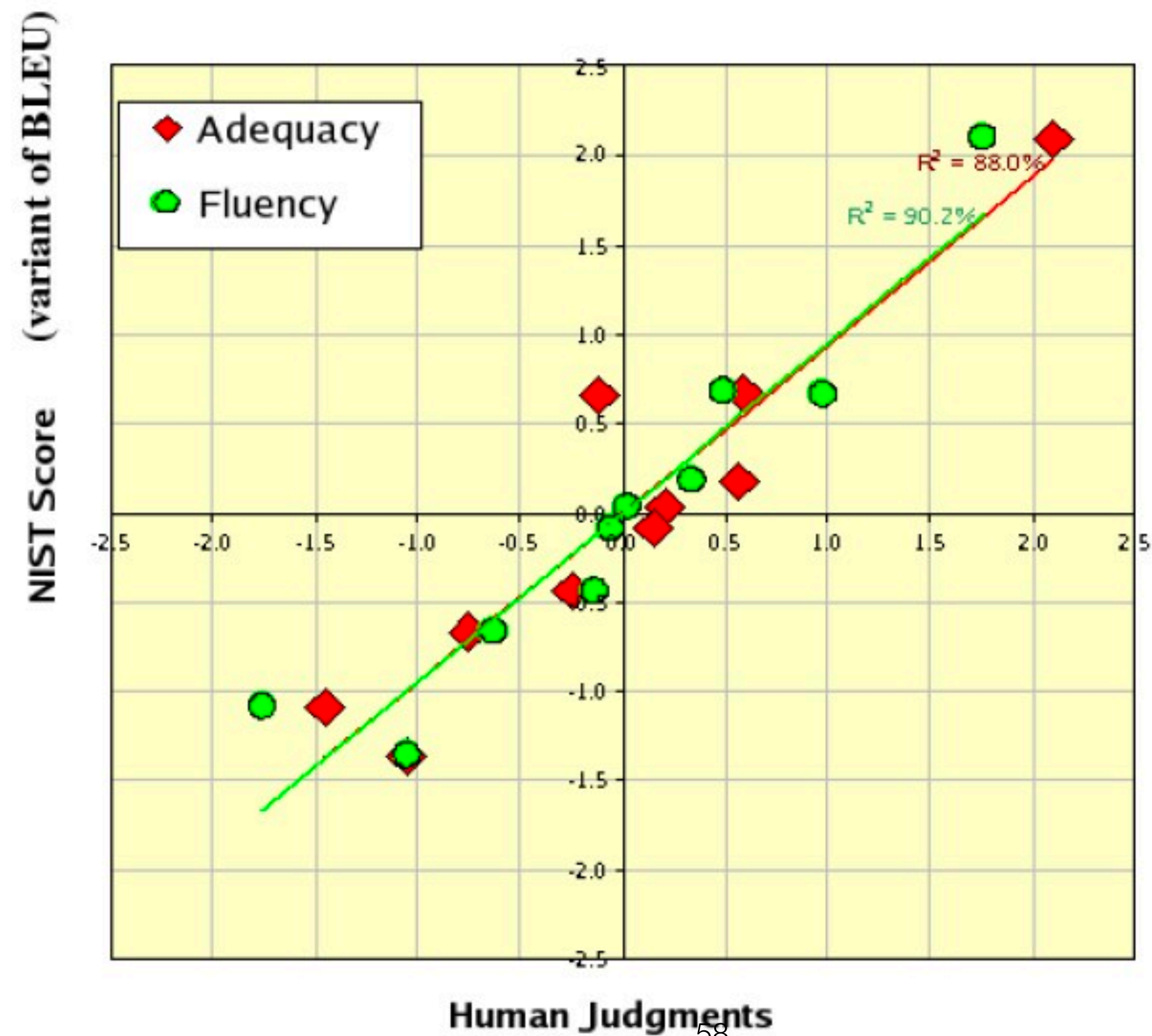
Precision-based metrics favor short translations



- Solution: Multiply score with a **brevity penalty** for translations shorter than reference, $BP = e^{1-r/h}$ $r = \text{reference length}, h = \text{hypothesis length}$

BLEU

- Correlates somewhat well with human judgements



(G. Doddington, NIST)

BLEU scores

https://www.nltk.org/_modules/nltk/translate/bleu_score.html

Sample BLEU scores for various system outputs $BP = e^{1-r/h}$

Length		Translation	p_1	p_2	p_3	p_4	BP	BLEU
5	Reference	<i>Vinay likes programming in Python</i>						
7	Sys1	<i>To Vinay it like to program Python</i>	$\frac{2}{7}$	0	0	0	1	.21
3	Sys2	<i>Vinay likes Python</i>	$\frac{3}{3}$	$\frac{1}{2}$	0	0	.51	.33
6	Sys3	<i>Vinay likes programming in his pajamas</i>	$\frac{4}{6}$	$\frac{3}{5}$	$\frac{2}{4}$	$\frac{1}{3}$	1	.76

Example from: <https://github.com/jacobeisenstein/gt-nlp-class/tree/master/notes>

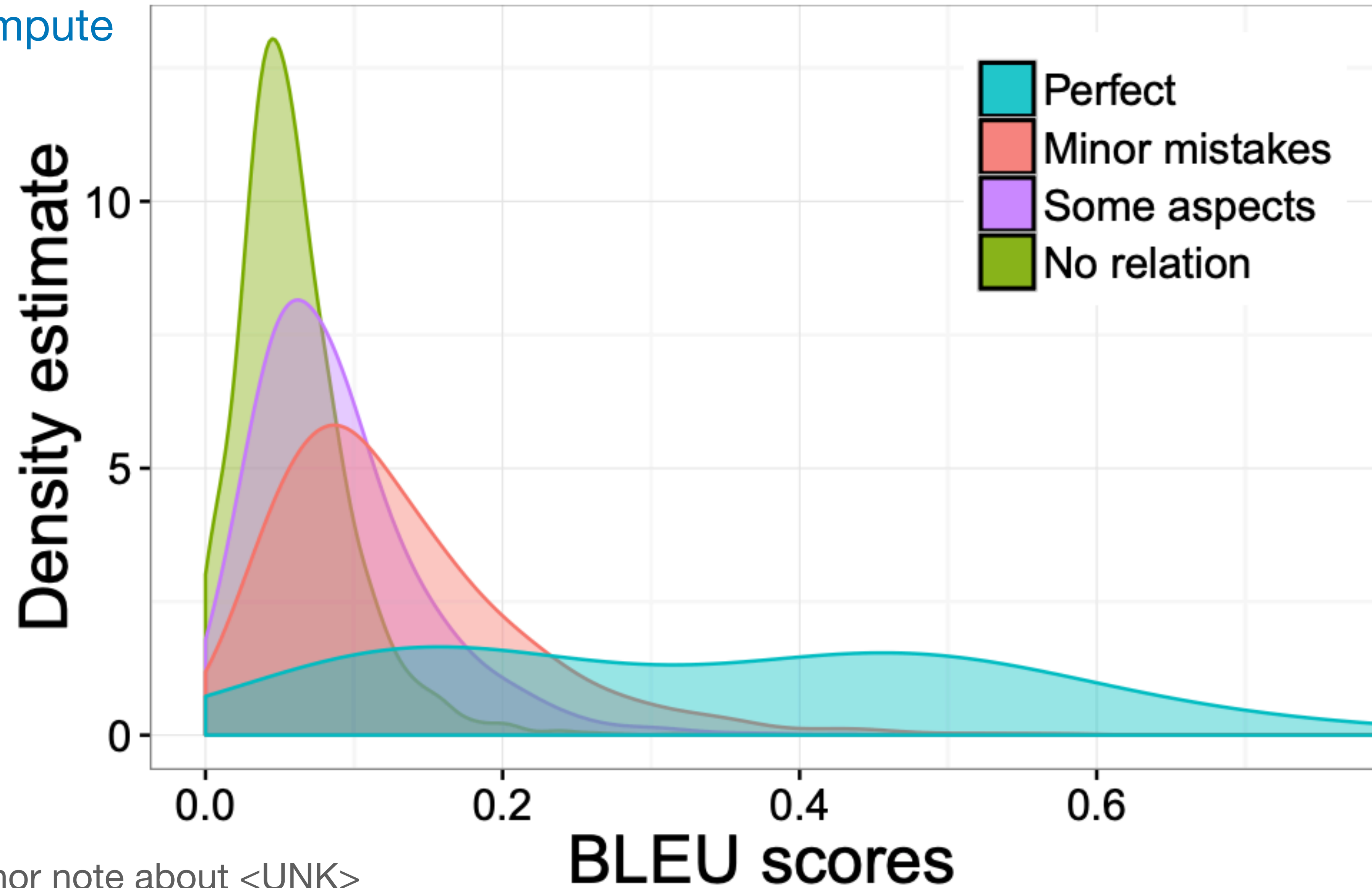
- Alternatives have been proposed:
 - **METEOR**: weighted F-measure
 - Translation Error Rate (**TER**): Edit distance between hypothesis and reference

Issues?

- Number is not that meaningful (BLEU will be higher for some language than others)
- Does not account for different word choices (synonyms)
- Does not account for morphology
- Does not penalize omitting important words

BLEU useful despite issues

- easy to compute
- automated
- consistent



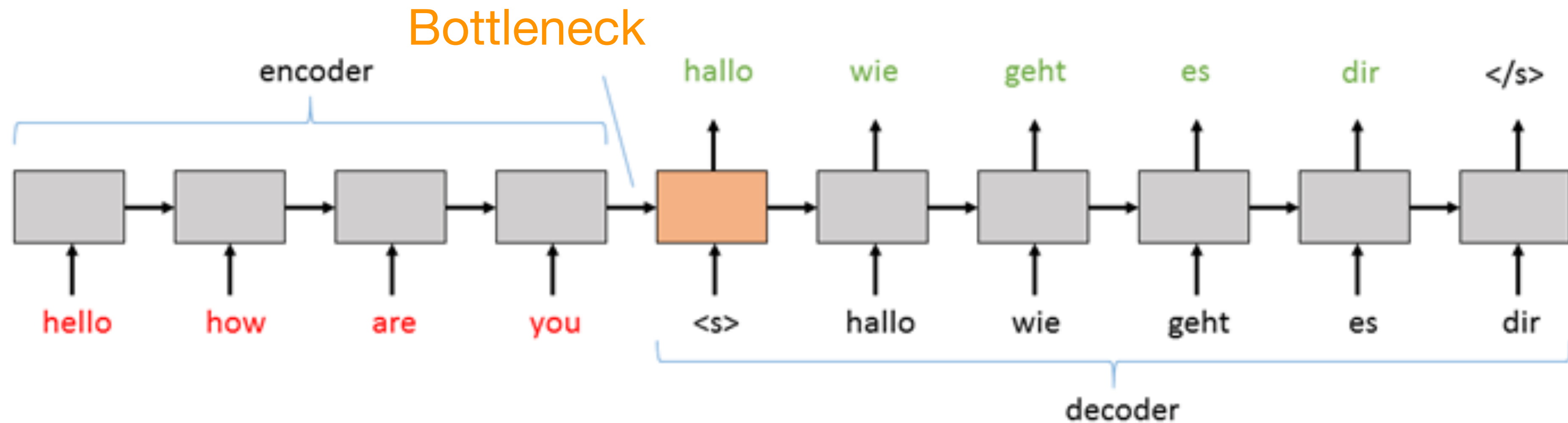
Minor note about <UNK>

Make sure you compare against the original reference
(Don't have <UNK>s in your reference)

Re-evaluating Automatic Metrics for Image Captioning
[Kilickaya et al, EACL 2017]

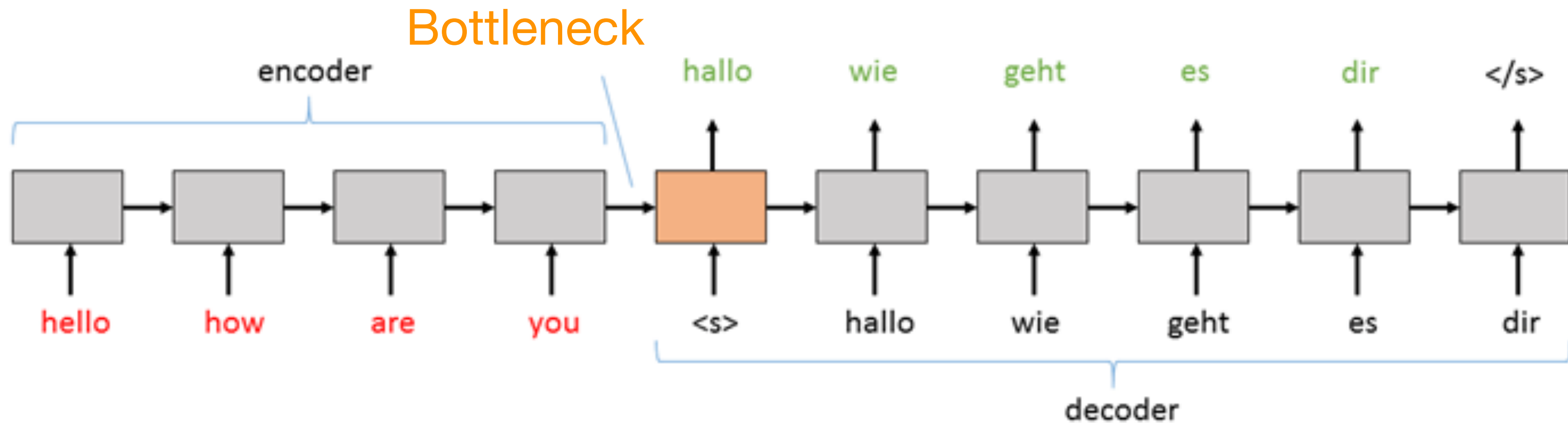
Sequence to sequence models with attention

Issues with vanilla seq2seq



- ▶ A single encoding vector, h^{enc} , needs to capture **all the information** about source sentence
- ▶ Longer sequences can lead to vanishing gradients
- ▶ Overfitting

Issues with vanilla seq2seq

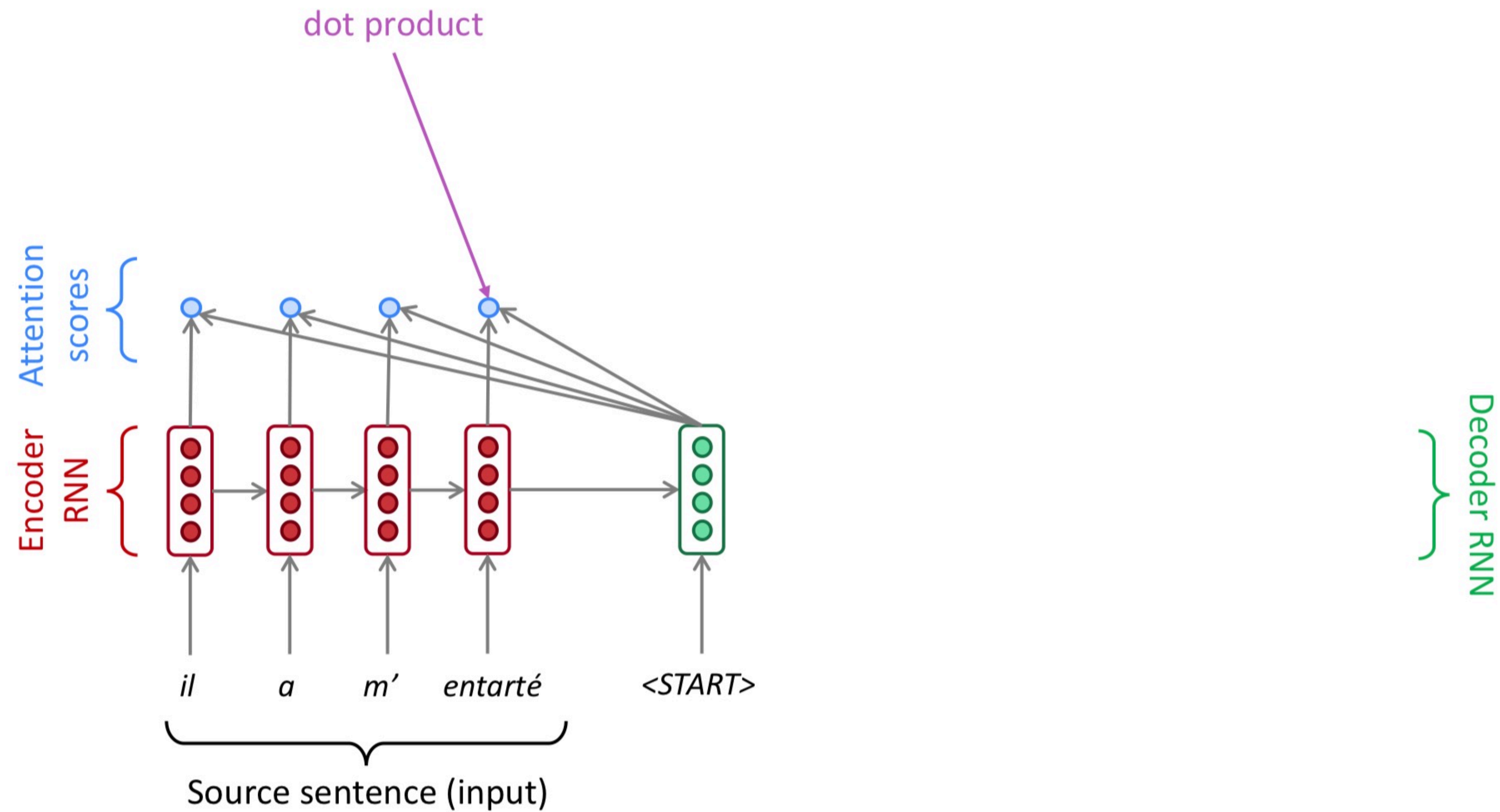


- ▶ A single encoding vector, h^{enc} , needs to capture **all the information** about source sentence
- ▶ **Longer sequences can lead to vanishing gradients**
- ▶ Overfitting

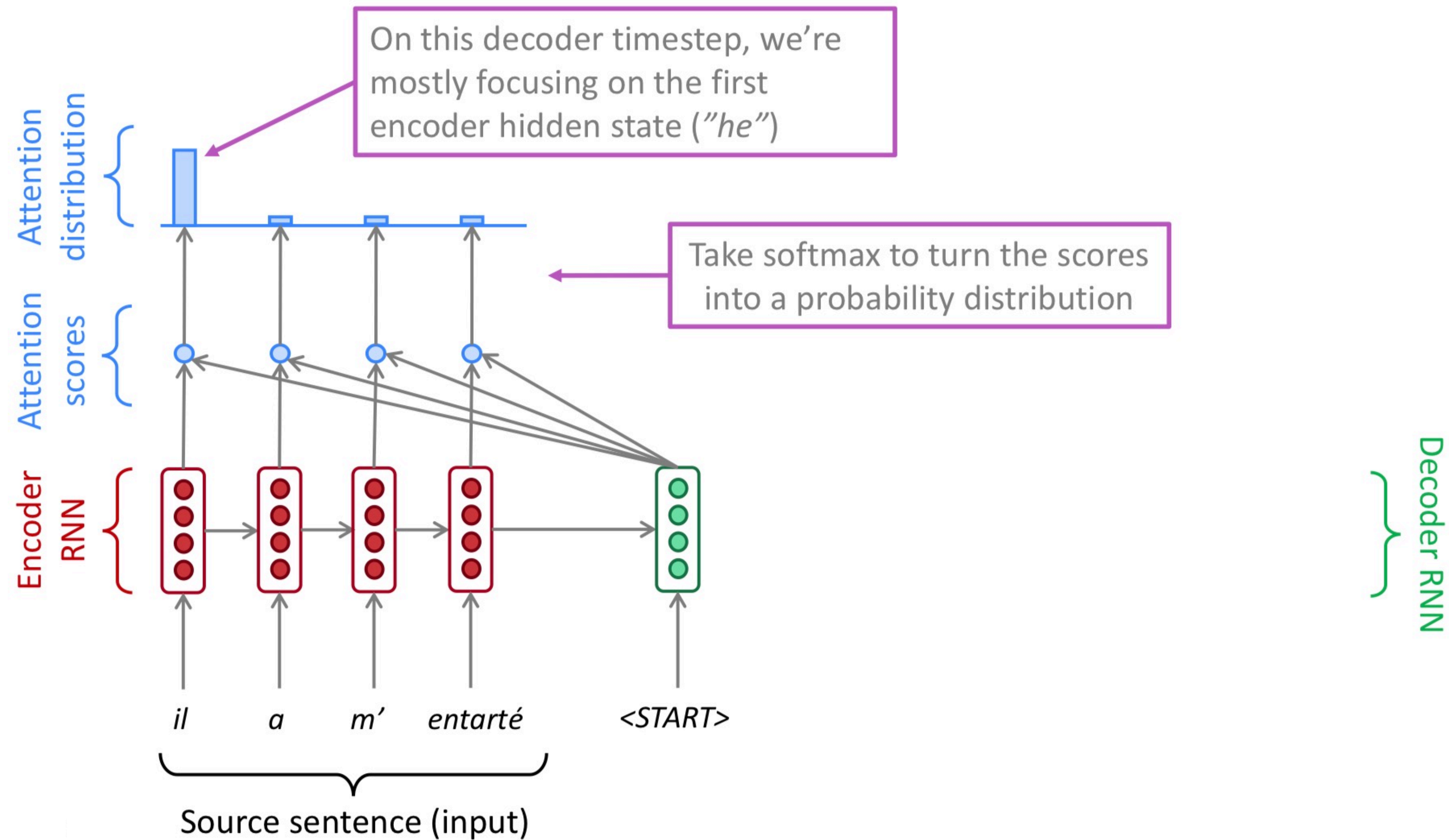
Attention

- ▶ The neural MT equivalent of alignment models
- ▶ **Key idea:** At each time step during decoding, **focus on a particular part** of source sentence
 - ▶ This depends on the **decoder's current hidden state** (i.e. notion of what you are trying to decode)
 - ▶ Usually implemented as a **probability** distribution over the **hidden states of the encoder** (h_i^{enc})

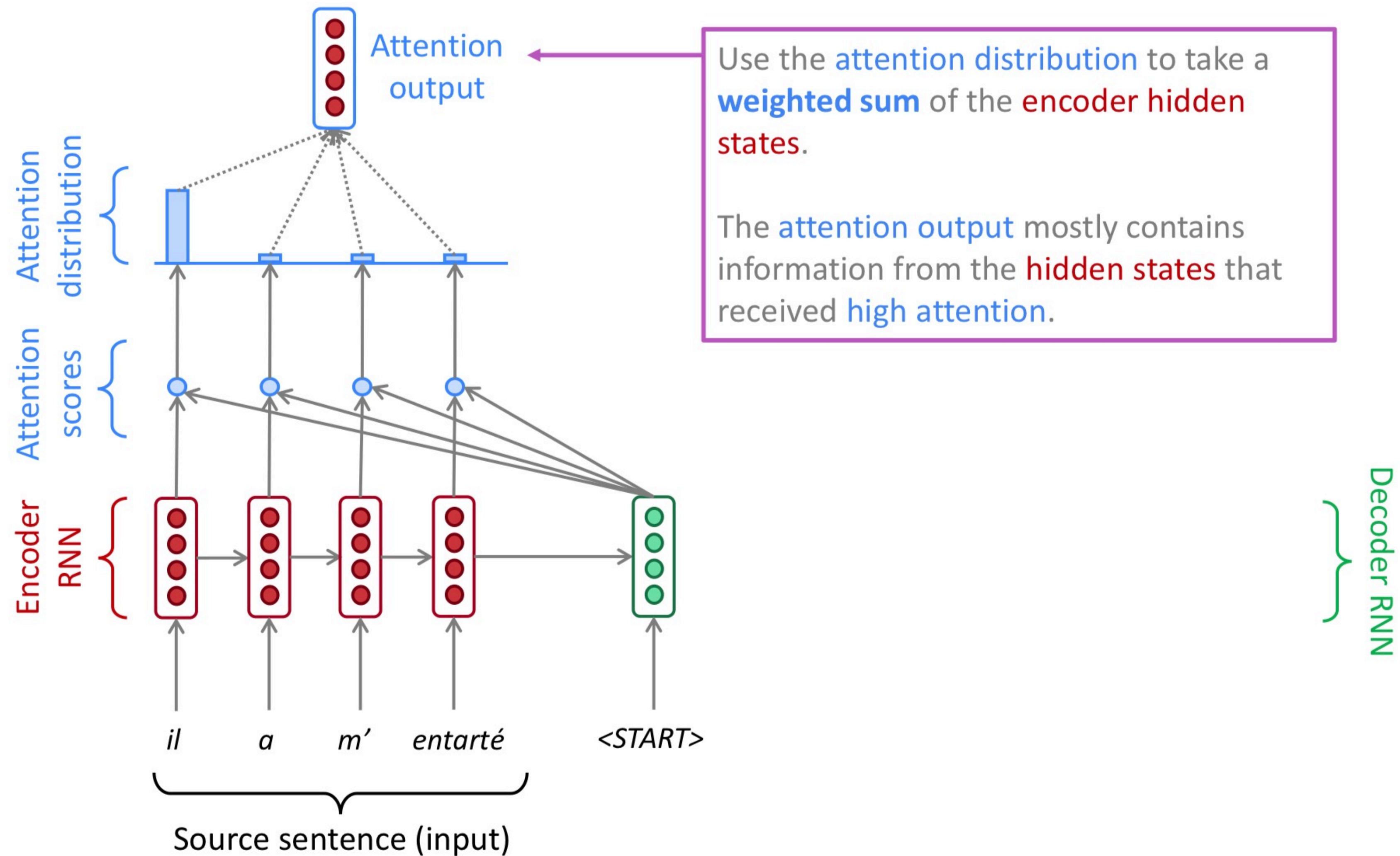
Seq2seq with attention



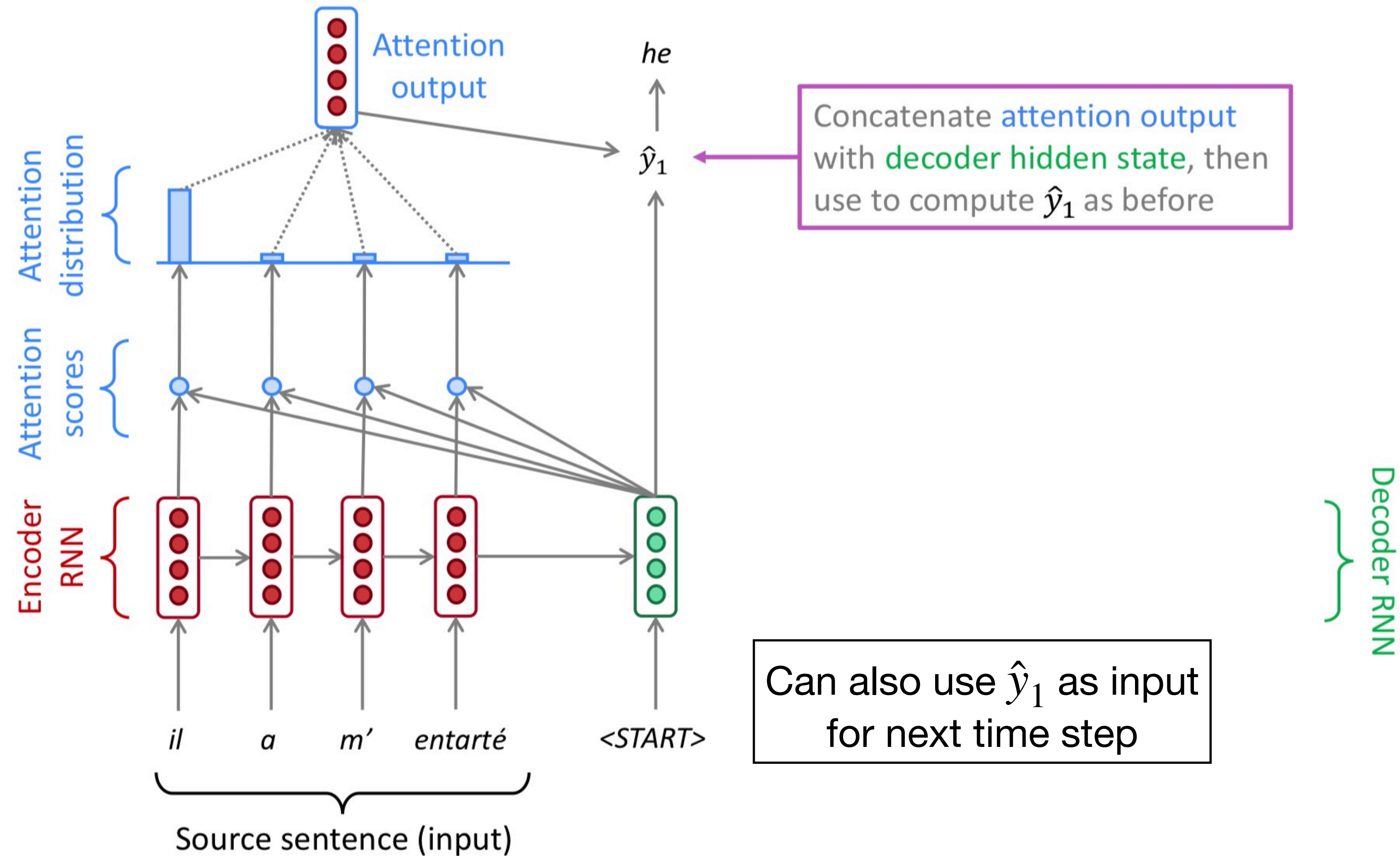
Seq2seq with attention



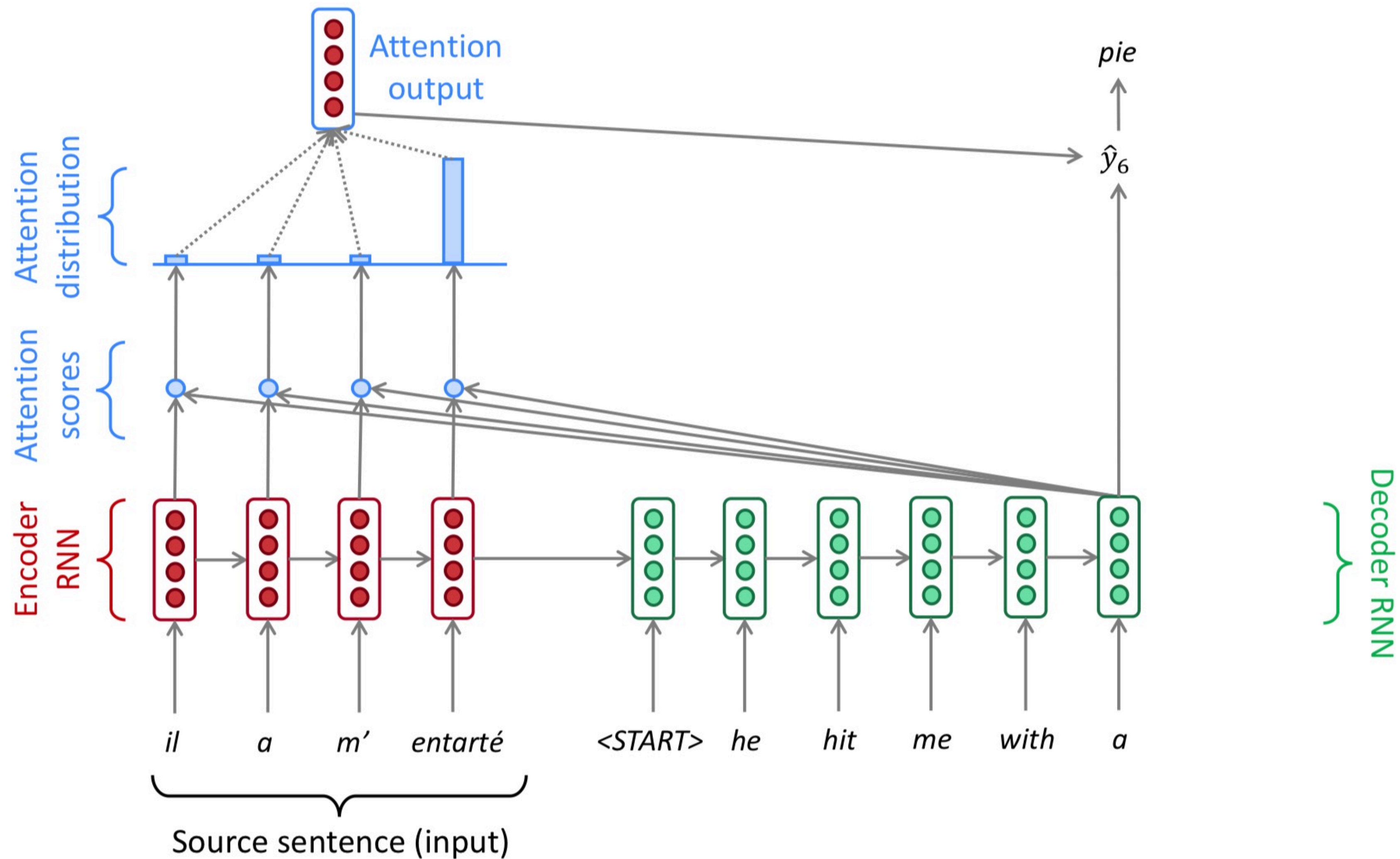
Seq2seq with attention



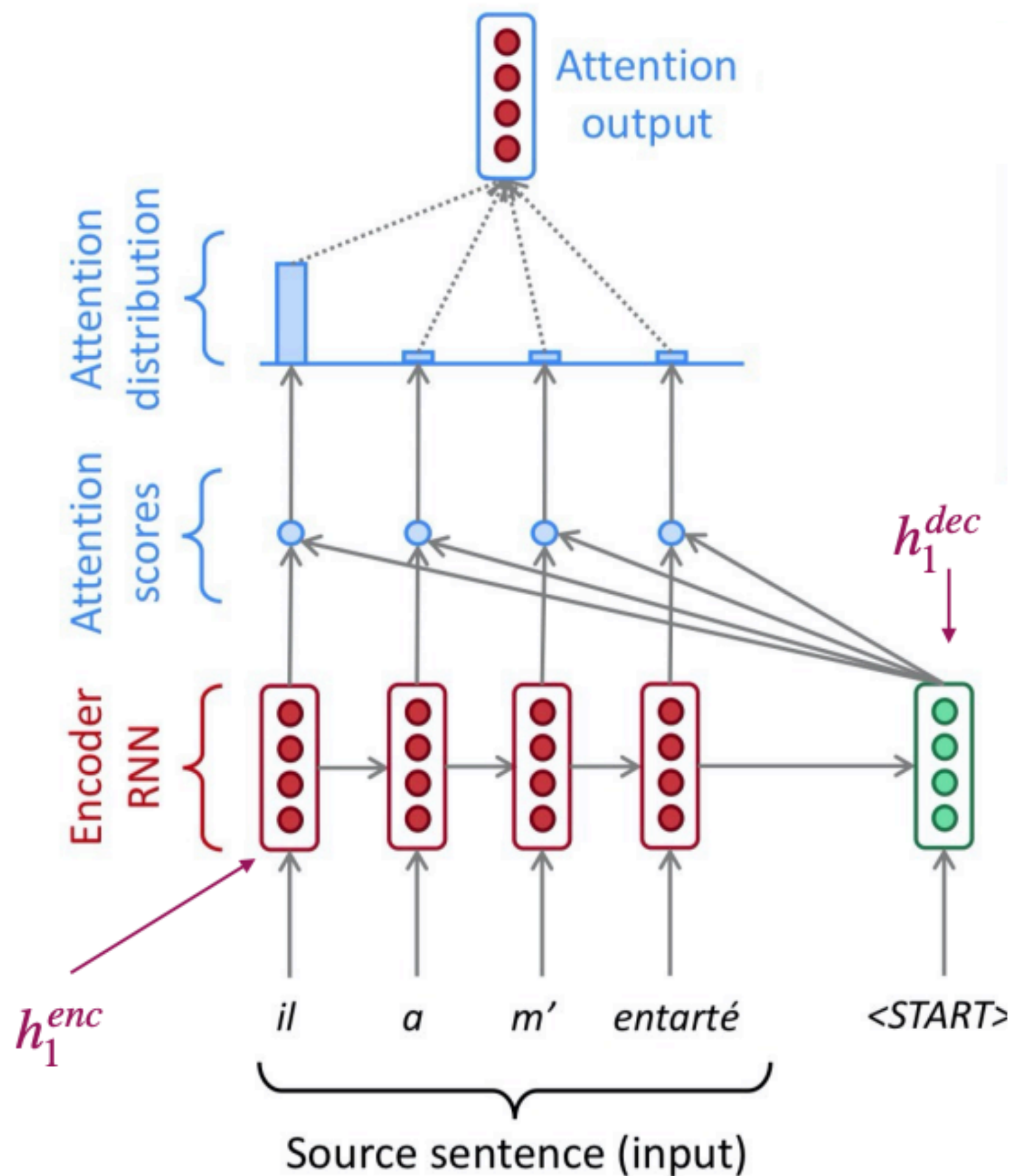
Seq2seq with attention



Seq2seq with attention



Computing attention



- ▶ Encoder hidden states: $h_1^{enc}, \dots, h_n^{enc}$
- ▶ Decoder hidden state at time t : h_t^{dec}
- ▶ First, get attention scores for this time step (we will see what g is soon!):

$$e^t = [g(h_1^{enc}, h_t^{dec}), \dots, g(h_n^{enc}, h_t^{dec})]$$

- ▶ Obtain the attention distribution using softmax:

$$\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^n$$

- ▶ Compute weighted sum of encoder hidden states:

$$a_t = \sum_{i=1}^n \alpha_i^t h_i^{enc} \in \mathbb{R}^h$$

- ▶ Finally, concatenate with decoder state and pass on to output layer:

$$[a_t; h_t^{dec}] \in \mathbb{R}^{2h}$$

Types of attention

- ▶ Assume encoder hidden states $h_1, h_2, \dots, h_n$ and decoder hidden state z

1. **Dot-product attention:**

$$g(h_i, z) = z^T h_i \in \mathbb{R}$$

Simplest (no extra parameters)
requires z and h_i to be same size

more efficient
(matrix
multiplication)

2. **Bilinear / multiplicative attention:**

$$g(h_i, z) = z^T W h_i \in \mathbb{R}, \text{ where } W \text{ is a weight matrix}$$

More flexible
than dot-product
(W is trainable)

3. **Additive attention (essentially MLP):**

$$g(h_i, z) = v^T \tanh(W_1 h_i + W_2 z) \in \mathbb{R}$$

where W_1, W_2 are weight matrices and v is a weight vector

Perform better for
larger dimensions

Attention can be applied to other modalities

Attention on other modalities

- Images

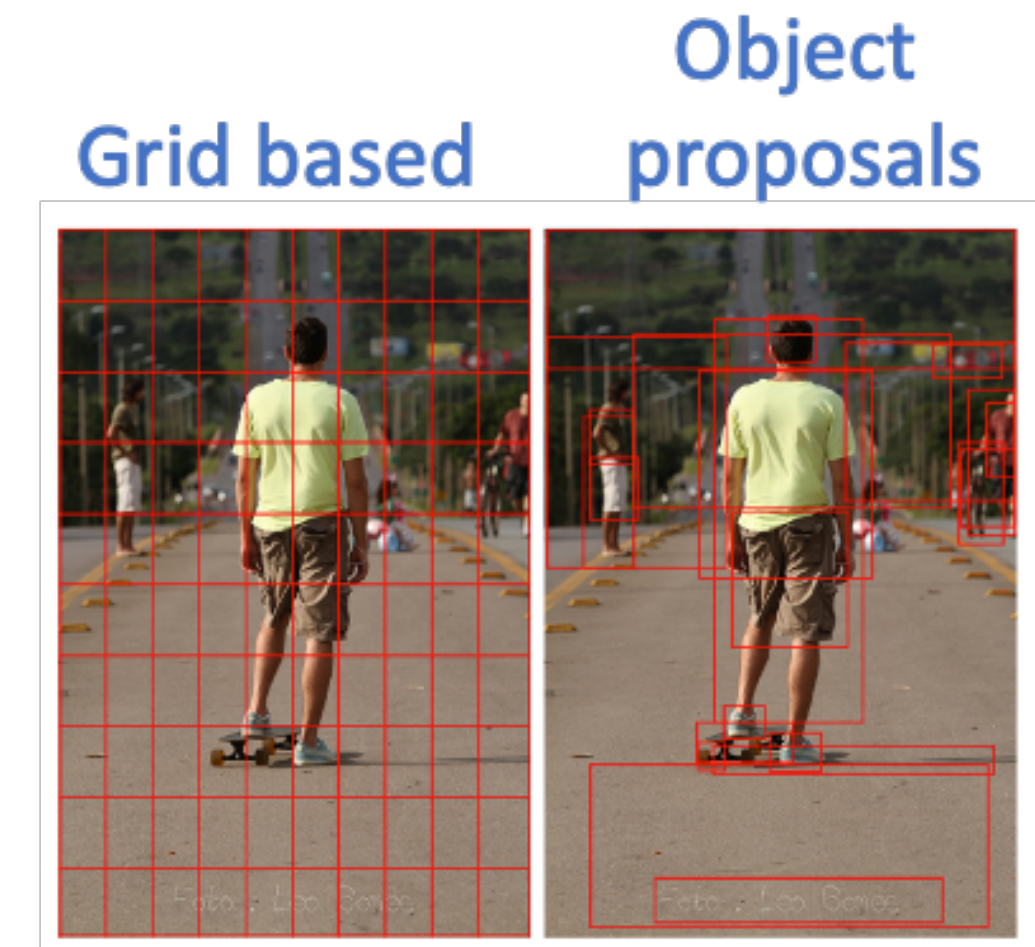
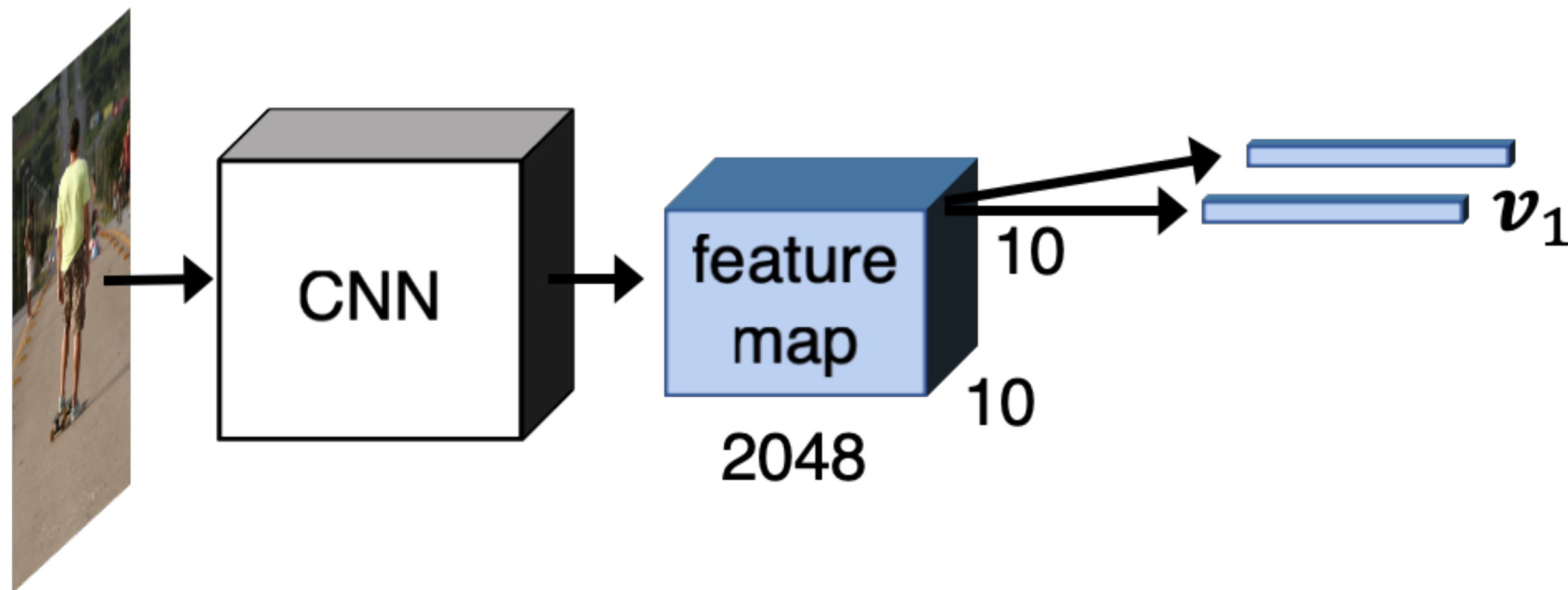
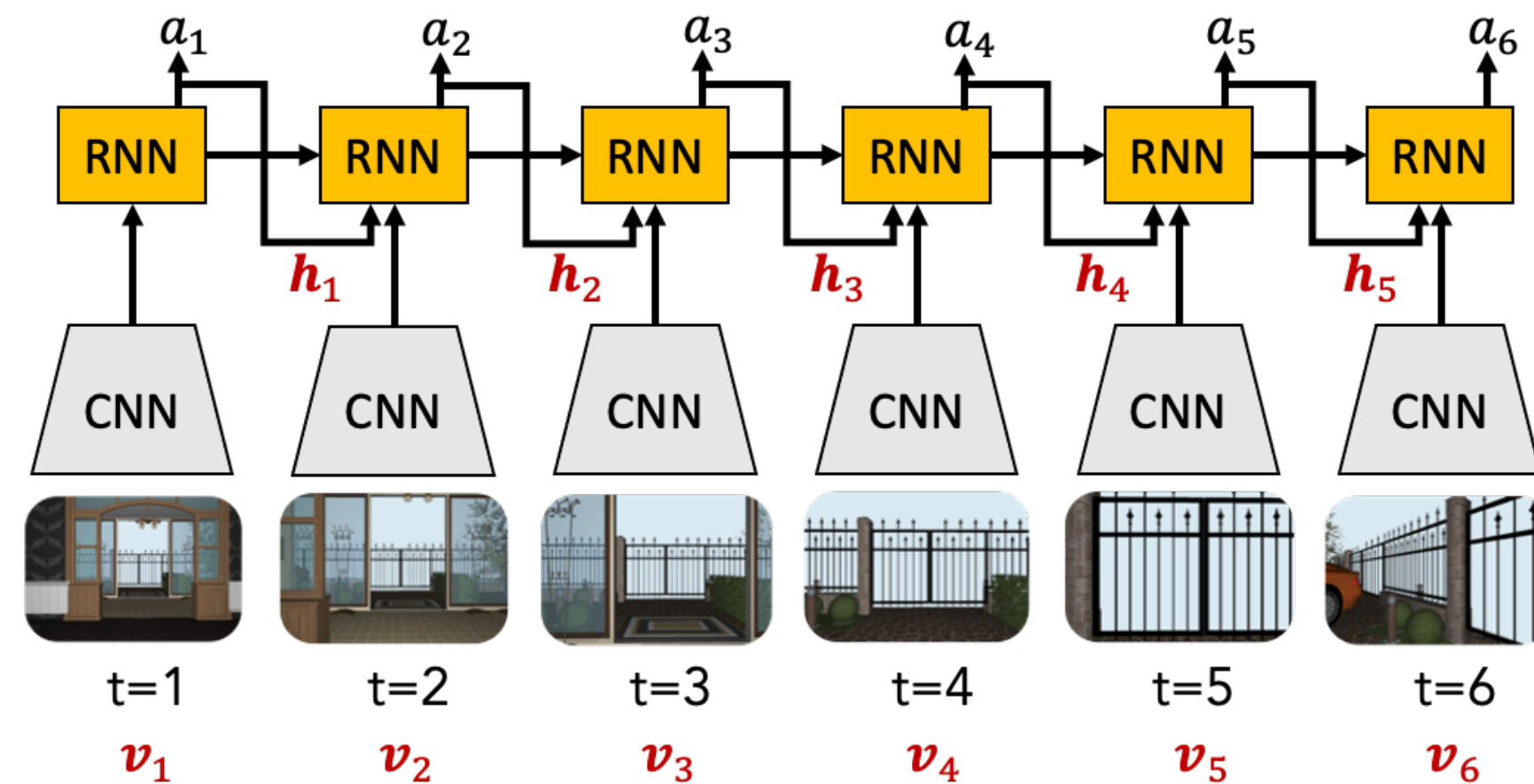


Image Credit: Peter Anderson

- Agent experience

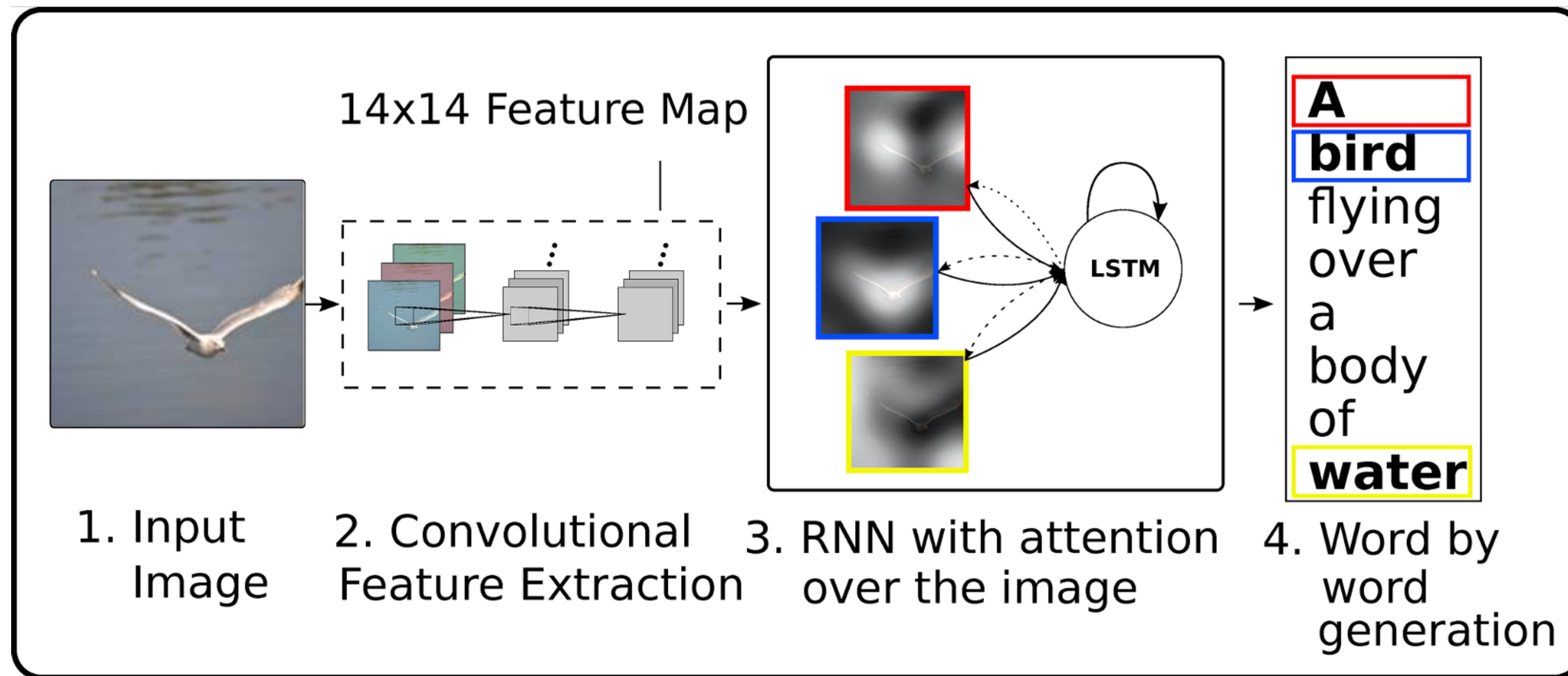


$$C = \{h_1, \dots, h_5\}$$

or

$$C = \{v_1, \dots, v_6\}$$

Image captioning example



Xu et al. ICML 2015

Soft vs Hard Attention

- **Soft:** Each attention candidate is weighted by α_i

$$\hat{\mathbf{v}} = \sum_{i=1}^k \alpha_i \mathbf{v}_i$$

- Easy to train (smooth and differentiable)
- But can be expensive over large input

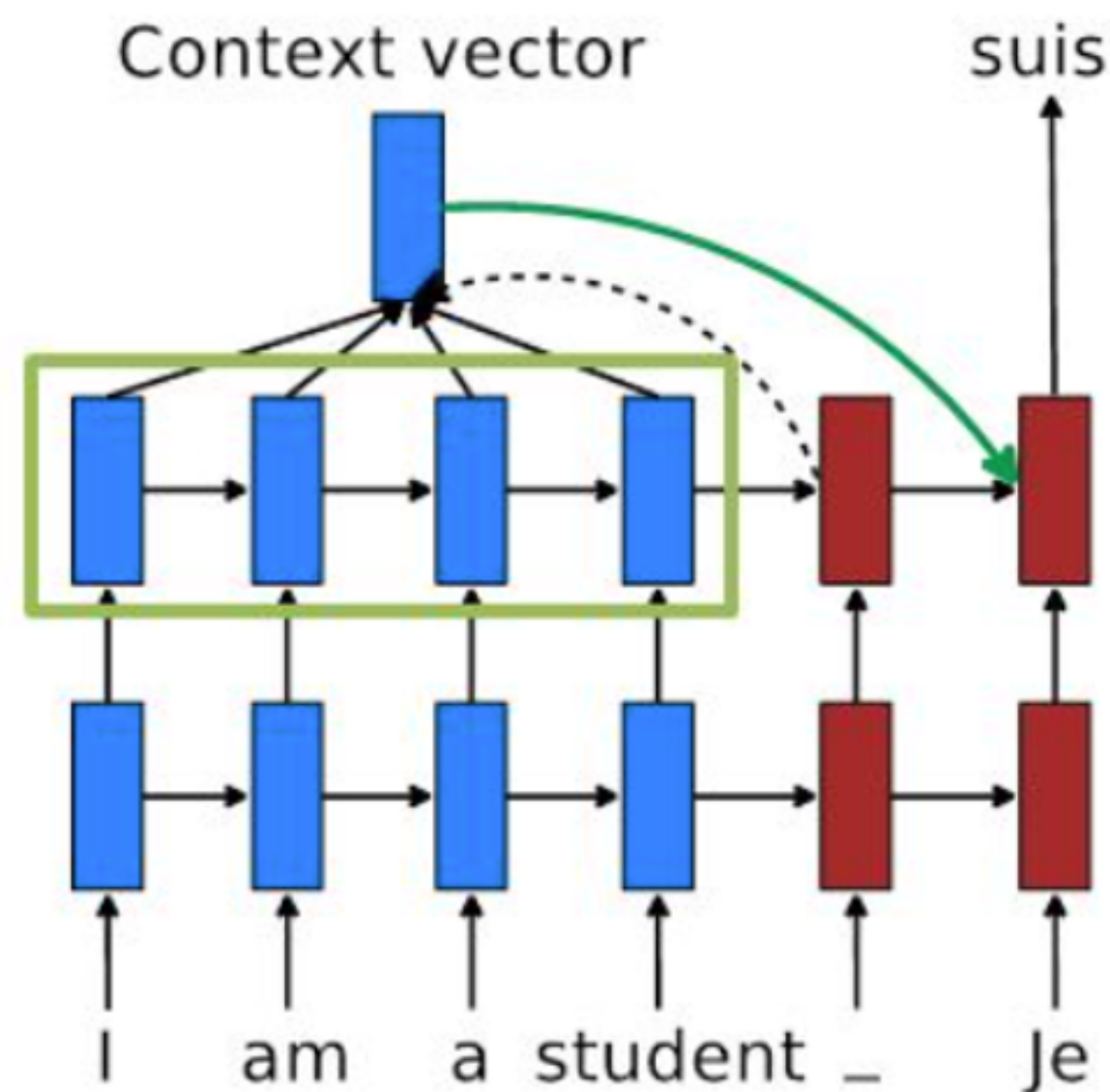
- **Hard:** Use α_i as a sample probability to pick *one* attention candidate as input to subsequent layers
 - Trainable with REINFORCE approaches (Xu et al. ICML 2015), or Gumbel-Softmax (Jang et al. ICLR 2017)



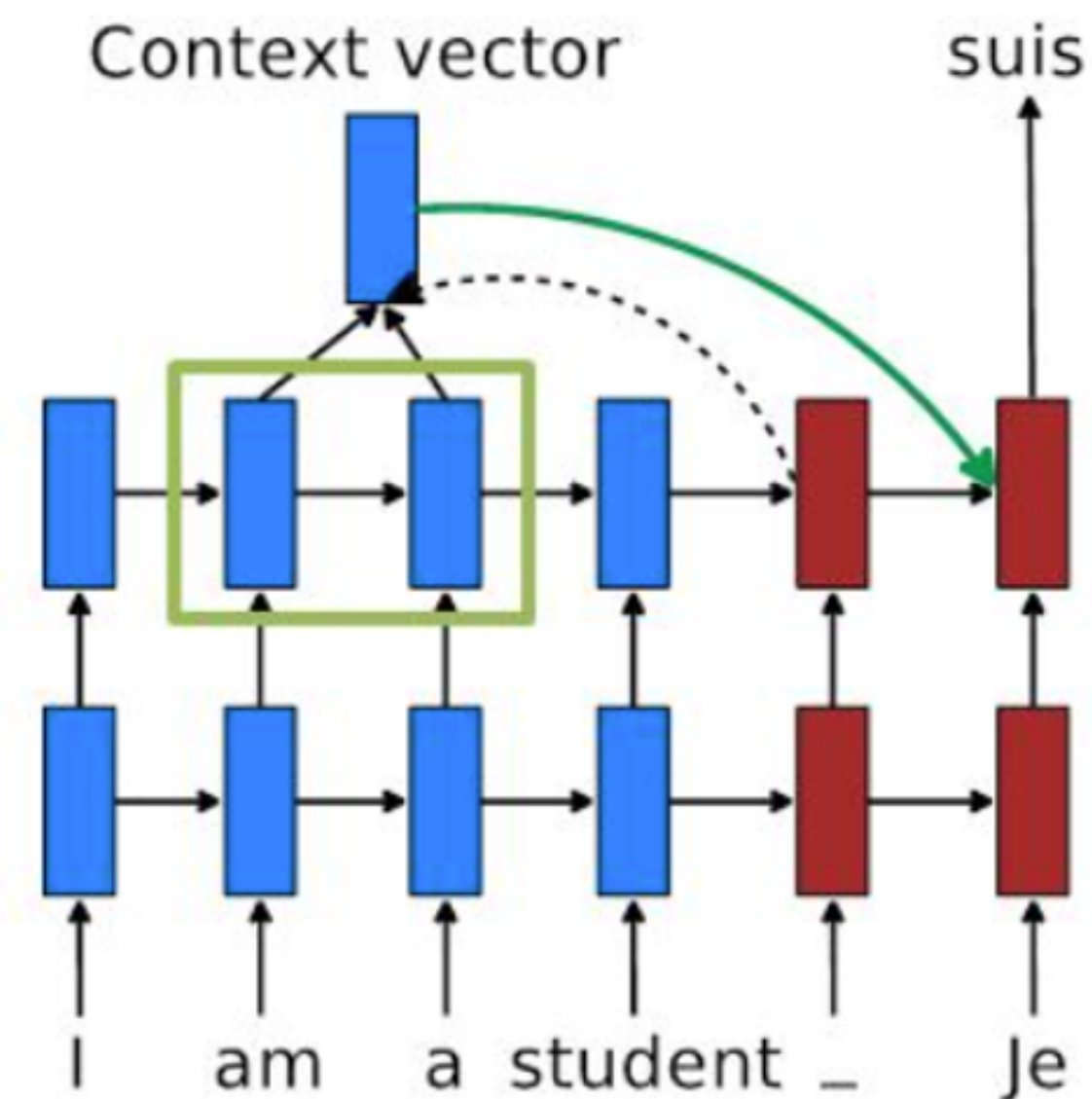
Xu et al. ICML 2015

Global vs Local Attention

- **Global**: attention over the entire input
- **Local**: attention over a window (or subset) of the input



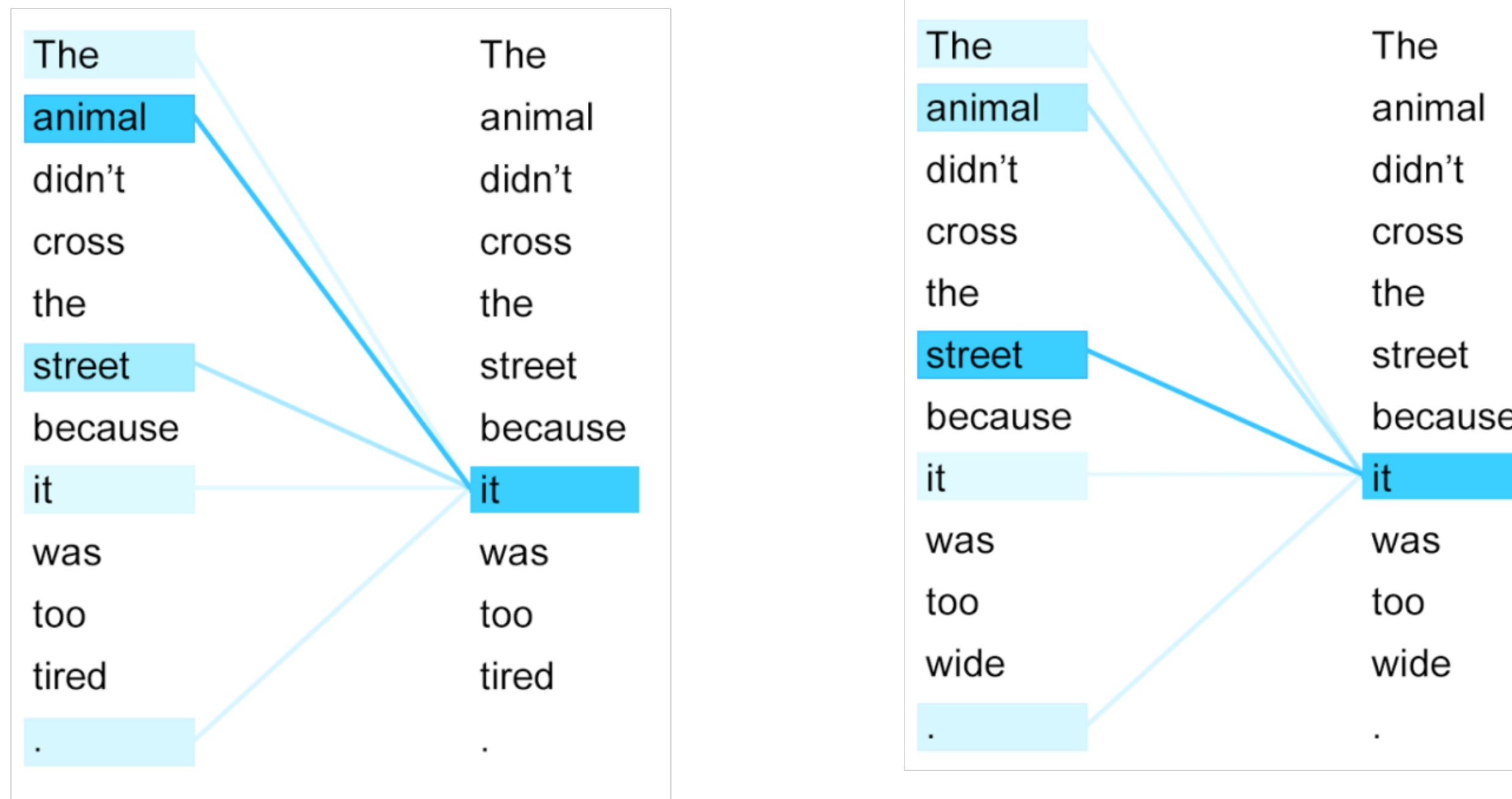
Global: ***all*** source states.



Local: ***subset*** of source states.

Self-Attention

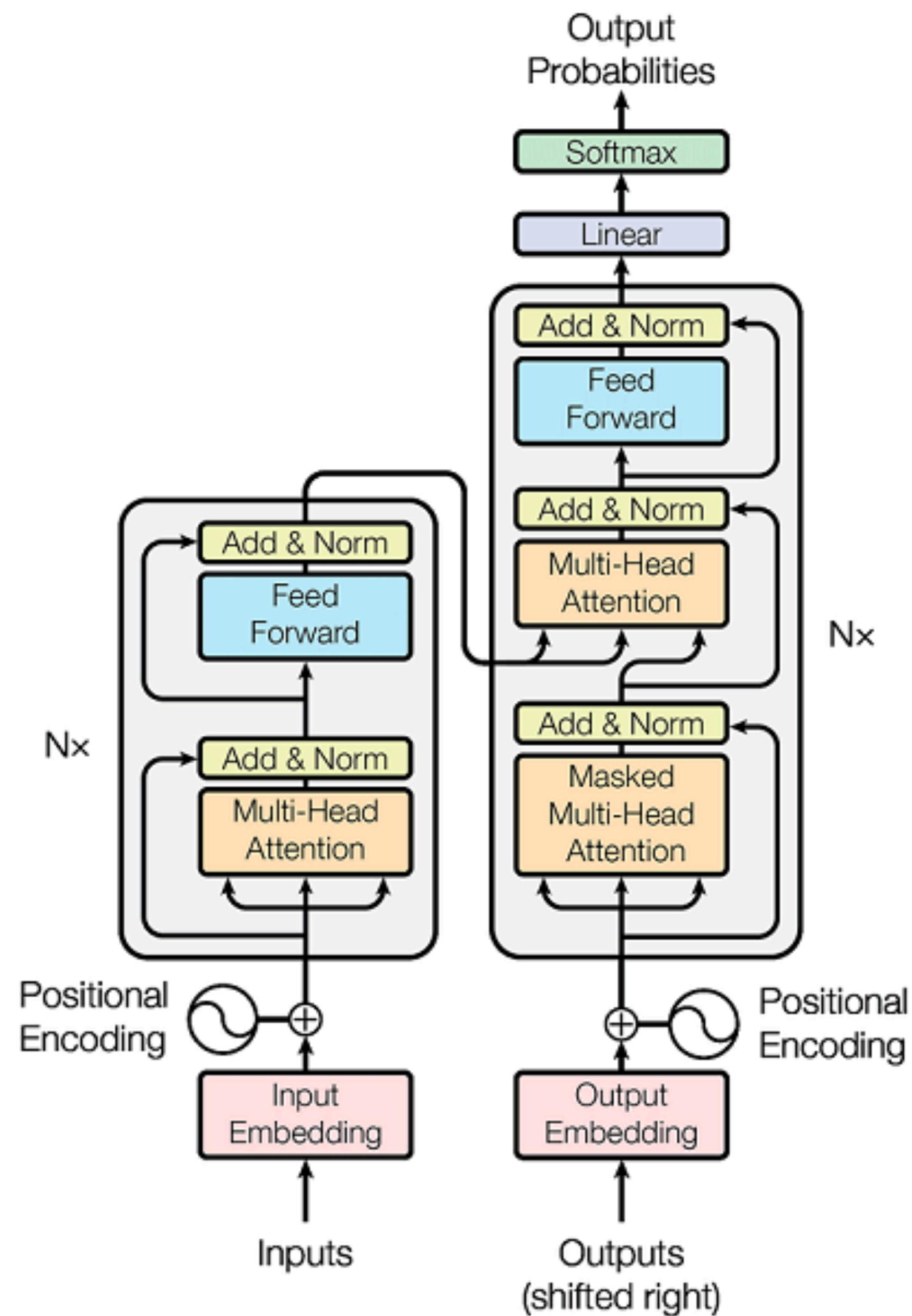
- Attention (correlation) with different parts of itself



<https://ai.googleblog.com/2017/08/transformer-novel-neural-network.html>

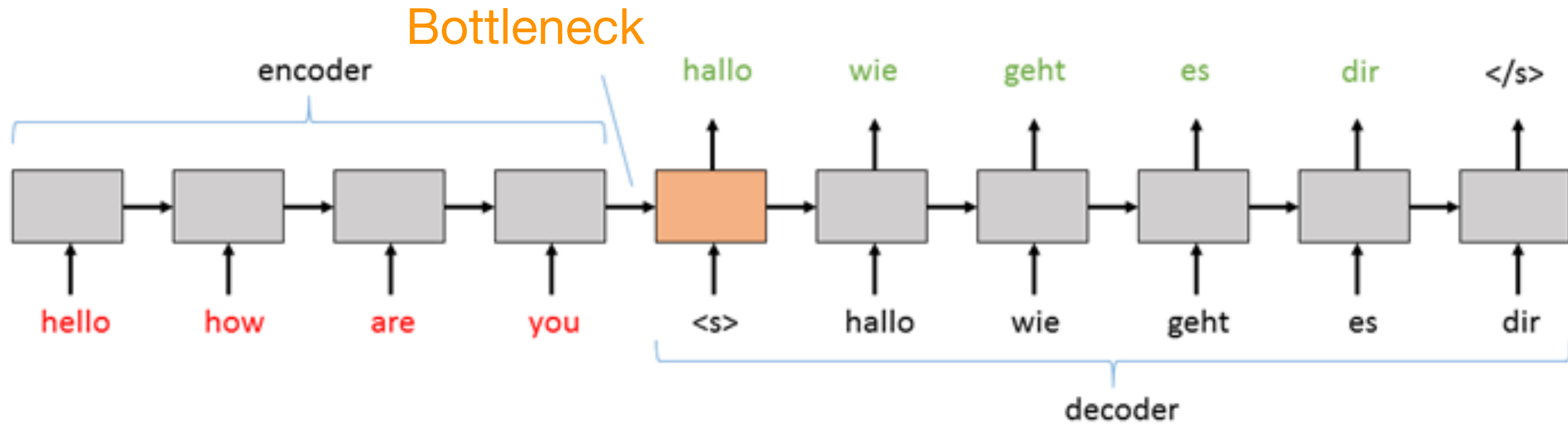
- Transformers: modules with scaled dot-product self-attention

Transformers: self-attention

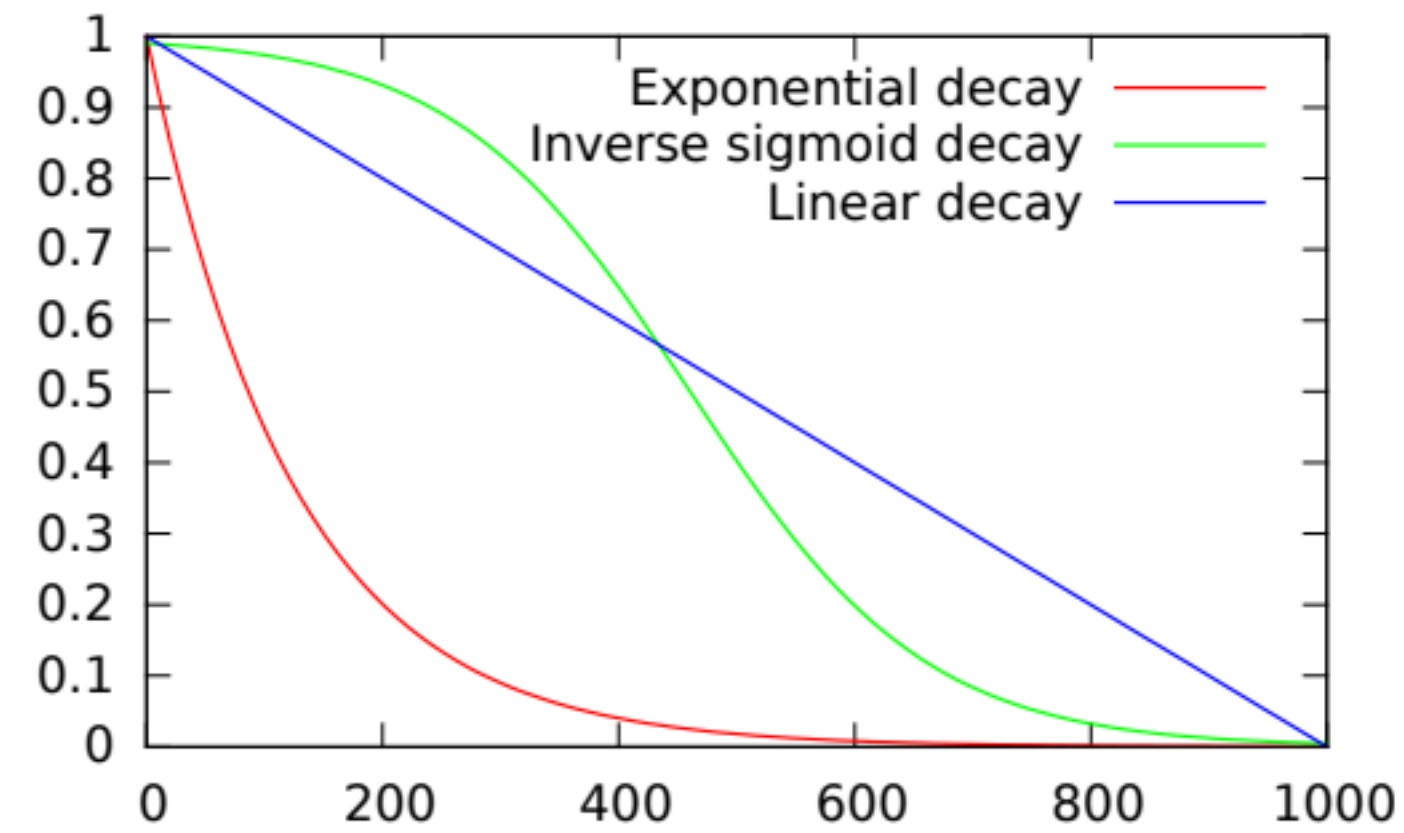


- More recent models (e.g. Transformer, Vaswani et al., 2017) have replaced RNNs entirely with attention mechanisms
- Theoretically limiting (since recurrence can help handle arbitrarily long sequences)
- Huge gains in practical performance

Issues with vanilla seq2seq

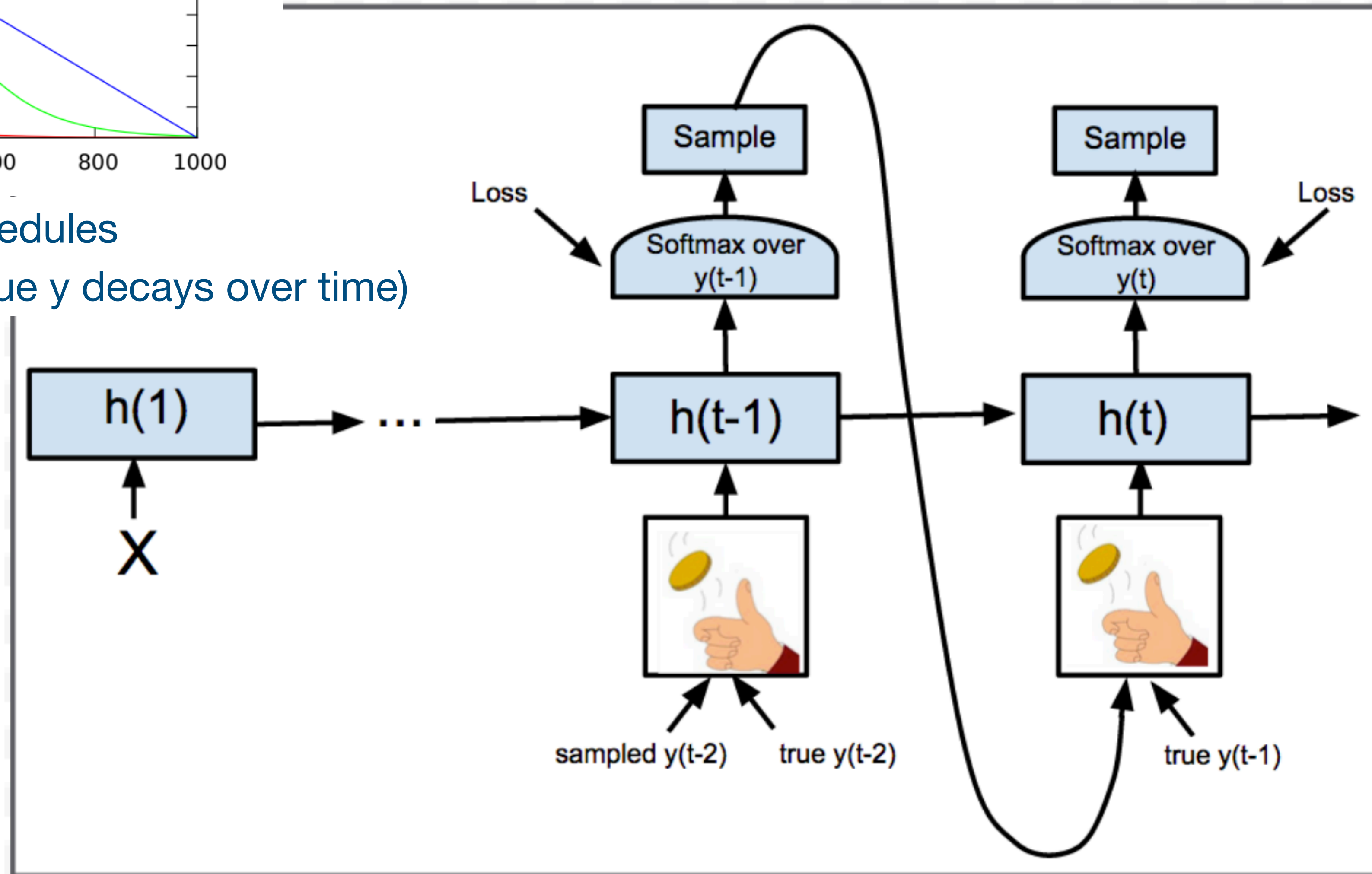


- ▶ A single encoding vector, h^{enc} , needs to capture **all the information** about source sentence
- ▶ Longer sequences can lead to vanishing gradients
- ▶ **Overfitting**



Possible decay schedules
(probability using true y decays over time)

Scheduled Sampling



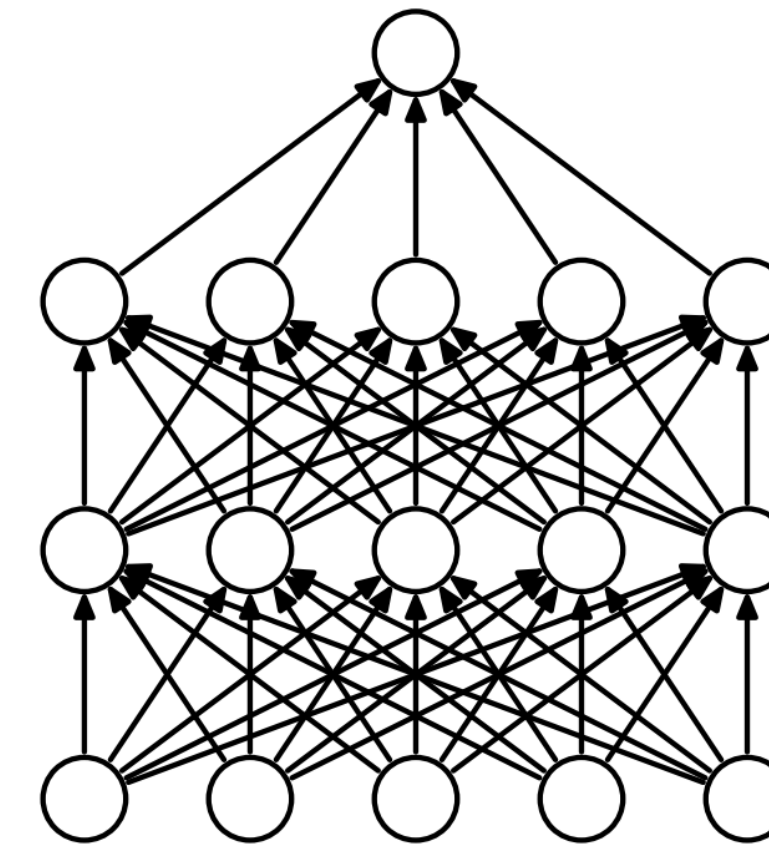
(figure credit: Bengio et al, 2015)

Regularization

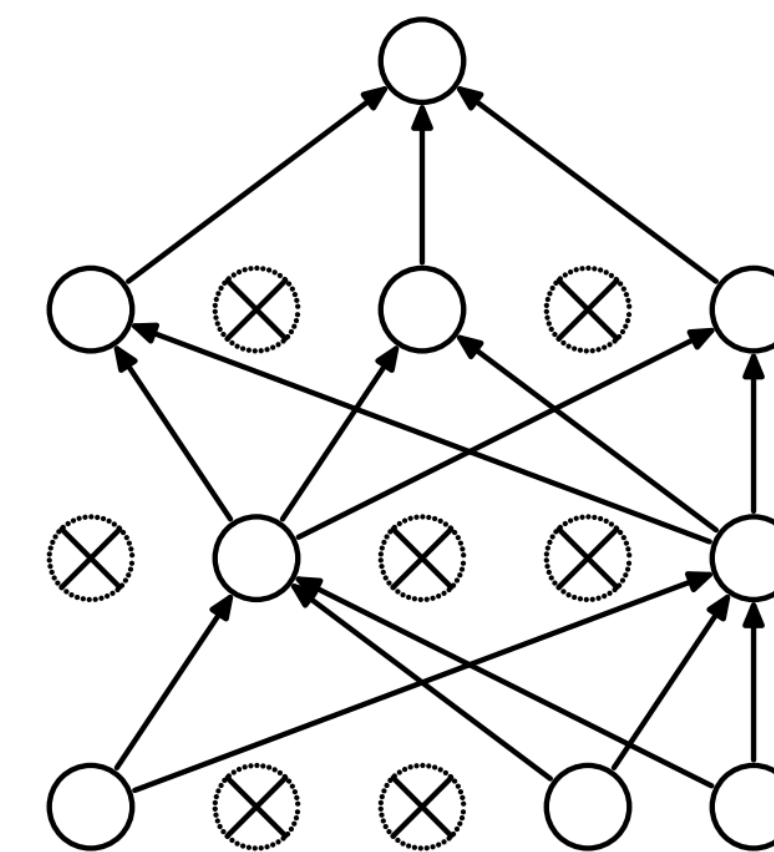
- Weight Decay
- Dropout
- Ensembling

Regularization: Dropout

- ▶ Form of regularization for RNNs (and any NN in general)
- ▶ **Idea:** “Handicap” NN by removing hidden units stochastically
 - ▶ set each hidden unit in a layer to 0 with probability p during training ($p = 0.5$ usually works well)
 - ▶ scale outputs by $1/(1 - p)$
 - ▶ hidden units forced to learn more general patterns
- ▶ **Test time:** Use all activations (no need to rescale)



(a) Standard Neural Net



(b) After applying dropout.

Dropout and attention improves translation

System	Ppl	BLEU
Winning WMT'14 system – <i>phrase-based</i> + <i>large LM</i> (Buck et al., 2014)		20.7
<i>Existing NMT systems</i>		
RNNsearch (Jean et al., 2015)		16.5
RNNsearch + unk replace (Jean et al., 2015)		19.0
RNNsearch + unk replace + large vocab + <i>ensemble</i> 8 models (Jean et al., 2015)		21.6
<i>Our NMT systems</i>		
Base	10.6	11.3
Base + reverse	9.9	12.6 (+1.3)
Base + reverse + dropout	8.1	14.0 (+1.4)
Base + reverse + dropout + global attention (<i>location</i>)	7.3	16.8 (+2.8)
Base + reverse + dropout + global attention (<i>location</i>) + feed input	6.4	18.1 (+1.3)
Base + reverse + dropout + local-p attention (<i>general</i>) + feed input	5.9	19.0 (+0.9)
Base + reverse + dropout + local-p attention (<i>general</i>) + feed input + unk replace		20.9 (+1.9)
Ensemble 8 models + unk replace		23.0 (+2.1)

WMT'14 English to German Results

(Luong et al, 2015)

Other challenges with NMT

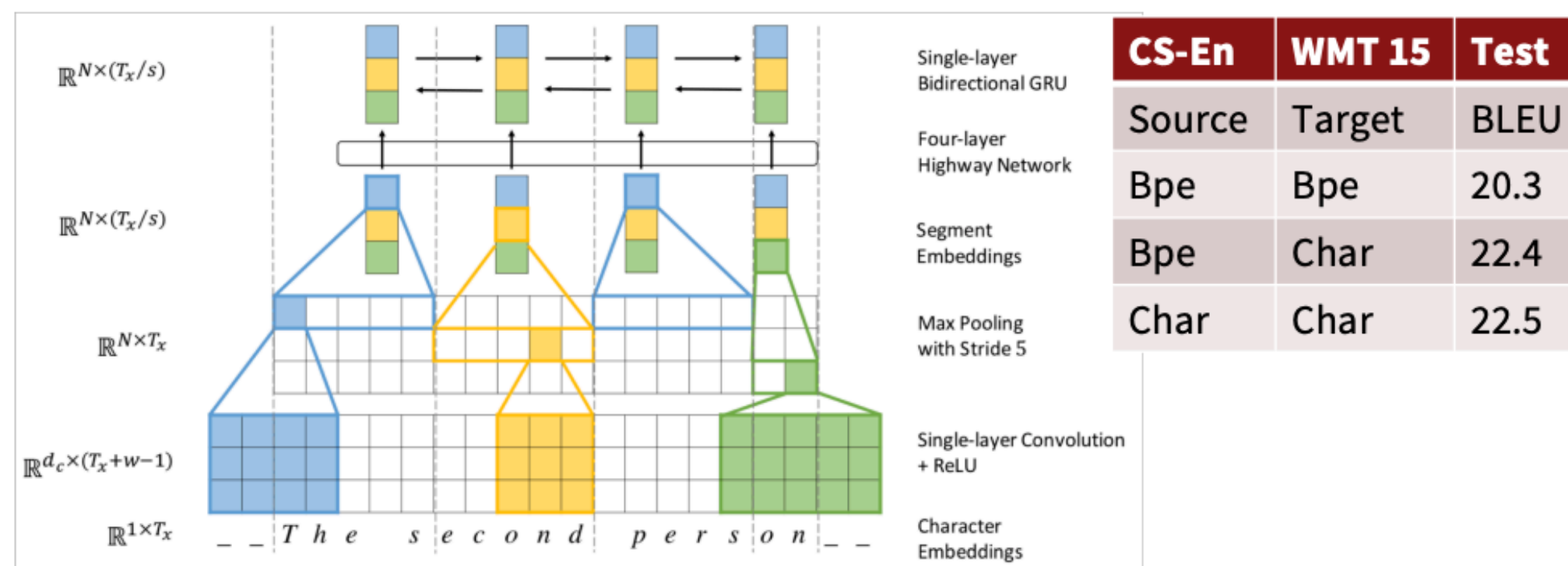
- Out of vocabulary (OOV)
- Low-resource languages
- Long-term context
- Common sense knowledge (e.g. hot dog, paper jam)
- Fairness and bias
- Interpretability

Out of vocabulary (OOV)

- Subword-modeling
 - Character level GRU
 - Byte-pair encoding

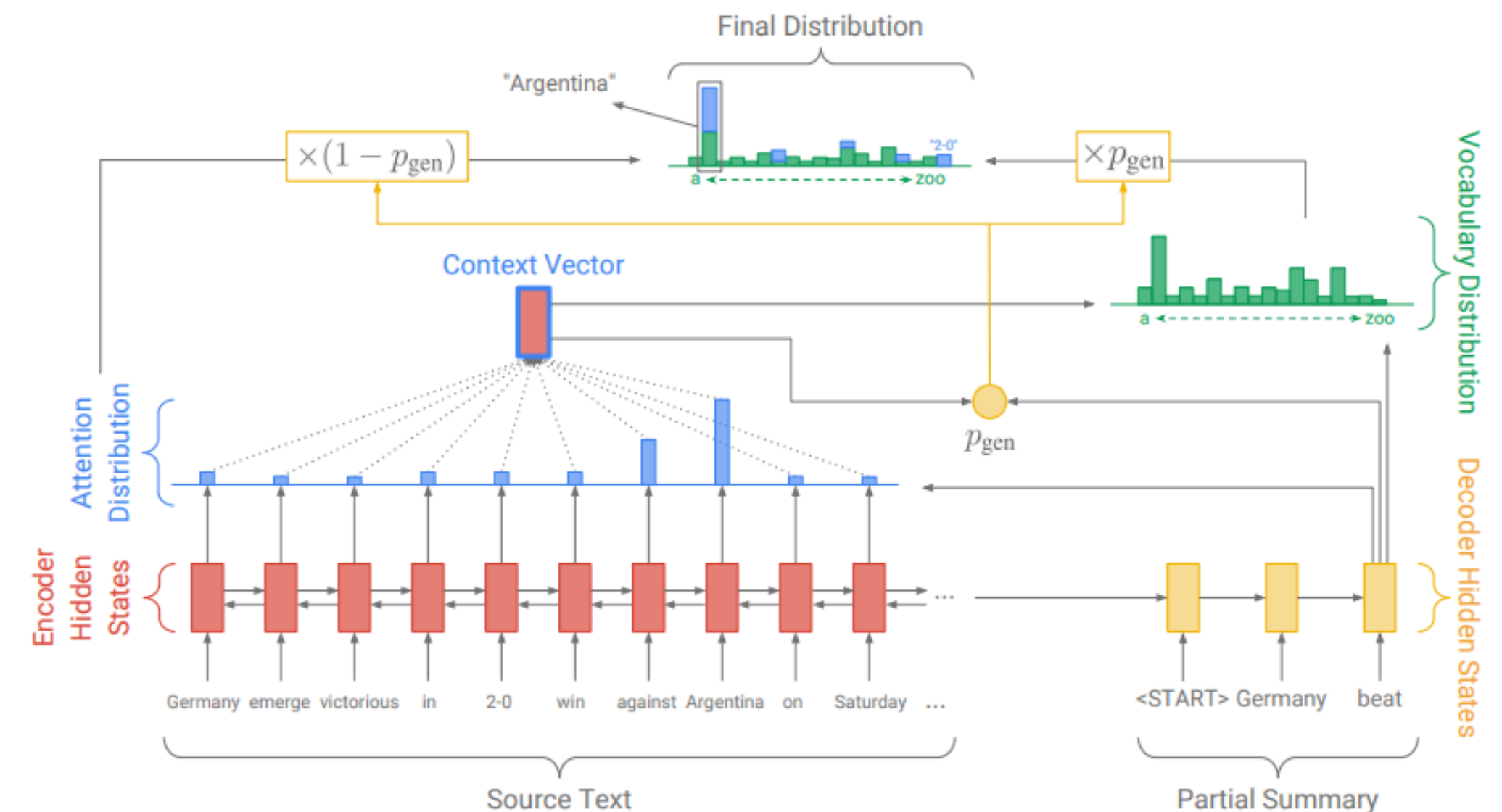
Fully Character-Level Neural Machine Translation without Explicit Segmentation

Jason Lee, Kyunghyun Cho, Thomas Hoffmann. 2017.
Encoder as below; decoder is a char-level GRU



(Lee et al, 2017)

- Copy mechanism



- Probability of generating from vocabulary or copying from input
- Probability of copying specific word (similar to attention)

(See et al, 2017)

